
전자정부 SW 개발 · 운영자를 위한

표준프레임워크

보안 개발 가이드

2024. 02

NIA 한국지능정보사회진흥원

표준프레임워크센터

▪ <제목 차례>	
▪ 제 1 장 개요	1
▪ 제1절 배경	1
▪ 제2절 가이드의 목적과 구성	1
▪ 제 2 장 보안 개발 가이드	3
▪ 제1절 입력 데이터 검증 및 표현	3
▪ 1. SQL Injection	3
▪ 2. 코드 삽입	7
▪ 3. 경로 조작 및 자원 삽입	10
▪ 4. XSS (Cross Site Scripting, XSS)	15
▪ 5. 운영체제 명령어 삽입	20
▪ 6. 위험한 형식 파일 업로드	22
▪ 7. 신뢰되지 않는 URL 주소로 자동접속 연결	25
▪ 8. 부적절한 XML 외부개체 참조	29
▪ 9. XML 삽입	32
▪ 10. LDAP 삽입 (Lightweight Directory Access Protocol)	37
▪ 11. 크로스사이트 요청 위조 (Cross-Site Request Forgery)	40
▪ 12. 서버사이드 요청 위조	44
▪ 13. HTTP 응답분할	47
▪ 14. 정수형 오버플로우	49
▪ 15. 보안기능 결정에 사용되는 부적절한 입력값	51
▪ 16. 포맷 스트링 삽입	54
▪ 제2절 보안기능	56
▪ 1. 적절한 인증 없는 중요기능 허용	56
▪ 2. 부적절한 인가	58
▪ 3. 중요한 자원에 대한 잘못된 권한 설정	60
▪ 4. 취약한 암호화 알고리즘 사용	62
▪ 5. 암호화되지 않은 중요정보	67
▪ 6. 하드코딩된 중요정보	72
▪ 7. 충분하지 않은 키 길이 사용	77
▪ 8. 적절하지 않은 난수값 사용	79
▪ 9. 취약한 비밀번호 허용	83
▪ 10. 부적절한 전자서명 확인	85
▪ 11. 부적절한 인증서 유효성 검증	87
▪ 12. 사용자 하드디스크에 저장되는 쿠키를 통한 정보노출	89
▪ 13. 주석문 안에 포함된 시스템 주요정보	92
▪ 14. 솔트 없이 일방향 해쉬함수 사용	94
▪ 15. 무결성 검사 없는 코드 다운로드	97
▪ 16. 반복된 인증시도 제한 기능 부재	99
▪ 제3절 시간 및 상태	101
▪ 1. 경쟁조건: 검사시점과 사용시점(TOCTOU Race Condition)	101
▪ 2. 종료되지 않는 반복문 또는 재귀함수	106
▪ 제4절 에러 처리	108
▪ 1. 오류 메시지를 통한 정보노출	108

▪ 2. 오류 상황 대응 부재	110
▪ 3. 부적절한 예외 처리	112
▪ 제5절 코드 오류	114
▪ 1. Null Pointer 역참조	114
▪ 2. 부적절한 자원 해제	117
▪ 3. 신뢰할 수 없는 데이터의 역직렬화	119
▪ 제6절 캡슐화	123
▪ 1. 잘못된 세션에 의한 데이터 정보노출	123
▪ 2. 제거되지 않고 남은 디버그 코드	126
▪ 3. Public 메소드부터 반환된 Private 배열	128
▪ 4. Private 배열에 Public 데이터 할당	131
▪ 제7절 API 오용	133
▪ 1. DNS lookup에 의존한 보안결정	133
▪ 2. 취약한 API 사용	135
▪	

제 1 장 개요

제1절 배경

전자정부 표준프레임워크는 공공사업에 적용되는 개발프레임워크의 표준 정립으로 응용 소프트웨어 표준화, 품질 및 재사용성 향상을 위해 꾸준히 발전해 왔다. 또한 전자정부 서비스는 클라우드를 기반으로 5G, AI, IoT 등 신기술 분야로 발전해 나가고 있다. 이러한 급변하는 기술 환경 속에서 안전한 소프트웨어의 중요성은 아무리 강조해도 지나치지 않다.

이에 ‘행정기관 및 공공기관 정보시스템 구축·운영 지침’에 따라 정보화 사업 수행 시 안전한 SW 개발을 위한 시큐어코딩 기법으로 행정안전부는 ‘소프트웨어 개발 보안 가이드’, ‘소프트웨어 보안약점 진단가이드’를 배포하고 있다.

‘소프트웨어 개발 보안 가이드’에서는 개발언어별로 안전하지 않은 코드와 안전한 코드의 예제를 제시하여 개발 실무에 활용하도록 되어 있다. 하지만, 전자정부 표준프레임워크를 적용하는 공공사업의 정보시스템이 많아지면서 표준프레임워크를 적용하는 정보시스템만을 위한 보안 개발 가이드의 필요성이 대두되었다.

제2절 가이드의 목적과 구성

본 가이드는 표준프레임워크를 적용하는 정보시스템을 위한 개발보안 가이드로 한국인터넷진흥원의 ‘소프트웨어 개발 보안 가이드’를 기반으로 작성되었다. 개발자들이 실무에서 전자정부 표준프레임워크를 사용하여 개발하면서 전자정부 표준프레임워크 센터로 문의한 시큐어코딩 관련한 질문과 그동안 보안패치했던 내용을 정리하여 포함시켰다. 또한 ‘소프트웨어 개발보안 가이드’의 예제 중 표준프레임워크에 관련된 예제만 포함시켰으

며, ‘소프트웨어 개발 보안 가이드’에 포함되어 있지 않은 보안 이슈는 추가하였다.

개발자는 전자정부 표준프레임워크 기반 시큐어코딩을 적용한 소프트웨어 개발 시 본 가이드를 참조할 수 있으며, 발주자는 소프트웨어 보안약점 진단 및 제거 요구사항 도출 시 활용할 수 있다. 아키텍처 수립 또는 설계 단계에서의 시큐어코딩을 위한 상세한 설명은 한국인터넷진흥원의 ‘소프트웨어 개발 보안 가이드’를 참고하면 된다.

유형	내용
입력데이터 검증 및 표현	프로그램 입력값에 대한 검증 누락 또는 부적절한 검증, 데이터의 잘못된 형식지정, 일관되지 않은 언어셋 사용 등으로 인해 발생하는 보안약점
보안기능	보안기능(인증, 접근제어, 기밀성, 암호화, 권한관리 등)을 부적절하게 구현 시 발생할 수 있는 보안약점
시간 및 상태	동시 또는 거의 동시 수행을 지원하는 병렬 시스템이나 하나 이상의 프로세스가 동작되는 환경에서 시간 및 상태를 부적절하게 관리하여 발생할 수 있는 보안약점
에러처리	에러를 처리하지 않거나, 불충분하게 처리하여 에러 정보에 중요정보(시스템 내부정보 등)가 포함될 때 발생할 수 있는 보안약점
코드오류	타입 변환 오류, 자원(메모리 등)의 부적절한 반환 등과 같이 개발자가 범할 수 있는 코딩오류로 인해 유발되는 보안약점
캡슐화	중요한 데이터 또는 기능성을 불충분하게 캡슐화하였을 때, 인가되지 않은 사용자에게 데이터 누출이 가능해지는 보안약점
API 오용	의도된 사용에 반하는 방법으로 API를 사용하거나, 보안에 취약한 API를 사용하여 발생할 수 있는 보안약점

표준프레임워크 예제 포함된 보안 항목, 개발환경의 보안 항목별 관련 툴 보유 여부, 표준프레임워크 4.2 PMD 룰 설명 등을 붙임으로 첨부하였다.

제 2 장 보안 개발 가이드

제1절 입력 데이터 검증 및 표현

프로그램 입력값에 대한 검증 누락 또는 부적절한 검증, 데이터의 잘못된 형식지정, 일관되지 않은 언어셋 사용 등으로 인해 발생하는 보안약점으로 SQL Injection, 크로스사이트 스크립트 (XSS) 등의 공격을 유발할 수 있다.

1. SQL Injection



가. 개요

- SQL Injection은 악성 코드가 삽입된 요청 문자열이 데이터베이스 인스턴스에 전달되게 함으로써, 구문분석 및 실행될 때 공격자가 질의문의 의미를 왜곡시키거나 그 구조를 변경하여 임의의 데이터베이스 명령어를 수행하는 보안약점을 말한다.
- 취약한 웹 응용프로그램에서는 일정한 기본 쿼리 문자열과 매개변수로 입력 받은 사용자 문자열을 연결하여 동적 쿼리 (Dynamic Query)¹⁾로 구성되기 때문에 개발자가 의도하지 않은 쿼리가 생성되어 정보유출에 악용될 수 있다.

나. 보안대책

- 사용자로부터 입력받은 값을 SQL 문의 연결 문자열로 사용

1) 동적쿼리(Dynamic Query) : DB에서 실시간으로 받는 쿼리. Parameterized Statement가 동적 쿼리가 됨

되지 않도록 하기 위하여, Prepared Statement²⁾객체 등을 이용하여 DB에 컴파일 된 쿼리문에 인자값으로 처리하도록 하는 방법을 사용 한다. 하지만, Prepared Statement를 사용 하더라도 사용자로부터 입력받은 값을 인자값으로 사용하지 않고 SQL command에 연결되는 문자열로 사용된다면 SQL Injection에 취약하다. 따라서, 외부입력 값은 SQL의 인자값으로만 사용하도록 하여야 한다.

- MyBatis Data Map 파일의 인자를 받는 쿼리문 정의 시 SQL문에 문자열을 직접 주입하게 되는 \${} 구문을 사용하지 않고, Prepared Statement 의 인자값으로 안전하게 설정하게 하는 #{} 구문을 사용한다.

다. 코드예제

- MyBatis에서 외부에서 입력되는 값으로 \${} 구문을 사용하면, SQL 쿼리문의 인자값으로 사용되지 않고, SQL command 에 연결하는 문자열로 사용되어 의도하지 않은 정보가 노출될 수 있다.

안전하지 않은 코드의 예 MyBatis

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
.....
<select id="boardSearch" parameterType="map" resultType="BoardDto">
//$기호를 사용하는 경우 외부에서 입력된 keyword값을 문자열에 결합한 형태로 쿼리에
반영되므로 안전하지 않다.
select * from tbl_board where title like '%${keyword}%' order by pos asc
</select>
```

- 외부 입력값을 문자열에 결합하는 형태로 쿼리에 반영하지 않고, 인자값으로 바인딩하려면 #{} 사용한다.

안전한 코드의 예 MyBatis

2) Prepared Statement : 컴파일된 쿼리 객체로 MySQL, Oracle, DB2, SQL Server 등에서 지원하며, Java의 JDBC, Perl의 DBI, PHP의 PDO, ASP의 ADO를 이용하여 사용가능

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
.....
<select id="boardSearch" parameterType="map" resultType="BoardDto">
//$ 대신 #기호를 사용하여 변수가 쿼리맵에 바인딩 될 수 있도록 수정하는 것이 안전하다.
select * from tbl_board where title like '%'||#{keyword}||'%' order by pos asc
</select>
```

- Hibernate의 경우 기본으로 PreparedStatement를 사용하지만, 아래 코드와 같이 파라미터 바인딩 (binding) 없이 사용할 경우 외부로부터 입력받은 값에 의해 쿼리의 구조가 변경 될 수 있다.

안전하지 않은 코드의 예 Hibernate

```
import org.hibernate.Query
import org.hibernate.Session
.....
//외부로부터 입력받은 값을 검증 없이 사용할 경우 안전하지 않다.
String name = request.getParameter("name");
//Hibernate는 기본으로 PreparedStatement를 사용하지만, 파라미터 바인딩 없이 사용 할 경우
안전하지 않다.
Query query = session.createQuery("from Student where studentName = " + name + " ");
```

- 아래 코드와 같이 외부 입력값이 위치하는 부분을 ? 또는 : 명명된 파라미터 변수로 설정하고, 실행 시에 해당 파라미터가 전달되는 바인딩(binding)을 함으로써 외부의 입력이 쿼리의 구조를 변경 시키는 것을 방지할 수 있다.

안전한 코드의 예 Hibernate

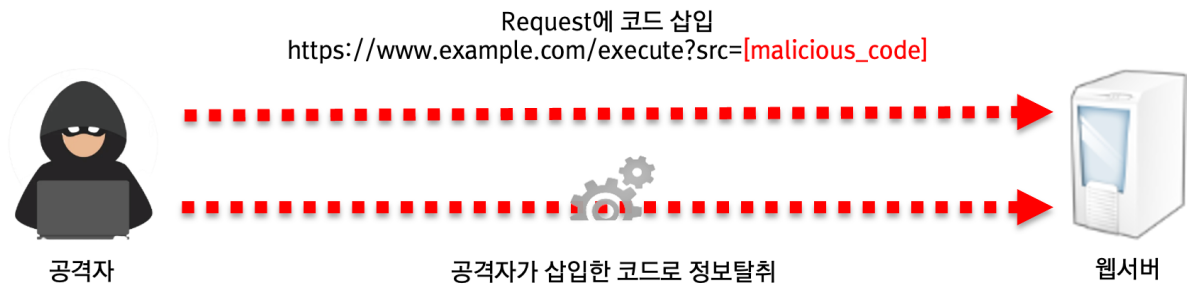
```
import org.hibernate.Query
import org.hibernate.Session
.....
String name = request.getParameter("name");
//1. 파라미터 바인딩을 위해 ?를 사용한다.
Query query = session.createQuery("from Student where studentName = ? ");
//2. 파라미터 바인딩을 사용하여 외부 입력값에 의해 쿼리 구조 변경을 못하게 사용하였다.
query.setString(0, name);
```

라. 참고자료

- CWE-89 SQL Injection, MITRE,
<https://cwe.mitre.org/data/definitions/89.html>
- Threat and Vulnerability, "SQL Injection", Microsoft,
<http://technet.microsoft.com/en-us/library/ms161953%28v=SQL.105%29.aspx>
- Input validation and Data Sanitization, Threat and Vulnerability, Prevent SQL Injection, CERT,
<https://wiki.sei.cmu.edu/confluence/display/java/IDS00-J.+Prevent+SQL+injection>
- SQL Injection Prevention Cheat Sheet, OWASP³⁾
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

3) OWASP(Open Web Application Security Project) : 웹 응용 및 소프트웨어 보안 문제를 해결하기 위해 전세계 전문가들로 구성된 비영리 재단. 3년에 1회 'OWASP Top 10'이라는 웹 응용 취약점 위험에 대해서 발표한다.

2. 코드 삽입



가. 개요

- 공격자가 소프트웨어의 의도된 동작을 변경하도록 임의 코드를 삽입하여 소프트웨어가 비정상적으로 동작하도록 하는 보안약점을 말한다. 코드 삽입은 프로그래밍 언어 자체의 기능에 의해서만 제한된다는 점에서 운영체제 명령어 삽입과 다르다.
- 취약한 프로그램에서 사용자의 입력 값에 코드가 포함되는 것을 허용할 경우, 공격자는 개발자가 의도하지 않은 코드를 실행하여 권한을 탈취하거나 인증 우회, 시스템 명령 실행 등을 할 수 있다.

나. 보안대책

동적코드를 실행할 수 있는 함수를 사용하지 않는다. 실행 가능한 동적코드를 입력 값으로 받지 않도록, 외부 입력 값에 대하여 화이트리스트 방식으로 구현한다. 또는 유효한 문자만 포함되도록 동적 코드에 사용되는 사용자 입력 값을 필터링한다.

다. 코드예제

- 다음 예제의 소스코드는 `javax.script.ScriptEngineManager`를 사용하여 `ScriptEngineManager()`로 사용자의 입력을 실행하여 출력한다. 이 경우, 공격자는 조작된 인수를 입력한

공격코드를 이용하여 새로운 파일을 만들거나 덮어씌울 수 있다.

안전하지 않은 코드의 예

```
public class CodeInjectionController {
    @RequestMapping(value = "/execute", method = RequestMethod.GET)
    public String execute(@RequestParam("src") String src) throws ScriptException {
        ScriptEngineManager scriptEngineManager = new ScriptEngineManager();
        ScriptEngine scriptEngine = scriptEngineManager.getEngineByName("javascript");
        // 외부 입력값인 src를 javascript eval 함수로 실행하고 있어 안전하지 않다.
        String retValue = (String)scriptEngine.eval(src);
        return retValue;
    }
}
```

- 다음 예제는 외부 입력 값을 javascript의 new Function()으로 동적으로 코드를 실행할 수 있다.

안전하지 않은 코드의 예

```
<body>
  <%
    String name = request.getParameter("name");
  %>
  ...
  <script>
    // 외부 입력값인 name을 javascript new Function()을 이용하여 문자열을 함수로 실행하고
    있다.
    (new Function(<%=name%>))();
  </script>
</body>
```

- 외부 입력값에 실행이 가능한 코드가 포함되어 있을 경우 입력 값을 필터링하여 사전에 검증하는 코드를 추가하면 코드삽입을 완화할 수 있다. 이런 조치를 취할 경우 입력 값의 형태에 따라 정규표현식을 변형하여 적용해야 한다.

안전한 코드의 예

```
@RequestMapping(value = "/execute", method = RequestMethod.GET)
public String execute(@RequestParam("src") String src) throws ScriptException {
    // 정규식을 이용하여 특수문자 입력시 예외를 발생시킨다.
    if (src.matches("[ \\w]*") == false) {
```

```

        throw new IllegalArgumentException();
    }
    ScriptEngineManager scriptEngineManager = new ScriptEngineManager();
    ScriptEngine scriptEngine = scriptEngineManager.getEngineByName("javascript");
}

```

- 스크립트 실행이 필요한 경우는 화이트리스트 방식을 적용하여 유효한 문자인 경우에만 실행되도록 하고 그 외의 경우는 모두 예외 처리한다.

안전한 코드의 예

```

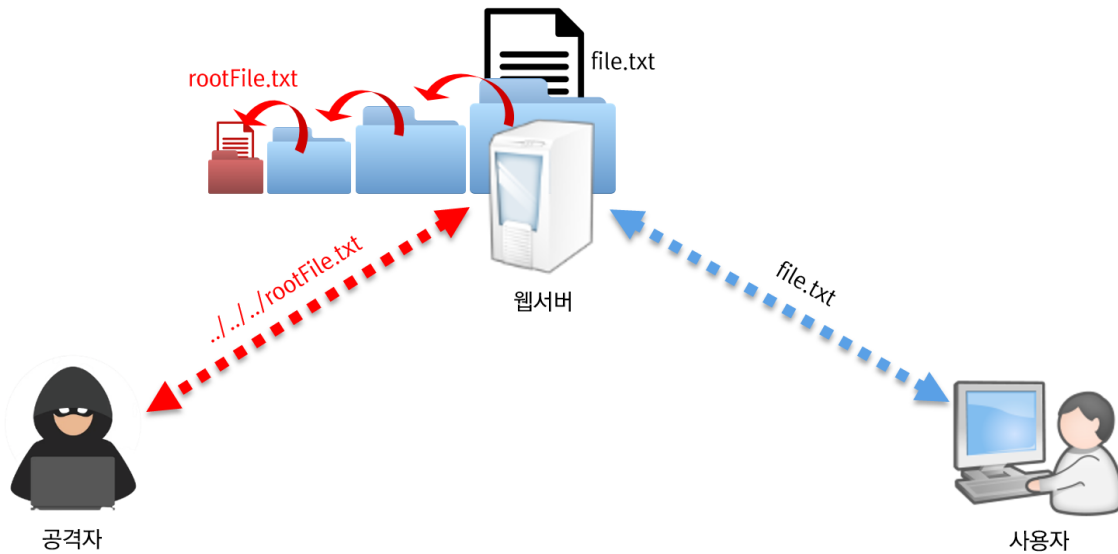
@RequestMapping(value = "/execute", method = RequestMethod.GET)
public String execute(@RequestParam("src") String src) throws ScriptException {
    // 유효한 문자 "_" 일 경우 실행할 메소드 호출한다.
    if (src.matches("UNDER_BAR") == true) {
        ...
        // 유효한 문자 "$" 일 경우 실행할 메소드 호출한다.
    } else if (src.matches("DOLLAR") == true) {
        ...
        // 유효하지 않은 특수문자 입력시 예외를 발생시킨다.
    } else {
        throw new IllegalArgumentException();
    } ...
}

```

라. 참고자료

- CWE-94: Improper Control of Generation of Code ('Code Injection') , MITRE, <http://cwe.mitre.org/data/definitions/94.html>
- CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection') , MITRE, <http://cwe.mitre.org/data/definitions/95.html>
- Code Injection Software Attack, OWASP, https://owasp.org/www-community/attacks/Code_Injection

3. 경로 조작 및 자원 삽입



가. 개요

- 검증되지 않은 외부 입력값을 통해 파일 및 서버 등 시스템 자원에 대한 접근 혹은 식별을 허용할 경우, 입력값 조작을 통해 시스템이 보호하는 자원에 임의로 접근할 수 있는 보안약점이다. 경로조작 및 자원삽입 약점을 이용하여 공격자는 자원의 수정·삭제, 시스템 정보누출, 시스템 자원 간 충돌로 인한 서비스 장애 등을 유발시킬 수 있다.
- 즉, 경로 조작 및 자원 삽입을 통해서 공격자가 허용되지 않은 권한을 획득하여, 설정에 관계된 파일을 변경하거나 실행시킬 수 있다.

나. 보안대책

외부의 입력을 자원(파일, 소켓의 포트 등)의 식별자로 사용하는 경우, 적절한 검증을 거치도록 하거나, 사전에 정의된 적합한 리스트에서 선택되도록 한다. 특히, 외부의 입력이 파일명인 경우에는 경로순회(directory traversal)⁴⁾ 공격의 위험이 있는 문자(`" / ₩ ..` 등)를 제거할 수 있는 필터를 이용

4) 경로순회 : `"/, "..` 등을 사용하여 상대경로 참조 방식을 이용해 웹 루트 디렉토리를 벗어나 다른 디렉토리의 중요파일에 접근하는 공격법. 경로추적이라고도 함

한다.

다. 코드예제

- 외부 입력값(P)이 버퍼로 내용을 옮길 파일의 경로설정에 사용되고 있다. 만일 공격자에 의해 P의 값으로 ../../../rootFile.txt와 같은 값을 전달하면 의도하지 않았던 파일의 내용이 버퍼에 쓰여 시스템에 악영향을 준다.

안전하지 않은 코드의 예

```
//외부로부터 입력받은 값을 검증 없이 사용할 경우 안전하지 않다.  
String fileName = request.getParameter("P");  
BufferedInputStream bis = null;  
BufferedOutputStream bos = null;  
FileInputStream fis = null;  
try {  
    response.setHeader("Content-Disposition", "attachment;filename="+fileName+");  
    ...  
    //외부로부터 입력받은 값이 검증 또는 처리 없이 파일처리에 수행되었다.  
    fis = new FileInputStream("C:/datas/" + fileName);  
    bis = new BufferedInputStream(fis);  
    bos = new BufferedOutputStream(response.getOutputStream());
```

- 외부 입력값에 대하여 상대경로를 설정할 수 없도록 경로순회 문자열(/ ₩ & .. 등)을 제거하고 파일의 경로설정에 사용한다.

안전한 코드의 예

```
String fileName = request.getParameter("P");  
BufferedInputStream bis = null;  
BufferedOutputStream bos = null;  
FileInputStream fis = null;  
try {  
    response.setHeader("Content-Disposition", "attachment;filename="+fileName+");  
    ...  
    // 외부 입력받은 값을 경로순회 문자열(/₩)을 제거하고 사용해야한다.  
    filename = filename.replaceAll("₩.", "").replaceAll("/", "").replaceAll("₩₩₩₩", "");  
    fis = new FileInputStream("C:/datas/" + fileName);  
    bis = new BufferedInputStream(fis);  
    bos = new BufferedOutputStream(response.getOutputStream());  
    int read;  
    while((read = bis.read(buffer, 0, 1024)) != -1) {  
        bos.write(buffer,0,read);  
    }  
}
```


치 후 사용하도록 한다.

안전한 코드의 예 공통컴포넌트 경로조작 확인

```
@RequestMapping(value="/EgovPageLink.do")
public String moveToPage(@RequestParam("link") String link){

    // 안전한 경로 문자열로 조치
    link = EgovWebUtil.filePathBlackList(link);

    return link;
}
```

- XML파일에 허용된 경로만 허용하도록 화이트리스트를 적용할 수 있다.
- /src/main/resources/egovframework/spring/com/ 에 위치한 빈 설정 파일을 생성한 후 경로 파라미터가 화이트리스트에 포함되도록 한 후, 메소드에 전달된 경로 파라미터가 화이트리스트에 포함되어 있는지 확인하는 코드를 추가한다.

안전한 코드의 예 공통컴포넌트 경로 화이트리스트

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
.....

<util:list id="egovPageLinkWhitelist" value-type="java.lang.String">
    <value>/egovframework/com/sym/mnu/stm/EgovSiteMap</value>
    <value>/cmm/sym/mpm/EgovSiteMap</value>
    <value>/egovframework/com/main_bottom</value>
</util:list>

.....
```

안전한 코드의 예 공통컴포넌트 경로 확인

```
@Resource(name = "egovPageLinkWhitelist")
protected List<String> egovWhitelist;

@RequestMapping(value="/EgovPageLink.do")
public String moveToPage(@RequestParam("link") String linkPage){

    // 안전한 경로 문자열로 조치
    link = EgovWebUtil.filePathBlackList(link);

    if (egovWhitelist.contains(linkPage) == false) {
        LOGGER.debug("Page Link WhiteList Error! Please check whitelist!");
    }
}
```

```

        link="egovframework/com/cmm/egovError";
    }

    return link;

```

- 화이트리스트를 DB에서 가져오는 방식을 적용할 시에는 메소드에 전달된 경로 파라미터가 DB에 저장된 화이트리스트에 포함되어 있는지 확인하는 코드를 추가한다.

안전한 코드의 예 공통컴포넌트 경로 확인 (DB)

```

@RequestMapping("/cop/cmy/previewCmmntyMainPage.do")
public String previewCmmntyMainPage(@ModelAttribute("searchVO") CommunityVO cmmntyVO,
    ModelMap model) throws Exception {

    // 안전한 경로 문자열로 조치
    tmplatCours = EgovWebUtil.filePathBlackList(tmplatCours);

    // 화이트 리스트 체크
    List<TemplateInfVO> templateWhiteList =
    egovTemplateManageService.selectTemplateWhiteList();
    LOGGER.debug("Template > WhiteList Count = {}", templateWhiteList.size());
    if ( tmplatCours == null ) tmplatCours = "";
    for(TemplateInfVO templateInfVO : templateWhiteList){
        LOGGER.debug("Template > whiteList TmplatCours =
        "+templateInfVO.getTmplatCours());
        if ( tmplatCours.equals(templateInfVO.getTmplatCours()) ) {
            return tmplatCours;
        }
    }

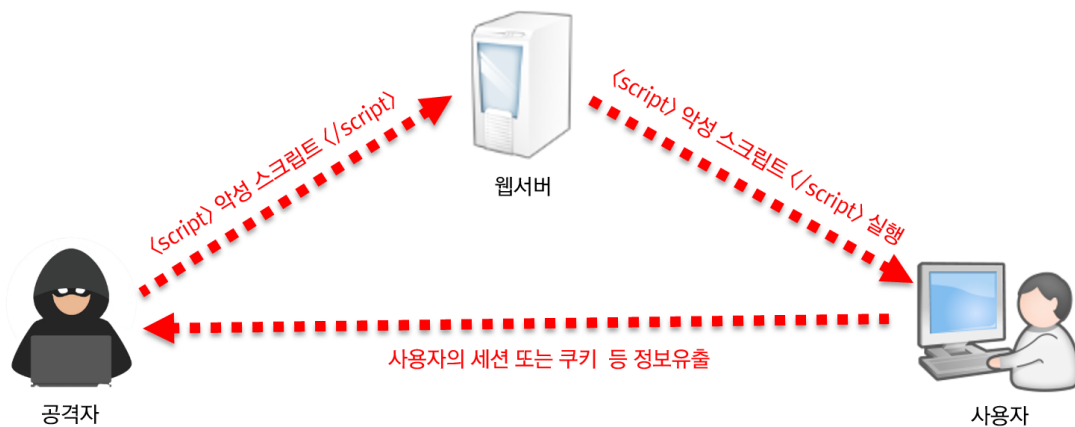
    LOGGER.debug("Template > WhiteList mismatch! Please check Admin page!");
    return "egovframework/com/cmm/egovError";
}

```

라. 참고자료

- CWE-99 Resource Injection, MITRE,
<https://cwe.mitre.org/data/definitions/99.html>
- CWE-22 Path Traversal, MITRE,
<https://cwe.mitre.org/data/definitions/22.html>
- Path Traversal, OWASP,
https://owasp.org/www-community/attacks/Path_Traversal

4. XSS (Cross Site Scripting, XSS)



가. 개요

검증되지 않은 외부 입력이 동적 웹페이지 생성에 사용될 경우, 웹 페이지에 악의적인 스크립트를 포함시켜 사용자로 하여금 그 웹페이지를 열람하게 한 후 사용자 측에서 악성 스크립트가 실행되도록 하는 공격 방법이다.

나. 보안대책

- 외부 입·출력값에 대해서 검증하고 무효화하기 위해 위험한 문자 (태그 <, >) 등을 문자 입력 시 문자 참조로 필터링하고, 서버에서 브라우저로 전송 시 문자를 인코딩한다. 아래 참조 문자로 변경하면 브라우저는 일반 문자로 인식하여 스크립트처럼 실행되지는 않는다.

ASCII 문자	참조 문자	ASCII 문자	참조 문자
&	&	“	"
<	<	‘	'
>	>	/	/
(	(	)	)

- JSTL 또는 잘 알려진 크로스 사이트 스크립트 방지 라이브

러리를 활용한다.

- HTML 태그를 허용하는 게시판에서는 허용되는 HTML 태그들을 화이트리스트로 만들어 해당 태그만 지원하도록 한다. (공통컴포넌트 v3.8 이상 egovframework.com.cmm.filter 패키지의 HTMLTagFilterRequestWrapper class 참조)

다. 코드예제

- 크로스사이트 스크립트(XSS)는 크게 3가지 공격 방법이 존재한다.
- Reflected XSS 공격은 검색 결과, 에러 메시지 등을 통해 서버가 외부에서 입력받은 악성 스크립트가 포함된 URL 파라미터 값을 사용자 브라우저에서 응답할 때 발생한다. 공격 스크립트가 삽입된 URL을 사용자가 쉽게 확인할 수 없도록 변형하여, 이메일, 메신저, 파일등을 통해 실행을 유도하는 공격이다.
- Stored XSS 공격은 웹 사이트의 게시판, 코멘트 필드, 사용자 프로필 등의 입력 form을 통해 악성 스크립트를 삽입하여 DB에 저장되면, 사용자가 사이트를 방문하여 저장되어 있는 페이지에 정보를 요청할 때, 서버는 악성 스크립트를 사용자에게 전달하여 사용자 브라우저에서 스크립트가 실행되면서 공격한다.
- DOM기반 XSS 공격은 외부에서 입력받은 악성 스크립트가 포함된 URL 파라미터 값이 서버를 거치지 않고, DOM 생성의 일부로 실행되면서 공격한다. Reflected XSS 및 Stored XSS 공격은 서버 애플리케이션 취약점으로 인해, 응답 페이지에 악성 스크립트가 포함되어 브라우저로 전달되면서 공격하는 것인 반면, DOM기반 XSS는 서버와 관계없이 발생하는 것이 차이점이다.

안전하지 않은 코드의 예

```
<% String keyword = request.getParameter("keyword"); %>
//외부 입력값에 대하여 검증 없이 화면에 출력될 경우 공격스크립트가 포함된 URL을 생성 할 수
있어 안전하지 않다.(Reflected XSS)
검색어 : <%=keyword%>

//게시판 등의 입력form을 통해 외부값이 DB에 저장되고, 이를 검증 없이 화면에 출력될 경우
공격스크립트가 실행되어 안전하지 않다.(Stored XSS)
검색결과 : ${m.content}

<script type="text/javascript">
//외부 입력값에 대하여 검증 없이 브라우저에서 실행되는 경우 서버를 거치지 않는
공격스크립트가 포함된 URL을 생성 할 수 있어 안전하지 않다. (DOM 기반 XSS)
document.write("keyword:" + <%=keyword%>);
</script>
```

- 외부 입력값 파라미터나 게시판등의 form에 의해 서버의 처리 결과를 사용자 화면에 출력하는 경우, 입력값에 대해서 문자열 치환 함수를 이용하여 스크립트 문자열을 제거하거나, JSTL을 이용하여 출력하거나, 잘 만들어진 외부 XSS 방지 라이브러리를 활용하는 것이 안전하다.
- 크로스사이트 스크립트의 경우 동작 상황에 따라 동일한 조치방법을 사용하면, 크로스사이트 스크립트 방지는 되더라도 원하는 동작이 정상적으로 되지 않을 수 있기 때문에, 잘 만들어진 외부 XSS방지 라이브러리를 이용하여 각 동작 상황에 따라 적절하게 사용하는 것을 권장한다.

안전한 코드의 예

```
<% String keyword = request.getParameter("keyword"); %>
// 방법1. 입력값에 대하여 스크립트 공격가능성이 있는 문자열을 치환한다.
keyword = keyword.replaceAll("&", "&amp;");
keyword = keyword.replaceAll("<", "&lt;");
keyword = keyword.replaceAll(">", "&gt;");
keyword = keyword.replaceAll(""", "&quot;");
keyword = keyword.replaceAll("'", "&#x27;");
keyword = keyword.replaceAll("/", "&#x2F;");
keyword = keyword.replaceAll("(", "&#x28;");
keyword = keyword.replaceAll(")", "&#x29;");
검색어 : <%=keyword%>

//방법2. JSP에서 출력값에 JSTL c:out 을 사용하여 처리한다.
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
검색결과 : <c:out value="${m.content}"/> 또는 <c:out value="${m.content}" escapeXml="true"
```



```
/>  
  
<script type="text/javascript">  
//방법3. 잘 만들어진 외부 라이브러리를 활용(NAVER Lucy-XSS-Filter, OWASP ESAPI, OWASP  
Java-Encoder-Project)  
document.write("keyword:" +  
<%=Encoder.encodeForJS(Encoder.encodeForHTML(keyword))%>);  
</script>
```

라. 참고자료

- CWE-79 Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting'), MITRE,
<https://cwe.mitre.org/data/definitions/79.html>
- Properly encode or escape output, CERT,
<https://wiki.sei.cmu.edu/confluence/display/java/IDS51-J.+Properly+encode+or+escape+output>
- XSS (Cross Site Scripting) Prevention Cheat Sheet, OWASP,
<https://owasp.org/www-community/xss-filter-evasion-cheat-sheet>
- DOM based XSS Prevention Cheat Sheet, OWASP,
<https://owasp.org/www-community/attacks/xss/>
- Understanding Malicious Content Mitigation for Web Developers"
http://www.cert.org/tech_tips/malicious_code_mitigation.html
- 크로스 사이트 스크립팅 공격 종류 및 대응 방법, 성윤기,
<https://www.kisa.or.kr/uploadfile/201312/201312161355109566.pdf>
- Lucy-xss-servlet-filter (naver-oss)
<https://github.com/naver/lucy-xss-servlet-filter>

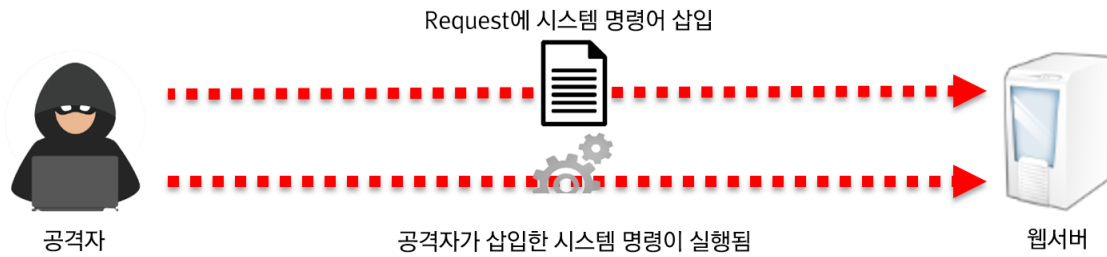
- 표준프레임워크 HtmlTagFilter 설정

<https://www.egovframe.go.kr/wiki/doku.php?id=egovframework:let:configuration>

- 표준프레임워크 HTMLTagFilterRequestWrapper 참조

<https://github.com/eGovFramework/egovframe-common-components/blob/master/src/main/java/egovframework/com/cmm/filter/HTMLTagFilterRequestWrapper.java>

5. 운영체제 명령어 삽입



가. 개요

- 적절한 검증절차를 거치지 않은 사용자 입력 값이 운영체제 명령어의 일부 또는 전부로 구성되어 실행되는 경우, 의도하지 않은 시스템 명령어가 실행되어 부적절하게 권한이 변경되거나 시스템 동작 및 운영에 악영향을 미칠 수 있다.
- 운영체제 명령어 삽입 공격은 웹 어플리케이션에서 시스템 명령을 호출하는 코드가 포함되어 있고, 사용자 입력이 시스템 호출에 사용되는 경우에만 가능하다.

나. 보안대책

웹 인터페이스를 통해 서버 내부로 시스템 명령어를 전달시키지 않도록 응용프로그램을 구성하고, 외부에서 전달되는 값을 검증 없이 시스템 내부 명령어로 사용하지 않는다. 외부 입력에 따라 명령어를 생성하거나 선택이 필요한 경우에는 명령어 생성에 필요한 값 들을 미리 지정해 놓고 외부 입력에 따라 선택하여 사용한다.

다. 코드예제

- `Runtime.getRuntime().exec()` 명령어는 시스템 명령어를 수행하라는 코드이다. 아래 코드는 외부 입력 값을 검증하지 않고 그대로 명령어로 실행하기 때문에 공격자의 입력에 따라 의도하지 않은 명령어가 실행될 수 있다.

안전하지 않은 코드의 예

```
//외부로 부터 입력 받은 값을 검증 없이 사용할 경우 안전하지 않다.  
String date = request.getParameter("date");  
String command = new String("cmd.exe /c backuplog.bat");  
Runtime.getRuntime().exec(command + date);
```

- 운영체제 명령어 실행 시에는 아래와 같이 외부에서 들어오는 값에 의하여 멀티라인을 지원하는 특수 문자(| ; & : *)나 파일 리다이렉트 특수문자(> >>) 또는 공백등을 제거하여 원하지 않은 운영체제 명령어가 실행 될 수 없도록 필터링을 수행한다.

안전한 코드의 예

```
String date = request.getParameter("date");  
String command = new String("cmd.exe /c backuplog.bat");  
//외부로부터 입력 받은 값을 필터링을 통해 우회문자를 제거하여 사용한다.  
date = date.replaceAll("\\|", ""); date = date.replaceAll(";", "");  
date = date.replaceAll("&", ""); date = date.replaceAll(":", "");  
date = date.replaceAll(">", ""); date = date.replaceAll("\\*", "");  
date = date.replaceAll("\\p{Space}", "");  
Runtime.getRuntime().exec(command + date);
```

라. 참고자료

- CWE-78 OS Command Injection, MITRE,
<https://cwe.mitre.org/data/definitions/78.html>
- Sanitize untrusted data passed to the Runtime.exec() method, CERT,
<https://wiki.sei.cmu.edu/confluence/display/java/IDS07-J.+Sanitize+untrusted+data+passed+to+the+Runtime.exec%28%29+method>
- Do not call system(), CERT,
<http://www.securecoding.cert.org/confluence/pages/viewpage.action?pageId=2130132>
- Reviewing Code for OS Injection, OWASP
https://owasp.org/www-pdf-archive/OWASP_Code_Review_Guide-V1_1.pdf

6. 위험한 형식 파일 업로드



가. 개요

서버 측에서 실행될 수 있는 스크립트 파일(jsp 파일 등)이 업로드 가능하고, 이 파일을 공격자가 웹을 통해 직접 실행시킬 수 있는 경우, 시스템 내부명령어를 실행하거나 외부와 연결하여 시스템을 제어할 수 있는 보안약점이다.

나. 보안대책

화이트 리스트 방식으로 허용된 확장자만 업로드를 허용한다. 업로드 되는 파일을 저장할 때에는 파일명과 확장자를 외부사용자가 추측할 수 없는 문자열로 변경하여 저장하며, 저장 경로는 'web document root' 밖에 위치시켜서 공격자의 웹을 통한 직접 접근을 차단한다. 또한 파일 실행여부를 설정할 수 있는 경우, 실행 속성을 제거한다.

다. 코드예제

- 업로드할 파일에 대한 유효성을 검사하지 않으면, 위험한 유형의 파일을 공격자가 업로드하거나 전송 할 수 있다.

안전하지 않은 코드의 예

```
MultipartRequest multi
= new MultipartRequest(request,savePath,sizeLimit,"euc-kr",new
DefaultFileRenamePolicy());
.....
//업로드 되는 파일명을 검증없이 사용하고 있어 안전하지 않다.
String fileName = multi.getFilesystemName("filename");
```

```

.....
sql = " INSERT INTO
board(email,r_num,w_date,pwd,content,re_step,re_num,filename) "
+ " values ( ?, 0, sysdate(), ?, ?, ?, ?, ? ) ";
preparedStatement pstmt = con.prepareStatement(sql);
pstmt.setString(1, stemail);
pstmt.setString(2, stpwd);
pstmt.setString(3, stcontent);
pstmt.setString(4, stre_step);
pstmt.setString(5, stre_num);
pstmt.setString(6, fileName);
pstmt.executeUpdate();
Thumbnail.create(savePath+"/"+fileName, savePath+"/"+s_+fileName, 150);

```

- 아래 코드는 업로드 파일의 확장자를 검사하여 허용되지 않은 확장자인 경우 업로드를 제한하고 있다.

안전한 코드의 예

```

MultipartRequest multi
= new MultipartRequest(request,savePath,sizeLimit,"euc-kr",new DefaultFileRenamePolicy());
.....
String fileName = multi.getFileName("filename");
if (fileName != null) {
//1.업로드 파일의 마지막 "." 문자열의 기준으로 실제 확장자 여부를 확인하고, 대소문자 구별을
해야한다.
String fileExt =
FileName.substring(fileName.lastIndexOf(".")+1).toLowerCase();
//2.되도록 화이트 리스트 방식으로 허용되는 확장자로 업로드를 제한해야 안전하다.
if (!"gif".equals(fileExt) && !"jpg".equals(fileExt) && !"png".equals(fileExt))
{
    alertMessage("업로드 불가능한 파일입니다.");
    return;
}
}
.....
sql = " INSERT INTO
board(email,r_num,w_date,pwd,content,re_step,re_num,filename) "
+ " values ( ?, 0, sysdate(), ?, ?, ?, ?, ? ) ";
PreparedStatement pstmt = con.prepareStatement(sql);
.....
Thumbnail.create(savePath+"/"+fileName, savePath+"/"+s_+fileName, 150);

```

라. 참고자료

- CWE-434 Unrestricted Upload of File with Dangerous Type, MITRE,
<https://cwe.mitre.org/data/definitions/434.html>
- Prevent arbitrary file upload, CERT,

<https://wiki.sei.cmu.edu/confluence/display/java/IDS56-J.+Prevent+arbitrary+file+upload>

7. 신뢰되지 않는 URL 주소로 자동접속 연결



가. 개요

- 사용자로부터 입력되는 URL로 요청을 Redirect 할 수 있는 웹 어플리케이션은 악의적인 사이트의 주소로 자동으로 연결되는 피싱 공격에 노출되는 취약점을 가질 수 있다.
- 일반적으로 클라이언트에서 전송된 URL 주소로 연결하기 때문에 안전하다고 생각할 수 있으나, 공격자는 해당 품의 요청을 변조함으로써 사용자가 위험한 URL로 접속할 수 있도록 공격할 수 있다.

나. 보안대책

자동 연결할 외부 사이트의 URL과 도메인은 화이트 리스트로 관리하고, 사용자 입력값을 자동 연결할 사이트 주소로 사용하는 경우에는 입력된 값이 화이트 리스트에 존재하는지 확인해야 한다.

다. 코드예제

- 다음과 같은 코드가 서버에 존재할 경우 공격자는 아래와 같은 링크를 통해 희생자가 피싱 사이트 등으로 접근하도록 할 수 있다.
- (예시 링크) `<a href="http://bank.example.com/redirect?url=http://attacker.example.net">Click</a>`

안전하지 않은 코드의 예

```
String id = (String)session.getValue("id");
String bn = request.getParameter("gubun");
//외부로부터 입력받은 URL이 검증없이 다른 사이트로 이동이 가능하여 안전하지 않다.
String rd = request.getParameter("redirect");
if (id.length() > 0) {
    String sql = "select level from customer where customer_id = ? ";
    conn = db.getConnection();
    pstmt = conn.prepareStatement(sql);
    pstmt.setString(1, id);
    rs = pstmt.executeQuery();
    rs.next();
    if ("0".equals(rs.getString(1)) && "01AD".equals(bn)) {
        response.sendRedirect(rd);
    }
    return;
}
```

- 다음의 예제와 같이, 외부로 연결할 URL과 도메인들은 화이트 리스트를 작성한 후, 그 중에서 선택하도록 함으로써 안전하지 않은 사이트로의 접근을 차단할 수 있다.

안전한 코드의 예

```
//이동 할 수 있는 URL범위를 제한하여 피싱 사이트등으로 이동 못하도록 한다.
String allowedUri[] = { "/main.do", "/login.jsp", "list.do" };
.....
String rd = request.getParameter("redirect");
try {
    rd = allowedUri[Integer.parseInt(rd)];
} catch(NumberFormatException e) {
    return "잘못된 접근입니다.";
} catch(ArrayIndexOutOfBoundsException e) {
    return "잘못된 입력입니다.";
}
}
if (id.length() > 0) {
    .....
    if ("0".equals(rs.getString(1)) && "01AD".equals(bn)) {
        response.sendRedirect(rd);
    }
    return;
}
```

- XML파일에 허용된 경로만 허용하도록 화이트리스트를 적용할 수 있다.
- /src/main/resources/egovframework/spring/com/ 에 위치한 빈 설정 파일을 생성한 후 경로 파라미터가 화이트리스트에 포함되도록 한 후, 메소드에 전달된 경로 파라미터가

화이트리스트에 포함되어 있는지 확인하는 코드를 추가한다.

안전한 코드의 예 공통컴포넌트 경로 화이트리스트

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
.....

<util:list id="egovPageLinkWhitelist" value-type="java.lang.String">
    <value>/egovframework/com/sym/mnu/stm/EgovSiteMap</value>
    <value>/cmm/sym/mpm/EgovSiteMap</value>
    <value>/egovframework/com/main_bottom</value>
</util:list>

.....
```

안전한 코드의 예 공통컴포넌트 경로 확인

```
@Resource(name = "egovPageLinkWhitelist")
protected List<String> egovWhitelist;

@RequestMapping(value="/EgovPageLink.do")
public String moveToPage(@RequestParam("link") String linkPage){

    // 안전한 경로 문자열로 조치
    link = EgovWebUtil.filePathBlackList(link);

    if (egovWhitelist.contains(linkPage) == false) {
        LOGGER.debug("Page Link WhiteList Error! Please check whitelist!");
        link="/egovframework/com/cmm/egovError";
    }

    return link;
}
```

- 화이트리스트를 DB에서 가져오는 방식을 적용할 시에는 메소드에 전달된 경로 파라미터가 DB에 저장된 화이트리스트에 포함되어 있는지 확인하는 코드를 추가한다.

안전한 코드의 예 공통컴포넌트 경로 확인 (DB)

```
@RequestMapping("/cop/cmy/previewCmmntyMainPage.do")
public String previewCmmntyMainPage(@ModelAttribute("searchVO") CommunityVO cmmntyVO,
ModelMap model) throws Exception {

    // 안전한 경로 문자열로 조치
    tmplatCours = EgovWebUtil.filePathBlackList(tmplatCours);

    // 화이트 리스트 체크
    List<TemplateInfVO> templateWhiteList =
egovTemplateManageService.selectTemplateWhiteList();
    LOGGER.debug("Template > WhiteList Count = {}",templateWhiteList.size());
}
```

```

        if ( tmplatCours == null ) tmplatCours = "";
        for(TemplateInfVO templateInfVO : templateWhiteList){
            LOGGER.debug("Template > whiteList TmplatCours =
"+templateInfVO.getTmplatCours());
            if ( tmplatCours.equals(templateInfVO.getTmplatCours()) ) {
                return tmplatCours;
            }
        }

        LOGGER.debug("Template > WhiteList mismatch! Please check Admin page!");
        return "egovframework/com/cmm/egovError";
    }
}

```

라. 참고자료

- CWE-601 URL Redirection to Untrusted Site, MITRE,
<https://cwe.mitre.org/data/definitions/601.html>
- Unvalidated Redirects and Forwards Cheat Sheet, OWASP
https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html

8. 부적절한 XML 외부개체 참조

가. 개요

XML 문서에는 DTD(Document Type Definition)를 포함할 수 있으며, DTD는 XML 엔티티(entity)를 정의한다. 부적절한 XML 외부개체 참조 보안약점은 서버에서 XML 외부 엔티티를 처리할 수 있도록 설정된 경우에 발생할 수 있다. 취약한 XML parser가 외부 값을 참조하는 XML 값을 처리할 때, 공격자가 삽입한 공격 구문이 동작되어 서버 파일 접근, 불필요한 자원 사용, 인증 우회, 정보 노출 등이 발생할 수 있다.

나. 보안대책

로컬 정적 DTD를 사용하도록 설정하고, 외부에서 전송된 XML 문서에 포함된 DTD를 완전하게 비활성화해야 한다. 비활성화를 할 수 없는 경우에는 외부 엔티티 및 외부 문서 유형 선언을 각파서에 맞는 고유한 방식으로 비활성화 한다.

다. 코드예제

- 다음의 예제는 XML 소스를 읽어서 분석하는 소스코드이다. 공격자가 아래와 같이 XML 외부 엔티티를 참조하는 receivedXML 데이터를 전송하고, 이를 파싱할 때 /etc/passwd 파일을 참조할 수 있다.

안전하지 않은 코드의 예
<pre>receivedXML <?xml version="1.0" encoding="ISO-8859-1"?> <!DOCTYPE foo [<!ELEMENT foo ANY > <ENTITY xxe SYSTEM "file:///etc/passwd" >]><foo>&xxe;</foo> public void unmarshal(File receivedXml) throws JAXBException, ParserConfigurationException, SAXException, IOException { JAXBContext jaxbContext = JAXBContext.newInstance(Student.class); Unmarshaller jaxbUnmarshaller = jaxbContext.createUnmarshaller(); // 입력받은 receivedXml 을 이용하여 Document를 생성한다. DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance(); dbf.setNamespaceAware(true); DocumentBuilder db = dbf.newDocumentBuilder();</pre>

```

Document document = db.parse(receivedXml);
// 외부 엔티티로 만들어진 document를 이용하여 마샬링을 수행하여 안전하지 않다.
Student employee = (Student) jaxbUnmarshaller.unmarshal( document );
}

```

- 다음의 예제는 class XXE에서 외부개체 참조 제한 설정 없이 secure.xml을 참조하고 있다. 이 때, secure.xml이 아래와 같을 때 /dev/tty 콘솔이 실행되어 입력 요청을 대기하는 서비스 거부 공격이 발생할 수 있다.

안전하지 않은 코드의 예

```

secure.xml
<?xml version="1.0"?>
<!DOCTYPE foo SYSTEM "file:/dev/tty">
<foo>bar</foo>

import javax.xml.parsers.SAXParsers;
import javax.xml.parsers.SAXParserFactory;
class XXE {
    public static void main(String[] args)
        throws FileNotFoundException, ParserConfigurationException, SAXException, IOException {
        SAXParserFactory factory = SAXParserFactory.newInstance();
        SAXParser saxParser = factory.newSAXParser();
        // 외부개체 참조 제한 설정 없이 secure.xml 파일을 읽어서 파싱하여 안전하지 않다.
        saxParser.parse(new FileInputStream("secure.xml"), new DefaultHandler());
    }
}

```

- 다음의 JAXP DocumentBuilderFactory를 사용하는 경우 아래와 같이 제한 설정을 추가할 수 있다.

안전한 코드의 예

```

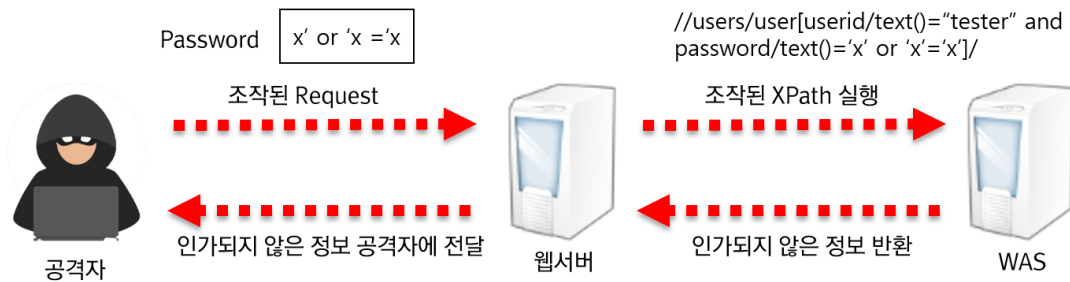
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
// XML 파서가 doctype을 정의하지 못하도록 설정한다.
dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
// 외부 일반 엔티티를 포함하지 않도록 설정한다.
dbf.setFeature("http://xml.org/sax/features/external-general-entities", false);
// 외부 파라미터도 포함하지 않도록 설정한다.
dbf.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
// 외부 DTD 비활성화한다.
dbf.setFeature("http://apache.org/xml/features/nonvalidating/load-external-dtd", false);
// XInclude를 사용하지 않는다.
dbf.setXIncludeAware(false);
// 생성된 파서가 엔티티 참조 노드를 확장하지 않도록 한다.
dbf.setExpandEntityReferences(false);
DocumentBuilder db = dbf.newDocumentBuilder();
Document document = db.parse(receivedXml);
Model model = (Model) u.unmarshal(document);

```

라. 참고자료

- CWE-611: Improper Restriction of XML External Entity Reference, MITRE, <https://cwe.mitre.org/data/definitions/611.html>
- XML Entity Prevention Cheat Sheet, OWASP, https://cheatsheetseries.owasp.org/cheatsheets/XML_External_Entity_Prevention_Cheat_Sheet.html

9. XML 삽입



가. 개요

검증되지 않은 외부 입력 값이 XQuery 또는 XPath 쿼리문을 생성하는 문자열로 사용되어 공격자가 쿼리문의 구조로 임의로 변경하고 임의의 쿼리를 실행하여 허가되지 않은 데이터를 열람하거나 인증절차를 우회할 수 있는 보안약점이다.

나. 보안대책

XQuery 또는 XPath 쿼리에 사용되는 외부 입력데이터에 대하여 특수문자 및 쿼리 예약어를 필터링하고 파라미터 (Parameter)화된 쿼리문을 지원하는 XQuery를 사용한다.

다. 코드예제

● XQuery 삽입

- 다음의 예제에서는 executeQuery를 통해 생성하는 쿼리의 파라미터의 일부로 외부입력값 (name)을 사용하고 있다. 만일 something' or '1'='1 을 name의 값으로 전달하면 다음과 같은 쿼리문을 수행할 수 있으며, 이를 통해 파일 내의 모든 값을 출력할 수 있게 된다.

- (예시) `doc('users.xml')/userlist/user[uname='something' or '1'='1']`

안전하지 않은 코드의 예

// 외부 입력 값을 검증하지 않고 XQuery 표현식에 사용한다.

```
String name = props.getProperty("name");
```

```
.....
```

```
// 외부 입력 값에 의해 쿼리 구조가 변경 되어 안전하지 않다.
String es = "doc('users.xml')/userlist/user[uname='"+name+"']";
XQPreparedExpression expr = conn.prepareExpression(es);
XQResultSequence result = expr.executeQuery();
```

- 다음의 예제에서는 외부 입력값을 받고 해당 값 기반의 XQuery상의 쿼리 구조를 변경시키지 않는 bindString 함수를 이용함으로써 외부 입력값을 통해 쿼리 구조가 변경될 수 없도록 한다.

안전한 코드의 예

```
// bindString 함수로 쿼리 구조가 변경되는 것을 방지한다.
String name = props.getProperty("name");
.....
String es = "doc('users.xml')/userlist/user[uname='$xname']";
XQPreparedExpression expr = conn.prepareExpression(es);
expr.bindString(new QName("xname"), name, null);
XQResultSequence result = expr.executeQuery();
```

● XPath 삽입

- 아래 예제에서는 nm과 pw에 대한 입력값 검증을 수행하지 않으므로 nm의 값으로 "tester", pw의 값으로 "x' or 'x'='x"을 전달하면 아래와 같은 쿼리문이 생성되어 인증과정을 거치지 않고 로그인할 수 있다.
- (예시) "//users/user[login/text()='tester' and password/text()='x' or 'x'='x']/home_dir/text()"

안전하지 않은 코드의 예

```
// 프로퍼티로부터 외부 입력값 name과 password를 읽어와 각각 nm, pw변수에 저장
String nm = props.getProperty("name");
String pw = props.getProperty("password");
.....
XPathFactory factory = XPathFactory.newInstance();
XPath xpath = factory.newXPath();
.....
// 검증되지 않은 입력값 외부 입력값 nm, pw 를 사용하여 안전하지 않은 질의문이 작성되어 expr 변수에 저장된다.
XPathExpression expr = xpath.compile("//users/user[login/text()='"+nm+" and password/text()='"+pw+"']/home_dir/text()");
// 안전하지 않은 질의문이 담긴 expr을 평가하여 결과를 result에 저장한다.
Object result = expr.evaluate(doc, XPathConstants.NODESET);
// result의 결과를 NodeList 타입으로 변환하여 nodes 저장한다.
NodeList nodes = (NodeList) result;
for (int i=0; i<nodes.getLength(); i++) {
```



```
String value = nodes.item(i).getNodeValue();
if (value.indexOf(">") < 0) {
    // 공격자가 이름과 패스워드를 확인할 수 있다.
    System.out.println(value);
}
}
```

- 다음의 예제는 XQuery를 사용하여 미리 쿼리 골격을 생성함으로써 외부입력으로 인해 쿼리 구조가 바뀌는 것을 막을 수 있다.

```
[ login.xq 파일 ]
declare variable $loginID as xs:string external; declare variable $password as xs:string
external;
//users/user[@loginID=$loginID and @password=$password]
// XQuery를 이용한 XPath Injection 방지
String nm = props.getProperty("name");
String pw = props.getProperty("password");
Document doc = new Builder().build("users.xml");
// 파라미터화된 쿼리가 담겨있는 login.xq를 읽어와서 파라미터화된 쿼리를 생성한다.
XQuery xquery = new XQueryFactory().createXQuery(new File("login.xq"));
Map vars = new HashMap();
// 검증되지 않은 외부값인 nm, pw를 파라미터화된 쿼리의 파라미터로 설정한다.
vars.put("loginID", nm);
vars.put("password", pw);
// 파라미터화된 쿼리를 실행하므로 외부값을 검증없이 사용하더라도 안전하다.
Nodes results = xquery.execute(doc, null, vars).toNodes();
for (int i=0; i<results.size(); i++) {
    System.out.println(results.get(i).toXML());
}
```

- 파라미터화된 쿼리를 지원하는 XQuery 구문으로 대체할 수 없는 경우에는 XPath 삽입을 유발할 수 있는 문자들을 입력 값에서 제거하고 XPath 구문을 생성, 실행하도록 한다.

안전한 코드의 예

```
// XPath 삽입을 유발할 수 있는 문자들을 입력값에서 제거
public String XPathFilter(String input) {
    if (input != null) return input.replaceAll("[',\"\\W\"]", "");
    else return "";
}

.....
// 외부 입력값에 사용
String nm = XPathFilter(props.getProperty("name"));
String pw = XPathFilter(props.getProperty("password"));

.....
```

```

XPathFactory factory = XPathFactory.newInstance();
XPath xpath = factory.newXPath();
.....
//외부 입력값인 nm, pw를 검증하여 쿼리문을 생성하므로 안전하다.
XPathExpression expr = xpath.compile("//users/user[login/text()='"+nm+"' and
password/text()='"+pw+']/home_dir/text()");
Object result = expr.evaluate(doc, XPathConstants.NODESET);
NodeList nodes = (NodeList) result;
for (int i=0; i<nodes.getLength(); i++) {
    String value = nodes.item(i).getNodeValue();
    if (value.indexOf(">") < 0) {
        System.out.println(value);
    }
}
}
.....

```

- 아래 코드는 입력값을 검증하지 않고 입력값을 XPath 구문 생성 및 실행에 사용하고 있다. 입력 값으로 any' or 'a' = 'a' 와 같이 XPath 구문을 조작하는 문자열을 전달하는 경우 //food[name ='any' or 'a' = 'a']/price 같이 구문이 만들어 지고 실행되어 모든 food의 가격(price)가 조회되게 된다.

안전하지 않은 코드의 예

```

public static void main(String[] args) throws Exception {
    if (args.length <= 0) {
        System.err.println("가격을 검색할 식품의 이름을 입력하세요."); return;
    }
    String name = args[0];
    DocumentBuilder docBuilder = DocumentBuilderFactory.newInstance().newDocumentBuilder();
    Document doc = docBuilder.parse("http://www.w3schools.com/xml/simple.xml");
    Xpath 템소 = XPathFactory.newInstance().newXPath();
    // 프로그램의 커맨드 옵션으로 입력되는 외부값 name을 사용하여 쿼리문을 직접
    // 작성하여 수행하므로 안전하지 않다.
    NodeList nodes = (NodeList) 템소.evaluate("//food[name='"+ name + "']/price", doc,
    XPathConstants.NODESET);
    for (int i = 0; i < nodes.getLength(); i++) {
        System.out.println(nodes.item(i).getTextContent());
    }
}
}

```

- 외부 입력값을 XPath 구문 생성 및 실행에 사용하는 경우 XPath 구문을 조작할 수 있는 문자열을 제거하고 사용할 수 있도록 한다.

안전한 코드의 예

```

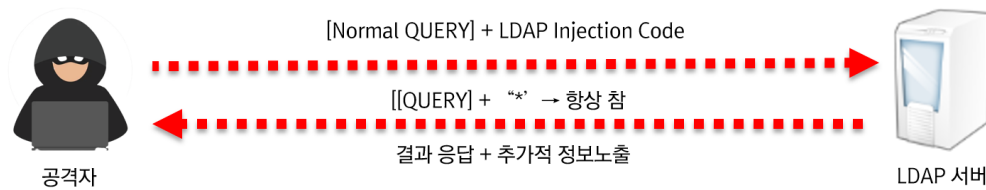
public static void main(String[] args) throws Exception {
    if (args.length <= 0) {
        System.err.println("가격을 검색할 식품의 이름을 입력하세요.");
        return;
    }
    //프로그램의 커맨드 옵션으로 입력되는 외부값 name에서 XPath 구문을 조작할 수
    // 는 문자를 제거하는 검증을 수행하여 안전하다.
    String name = args[0];
    if (name != null) {
        name = name.replaceAll("[()\\W\\-\\'\\W\\[\\W\\]:,*/]", "");
    }
    DocumentBuilder docBuilder =
        DocumentBuilderFactory.newInstance().newDocumentBuilder();
    Document doc = docBuilder.parse("http://www.w3schools.com/xml/simple.xml");
    XPath xpath = XPathFactory.newInstance().newXPath();
    NodeList nodes = (NodeList) xpath.evaluate("//food[name='" + name + "']/price",
        doc, XPathConstants.NODESET);
    for (int i = 0; i < nodes.getLength(); i++) {
        System.out.println(nodes.item(i).getTextContent());
    }
}

```

라. 참고자료

- CWE-652 XQuery Injection, MITRE,
<https://cwe.mitre.org/data/definitions/652.html>
- Prevent XML Injection, CERT,
<https://wiki.sei.cmu.edu/confluence/display/java/IDS16-J.+Prevent+XML+Injection>
- CWE-643 XPath Injection, MITRE,
<https://cwe.mitre.org/data/definitions/643.html>
- Prevent XPath Injection, CERT,
<https://www.securecoding.cert.org/confluence/display/java/IDS53-J.+Prevent+XPath+Injection>
- XPATH Injection, OWASP,
https://www.owasp.org/index.php/XPATH_Injection

10. LDAP 삽입 (Lightweight Directory Access Protocol)



가. 개요

- 공격자가 외부 입력을 통해서 의도하지 않은 LDAP 명령어를 수행할 수 있다. 즉, 웹 응용프로그램이 사용자가 제공한 입력을 올바르게 처리하지 못하면, 공격자가 LDAP 명령문의 구성을 바꿀 수 있다. 이로 인해 프로세스가 명령을 실행한 컴포넌트와 동일한 권한(Permission)을 가지고 동작하게 된다.
- 외부 입력값을 적절한 처리 없이 LDAP 쿼리문이나 결과의 일부로 사용하는 경우, LDAP 쿼리문이 실행될 때 공격자는 LDAP 쿼리문의 내용을 마음대로 변경할 수 있다.

나. 보안대책

DN(Distinguished Name)과 필터에 사용되는 사용자 입력값에는 특수문자가 포함되지 않도록 특수문자를 제거한다. 만약 특수문자를 사용해야 하는 경우에는 특수문자(= + < > # ; 와 등)가 실행명령이 아닌 일반문자로 인식되도록 처리한다.

다. 코드예제

- userSN과 userPassword변수의 값으로 *을 전달할 경우 필터 문자열은 "(&(sn=S*)(userPassword=*))" 가 되어 항상 참이 되며 이는 의도하지 않은 동작을 유발시킬 수 있다.

안전하지 않은 코드의 예

```
private void searchRecord(String userSN, String userPassword) throws
```

```

NamingException {
    Hashtable<String, String> env = new Hashtable<String, String>();
    env.put(Context.INITIAL_CONTEXT_FACTORY, "com.sun.jndi.LdapCtxFactory");
    try {
        DirContext dctx = new InitialDirContext(env);
        SearchControls sc = new SearchControls();
        String[] attributeFilter = { "cn", "mail" };
        sc.setReturningAttributes(attributeFilter);
        sc.setSearchScope(SearchControls.SUBTREE_SCOPE);
        String base = "dc=example,dc=com";
        //userSN과 userPassword 값에 LDAP필터를 조작할 수 있는 공격 문자열에 대한 검증이 없어
        안전하지 않다.
        String filter = "(&(sn=" + userSN + ")(userPassword=" + userPassword + "))";
        NamingEnumeration<?> results = dctx.search(base, filter, sc);
        while (results.hasMore()) {
            SearchResult sr = (SearchResult) results.next();
            Attributes attrs = sr.getAttributes();
            Attribute attr = attrs.get("cn");
            .....
        }
        dctx.close();
    } catch (NamingException e) { ... }
}

```

- 검색을 위한 필터 문자열로 사용되는 외부의 입력에서 위험한 문자열을 제거하여 위험성을 부분적으로 감소시킬 수 있다.

안전한 코드의 예

```

private void searchRecord(String userSN, String userPassword) throws
NamingException {
    Hashtable<String, String> env = new Hashtable<String, String>();
    env.put(Context.INITIAL_CONTEXT_FACTORY, "com.sun.jndi.LdapCtxFactory");
    try {
        DirContext dctx = new InitialDirContext(env);
        SearchControls sc = new SearchControls();
        String[] attributeFilter = { "cn", "mail" };
        sc.setReturningAttributes(attributeFilter);
        sc.setSearchScope(SearchControls.SUBTREE_SCOPE);
        String base = "dc=example,dc=com";
        // userSN과 userPassword 값에서 LDAP 필터를 조작할 수 있는 문자열을 제거하고 사용
        if (!userSN.matches("[WwWwWs]*") || !userPassword.matches("[WwW]*")) {
            throw new IllegalArgumentException("Invalid input");
        }
        String filter = "(&(sn=" + userSN + ")(userPassword=" + userPassword + "))";
        NamingEnumeration<?> results = dctx.search(base, filter, sc);
        while (results.hasMore()) {
            SearchResult sr = (SearchResult) results.next();
            Attributes attrs = sr.getAttributes();
            Attribute attr = attrs.get("cn");
            .....
        }
        dctx.close();
    }
}

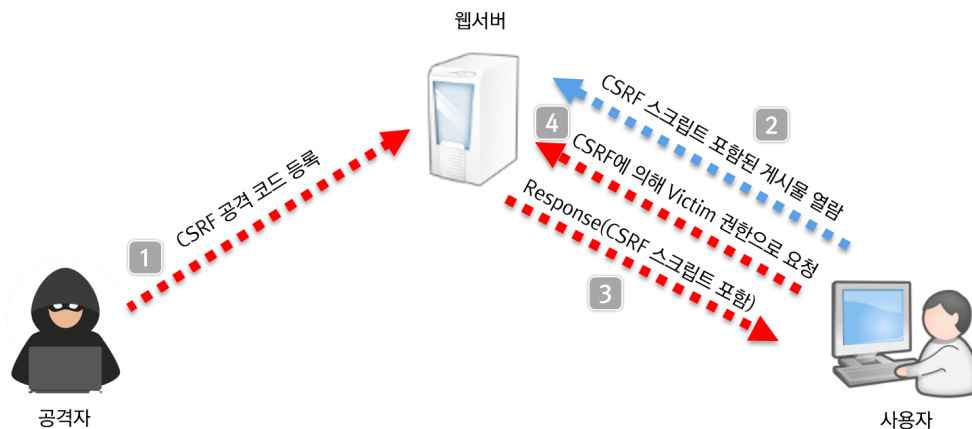
```

```
} catch (NamingException e) { ... }  
}
```

라. 참고자료

- CWE-90 LDAP Injection, MITRE,
<https://cwe.mitre.org/data/definitions/90.html>
- Prevent LDAP injection, CERT,
<https://wiki.sei.cmu.edu/confluence/display/java/IDS54-J.+Prevent+LDAP+injection>
- LDAP injection, OWASP
https://cheatsheetseries.owasp.org/cheatsheets/LDAP_Injection_Prevention_Cheat_Sheet.html
- "LDAP Resources" <http://ldapman.org/>

11. 크로스사이트 요청 위조 (Cross-Site Request Forgery)



가. 개요

- 특정 웹사이트에 대해서 사용자가 인지하지 못한 상황에서 사용자의 의도와는 무관하게 공격자가 의도한 행위(수정, 삭제, 등록 등)를 요청하게 하는 공격을 말한다. 웹 응용프로그램이 사용자로부터 받은 요청에 대해서 사용자가 의도한 대로 작성되고 전송된 것인지 확인하지 않는 경우 발생 가능 하고 특히 해당 사용자가 관리자인 경우 사용자 권한관리, 게시물삭제, 사용자 등록 등 관리자 권한으로만 수행 가능한 기능을 공격자의 의도대로 실행시킬 수 있게 된다.
- 공격자는 사용자가 인증한 세션이 특정 동작을 수행하여도 계속 유지되어 정상적인 요청과 비정상 적인 요청을 구분하지 못하는 점을 악용한다. 웹 응용프로그램에 요청을 전달할 경우, 해당 요청의 적법성을 입증하기 위하여 전달되는 값이 고정되어 있고 이러한 자료가 GET 방식으로 전달된다면 공격자가 이를 쉽게 알아내어 원하는 요청을 보냄으로써 위험한 작업을 요청할 수 있게 된다.

나. 보안대책

입력화면 폼 작성 시 GET 방식보다는 POST 방식을 사용하고 입력화면 폼과 해당 입력을 처리하는 프로그램 사이에

토큰을 사용하여, 공격자의 직접적인 URL 사용이 동작하지 않도록 처리한다. 특히 중요한 기능에 대해서는 사용자 세션 검증과 더불어 재인증을 유도한다.

다. 코드예제

- 사용자가 취약한 사이트(example.com)에서 로그아웃하지 않은 상태에서 악의적인 사이트를 방문하여 취약한 사이트를 가리키고 있는 HTML FORM의 action 어트리뷰트를 실행하게 함으로 CSRF 공격에 쉽게 노출될 수 있다. 다음은 악의적인 사이트의 코드로 취약한 사이트(example.com) 사용자로 하여금 submit 하도록 유도한 예이다. 이와 같은 공격은 악의적인 사이트 뿐 아니라 취약한 사이트(example.com) 내 CSRF 스크립트가 포함된 게시물을 통해서도 이루어질 수 있다.

공격자의 CSRF 공격 코드의 예

```
<h1>You Are a Winner!</h1>
<form action="http://example.com/api/account" method="post">
  <input type="hidden" name="Transaction" value="withdraw" />
  <input type="hidden" name="Amount" value="1000000" />
  <input type="submit" value="Click Me"/>
</form>
```

- 정상 요청 여부를 판단하기 위해 토큰을 이용한다. 사용자가 입력(신청) 페이지를 요청하면 두 가지 토큰을 포함시켜 반환한다. 그 중 하나의 토큰은 세션에 저장하고, 다른 토큰은 HIDDEN 필드 항목의 값으로 설정한다. 이 토큰들은 악의적인 사용자들이 값을 추측할 수 없도록 무작위로 생성된다. 입력(신청)을 처리하는 페이지에서는 입력(신청) 페이지에서 요청 파라미터로 전달된 HIDDEN 필드의 토큰 값과 세션에 저장된 토큰 값을 비교하여 일치하는 경우에만 정상 요청으로 판단하여 입력(신청)이 처리될 수 있도록 한다.

안전한 코드의 예


```

// 입력화면이 요청되었을 때, 임의의 토큰을 생성한 후 세션에 저장한다.
session.setAttribute("SESSION_CSRF_TOKEN", UUID.randomUUID().toString());
// 입력화면에 임의의 토큰을 HIDDEN 필드항목의 값으로 설정해 서버로 전달되도록 한다.
<input type="hidden" name="param_csrf_token" value="${SESSION_CSRF_TOKEN}"/>
// 요청 파라미터와 세션에 저장된 토큰을 비교해서 일치하는 경우에만 요청을 처리한다.
String pToken = request.getParameter("param_csrf_token");
String sToken = (String)session.getAttribute("SESSION_CSRF_TOKEN");
if (pToken != null && pToken.equals(sToken) {
// 일치하는 토큰이 존재하는 경우 -> 정상 처리
.....
} else {
// 토큰이 없거나 값이 일치하지 않는 경우 -> 오류 메시지 출력
.....
}

```

- 표준프레임워크 3.0부터 Spring Security 설정 간소화⁵⁾ 방법을 제공한다. CSRF 기능 사용여부 속성값을 true로 할 수 있으며, 이 경우 <form:form> tag를 사용하면 CsrfRequestDataValueProcessor를 사용하여 Csrf 토큰이 자동으로 포함된다. 모든 POST, PUT, DELETE 메소드에 CSRF 토큰을 포함시켜야 하며, _csrf 요청 속성을 사용하여 현재 CSRF 토큰을 얻을 수 있으며, JSP 파일에서 다음과 같이 사용할 수 있다. <sec: 와 <form: 커스텀태그는 Namespace로써 자유롭게 변경 가능하다.

안전한 코드의 예1

```

<form name="userForm" method="post" action="/user/userUpdate.do">
  <input type="hidden" name="${ _csrf.parameterName }" value="${ _csrf.token }"/>
  <input type="submit" value="저장" />
</form>

```

안전한 코드의 예2

```

<form name="userForm" method="post" action="/user/userUpdate.do">
  <sec:csrfInput />
  <input type="submit" value="저장" />
</form>

```

안전한 코드의 예3

```

<form:form name="userForm" method="post" action="/user/userUpdate.do">
  <!-- form:form 태그사용시 csrf hidden 값이 자동으로 추가됨 -->
</form:form>

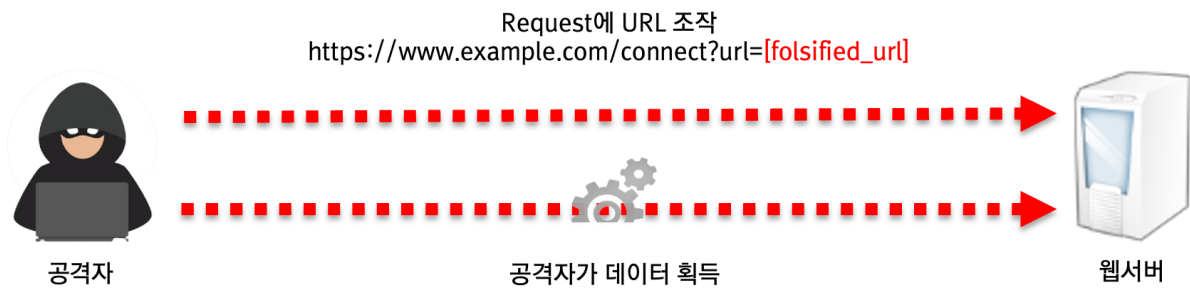
```

5) https://www.egovframe.go.kr/wiki/doku.php?id=egovframework:rte3:fdl:server_security:xmlschema_v3_8 참조

라. 참고자료

- CWE-352 Cross-Site Request Forgery(CSRF), MITRE,
<https://cwe.mitre.org/data/definitions/352.html>
- "Security Corner: Cross-Site Request Forgeries", Chris Shiflett,
<http://shiflett.org/articles/cross-site-request-forgeries>
- Cross-Site_Request_Forgery_(CSRF),
<https://owasp.org/www-community/attacks/csrf>
- Spring Security's Cross Site Request Forgery support
3833<https://docs.spring.io/spring-security/site/docs/3.2.0.CI-SNAPSHOT/reference/html/csrf.html>
- 표준프레임워크 Spring Security 간소화 설정
https://www.egovframe.go.kr/wiki/doku.php?id=egovframework:rte3:fdl:server_security:xmlschema_v3_8

12. 서버사이드 요청 위조



가. 개요

적절한 검증절차를 거치지 않은 사용자 입력 값을 서버간의 요청에 사용하여 악의적인 행위가 발생할 수 있는 보안약점이다. 외부에 노출된 웹 서버에 취약한 애플리케이션이 존재하는 경우 공격자는 URL 또는 요청문을 위조하여 접근통제를 우회하는 방식으로 비정상적인 동작을 유도하거나 신뢰된 네트워크에 있는 데이터를 획득할 수 있다.

나. 보안대책

식별할 수 있는 범위 내에서 사용자의 입력 값을 다른 시스템의 서비스 호출에 사용하는 경우, 사용자의 입력 값을 화이트리스트 방식으로 필터링한다. 사용자가 지정하는 무작위 URL을 받아들여야 한다면, 내부의 URL을 블랙리스트로 지정하여 필터링 한다. 또한 동일한 내부 네트워크에 있더라도 기기 인증, 접근권한을 확인하여 요청이 이루어질 수 있도록 한다.

다. 코드예제

- 다음 예제는 사용자로부터 입력받은 값의 검증 없이 웹페이지를 접속하도록 구현되어 있다. 이 때, 공격자는 URL을 조작하여 내부 서버에 질의하게 하여 데이터를 획득할 수 있다.

참고: URL 조작의 예

설명	URL 조작의 예
내부망 중요 정보 획득	http://site_example.com/connect?url=http://192.168.0.45/member/list.json
외부 접근 차단된 admin 페이지 접근	http://site_example.com/connect?url=http://192.168.0.45/admin
도메인 체크를 우회하여 중요 정보 획득	http://site_example.com/connect?url=http://site_example.com:x@192.168.0.45/member/list.json
단축 URL을 이용한 Filter 우회	http://site_example.com/connect?url=http://bit.ly/sjdk3kjhl3
도메인을 사실IP로 설정해 중요정보 획득	http://site_example.com/connect?url=http://internal.site.com/member/list.json
서버내 파일 열람	http://site_example.com/connect?url=http://attack/fileview.html

안전하지 않은 코드의 예

```
protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
    // 사용자 입력값(url)을 검증없이 사용하여 안전하지 않다.
    URL url = new URL(req.getParameter("url"));
    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
}
```

- 다음은 사전에 정의된 URL 목록을 맵(Map) 객체에 정의하고 키 값으로 입력받아 키에 매칭되는 URL만 사용할 수 있으므로 URL값을 임의로 조작할 수 없다.

안전한 코드의 예

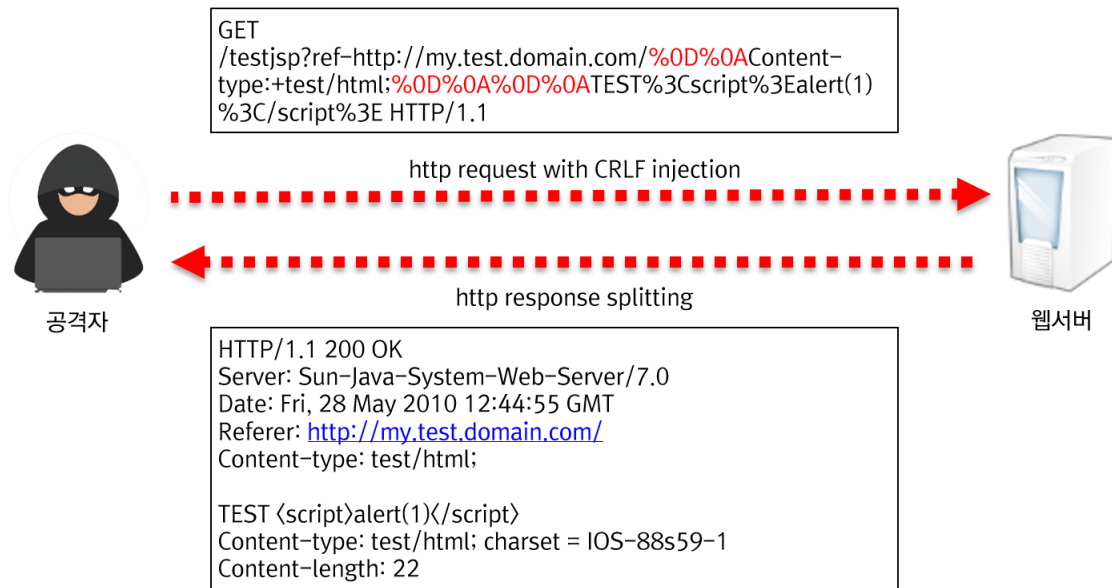
```
public class Connect {
    // key, value 형식으로 URL의 리스트를 작성한다.
    private Map<String, URL> urlMap;
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        // 사용자에게 urlMap의 key를 입력받아 urlMap에서 URL값을 참조한다.
        URL url = urlMap.get(req.getParameter("url"));
        // urlMap에서 참조한 값으로 Connection을 만들어 접속한다.
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    }
}
```

라. 참고자료

- CWE-918: Server-Side Request Forgery (SSRF), MITRE, <https://cwe.mitre.org/data/definitions/918.html>

- Server Side Request Forgery, OWASP,
https://owasp.org/www-community/attacks/Server_Side_Request_Forgery

13. HTTP 응답분할



가. 개요

HTTP 요청에 들어 있는 파라미터(Parameter)가 HTTP 응답 헤더에 포함되어 사용자에게 다시 전달 될 때, 입력값에 CR(Carriage Return)이나 LF(Line Feed)와 같은 개행문자가 존재하면 HTTP 응답이 2개 이상으로 분리될 수 있다. 이 경우 공격자는 개행문자를 이용하여 첫 번째 응답을 종료 시키고, 두 번째 응답에 악의적인 코드를 주입하여 XSS 및 캐시 훼손(Cache Poisoning) 공격 등을 수행할 수 있다.

나. 보안대책

요청 파라미터의 값을 HTTP응답헤더(예를 들어, Set-Cookie 등)에 포함시킬 경우 CR, LF와 같은 개행문자를 제거한다.

다. 코드예제

- 외부 입력값을 사용하여 반환되는 쿠키의 값을 설정하고 있다. 그런데, 공격자가 Wiley Hacker `WrWn`HTTP/1.1 200 OK`WrWn`를 lastLogin의 값으로 설정할 경우, 응답이 분리되어 전달되며 분리된 응답 본문의 내용을 공격자가 마음대로 수정할 수 있다.

안전하지 않은 코드의 예

```
//외부로부터 입력받은 값을 검증 없이 사용할 경우 안전하지 않다.  
String lastLogin = request.getParameter("last_login");  
if (lastLogin == null || "".equals(lastLogin)) {  
    return;  
}  
// 쿠키는 Set-Cookie 응답헤더로 전달되므로 개행문자열 포함 여부 검증이 필요  
Cookie c = new Cookie("LASTLOGIN", lastLogin);  
c.setMaxAge(1000);  
c.setSecure(true);  
response.addCookie(c);  
response.setContentType("text/html");
```

- 외부에서 입력되는 값에 대하여 null 여부를 체크하고, 응답이 여러 개로 나뉘지는 것을 방지하기 위해 개행문자를 제거하고 응답헤더의 값으로 사용한다.

안전한 코드의 예

```
String lastLogin = request.getParameter("last_login");  
if (lastLogin == null || "".equals(lastLogin)) {  
    return;  
}  
// 외부 입력값에서 개행문자(WrWn)를 제거한 후 쿠키의 값으로 설정  
lastLogin = lastLogin.replaceAll("[\\W\\r\\Wn]", "");  
Cookie c = new Cookie("LASTLOGIN", lastLogin);  
c.setMaxAge(1000);  
c.setSecure(true);  
response.addCookie(c);
```

라. 참고자료

- CWE-113 HTTP Response Splitting, MITRE,
<https://cwe.mitre.org/data/definitions/113.html>
- HTTP Response Splitting, OWASP
https://owasp.org/www-community/attacks/HTTP_Response_Splitting

14. 정수형 오버플로우

가. 개요

정수형 오버플로우는 정수형 크기는 고정되어 있는데 저장할 수 있는 범위를 넘어서, 크 기보다 큰 값을 저장하려 할 때 실제 저장되는 값이 의도치 않게 아주 작은 수이거나 음수가 되어 프로그램이 예기치 않게 동작 될수 있다. 특히 반복문 제어, 메모리 할당, 메모리 복사 등을 위한 조건으로 사용자 가 제공하는 입력값을 사용하고 그 과정에서 정수형 오버플로우가 발생하는 경우 보안상 문제를 유발 할 수 있다.

나. 보안대책

언어/플랫폼별 정수타입의 범위를 확인하여 사용한다. 정수형 변수를 연산에 사용하는 경우, 결과 값의 범위를 체크하는 모듈을 사용한다. 특히 외부입력값을 동적 메모리 할당에 사용하는 경우, 변수 값이 적절한 범위 내에 존재하는 값인지 확인한다.

다. 코드예제

- 다음의 예제는 외부의 입력(slf_msg_param_num)을 이용하여 동적으로 계산한 값을 배열의 크기 (size)를 결정하는데 사용하고 있다. 만일 외부 입력으로부터 계산된 값 (param_ct)이 오버 플로우에 의해 음수값이 되면, 배열의 크기가 음수가 되어 시스템에 문제가 발생할 수 있다.

안전하지 않은 코드의 예 JAVA

```
String msg_str = "";
String tmp = request.getParameter("slf_msg_param_num");
tmp = StringUtil.isNullTrim(tmp);
if (tmp.equals("0")) {
    msg_str = PropertyUtil.getValue(msg_id);
} else {
    // 외부 입력값을 정수형으로 사용할 때 입력값의 크기를 검증하지 않고 사용
    int param_ct = Integer.parseInt(tmp);
```



```
String[] strArr = new String[param_ct];
```

- 동적 메모리 할당을 위해 외부 입력값을 배열의 크기로 사용하는 경우 그 값이 음수가 아닌지 검사 하는 작업이 필요하다.

안전한 코드의 예

```
String msg_str = "";
String tmp = request.getParameter("slf_msg_param_num");
tmp = StringUtil.isNullTrim(tmp);
if (tmp.equals("0")) {
    msg_str = PropertyUtil.getValue(msg_id);
} else {
    // 외부 입력값을 정수형으로 사용할 때 입력값의 크기를 검증하고 사용
    try {
        int param_ct = Integer.parseInt(tmp);
        if (param_ct < 0) {
            throw new Exception();
        }
        String[] strArr = new String[param_ct];
    } catch (Exception e) {
        msg_str = "잘못된 입력(접근) 입니다.";
    }
}
```

라. 참고자료

- CWE-190 Integer Overflow, MITRE,
<https://cwe.mitre.org/data/definitions/190.html>
- Enforce limits on integer values originating from tainted sources, CERT,
<https://wiki.sei.cmu.edu/confluence/display/c/INT04-C.+Enforce+limits+on+integer+values+originating+from+tainted+sources>
- Verify that all integer values are in range, CERT,
<https://wiki.sei.cmu.edu/confluence/display/c/INT08-C.+Verify+that+all+integer+values+are+in+range>

15. 보안기능 결정에 사용되는 부적절한 입력값

가. 개요

- 응용프로그램이 외부 입력값에 대한 신뢰를 전제로 보호메커니즘을 사용하는 경우 공격자가 입력값을 조작할 수 있다면 보호메커니즘을 우회할 수 있게 된다.
- 개발자들이 흔히 쿠키, 환경변수 또는 히든필드와 같은 입력값이 조작될 수 없다고 가정 하지만 공격자는 다양한 방법을 통해 이러한 입력값들을 변경할 수 있고 조작된 내용은 탐지되지 않을 수 있다. 인증이나 인가와 같은 보안결정이 이런 입력값(쿠키, 환경변수, 히든필드 등)에 기반해 수행되는 경우 공격자는 이런 입력값을 조작하여 응용프로그램의 보안을 우회할 수 있으므로 충분한 암호화, 무결성 체크를 수행하고 이와 같은 메커니즘이 없는 경우엔 외부사용자에 의한 입력값을 신뢰해서는 안된다.

나. 보안대책

상태정보나 민감한 데이터 특히 사용자 세션정보와 같은 중요한 정보는 서버에 저장하고 보안확인 절차도 서버에서 실행한다. 보안설계관점에서 신뢰할 수 없는 입력값이 응용프로그램 내부로 들어올 수 있는 지점과 보안결정에 사용되는 입력값을 식별하고 제공되는 입력값에 의존할 필요가 없는 구조로 변경할 수 있는지 검토한다.

다. 코드예제

- 구입품목의 가격을 사용자 웹브라우저에서 처리하고 있어 이 값이 사용자에게 의해 변경되는 경우 가격 (단가)정보가 의도하지 않은 값으로 할당될 수 있다.

안전하지 않은 코드의 예 JAVA

```
<input type="hidden" name="price" value="1000"/>
```

```
<br/>품목 : HDTV
<br/>수량 : <input type="hidden" name="quantity" />개
<br/><input type="submit" value="구입" />
```

```
try {
// 서버가 보유하고 있는 가격(단가) 정보를 사용자 화면에서 받아서 처리
price = request.getParameter("price");
quantity = request.getParameter("quantity");
total = Integer.parseInt(quantity) * Float.parseFloat(price);
} catch (Exception e) {
.....
}
```

- 사용자 권한, 인증 여부 등 보안결정에 사용하는 값은 사용자 입력값을 사용하지 않고 서버 내부의 값을 활용한다. 또한 사용자 입력에 의존해야하는 값을 제외하고는 반드시 서버가 보유하고 있는 정보를 이용하여 처리한다.

안전한 코드의 예

```
<input type="hidden" name="price" value="1000"/>
<br/>품목 : HDTV
<br/>수량 : <input type="hidden" name="quantity" />개
<br/><input type="submit" value="구입" />
```

```
try {
item = request.getParameter("item");
// 가격이 아니라 item 항목을 가져와서 서버가 보유하고 있는 가격 정보를
// 이용하여 전체 가격을 계산
price = productService.getPrice(item);
quantity = request.getParameter("quantity");
total = Integer.parseInt(quantity) * price;
} catch (Exception e) {
.....
}
.....
```

라. 참고자료

- CWE-807 Reliance on Untrusted Inputs in a Security Decision, MITRE,
<https://cwe.mitre.org/data/definitions/807.html>
- Do not trust the values of environment variables, CERT,
<https://wiki.sei.cmu.edu/confluence/display/java/ENV02-J.+Do+not+trust+the+values+of+environment+variables>

- Session Management, OWASP

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

16. 포맷 스트링 삽입

가. 개요

외부로부터 입력된 값을 검증하지 않고 입·출력 함수의 포맷 문자열로 그대로 사용하는 경우 발생할 수 있는 보안약점이다. 공격자는 포맷 문자열을 이용하여 취약한 프로세스를 공격하거나 메모리 내용을 읽거나 쓸 수 있다. 그 결과, 공격자는 취약한 프로세스의 권한을 취득하여 임의의 코드를 실행할 수 있다.

나. 보안대책

printf(), snprintf() 등 포맷 문자열을 사용하는 함수를 사용할 때는 사용자 입력값을 직접적으로 포맷 문자열로 사용하거나 포맷 문자열 생성에 포함시키지 않는다. 포맷문자열을 사용하는 함수에 사용자 입력값을 사용할 때는 사용자가 포맷 스트링을 변경할 수 있는 구조로 쓰지 않는다. 특히, %n, %hn은 공격자가 이를 이용해 특정 메모리 위치에 특정값을 변경할 수 있으므로 포맷 스트링 매개변수로 사용하지 않는다. 사용자 입력값을 포맷 문자열을 사용하는 함수에 사용할 때는 가능하면 %s 포맷 문자열을 지정하고, 사용자 입력값은 2번째 이후의 파라미터로 사용한다.

다. 코드예제

- 이 프로그램에서 공격자 는 %1\$tm, %1\$te, 또는 %1\$tY과 같은 문자열을 입력하여 포맷 문자열에 포함시킴으로써, 실제 유효기간 validDate가 출력되도록 할 수 있다.

안전하지 않은 코드의 예 JAVA

```
// 외부 입력값에 포맷 문자열 포함 여부를 확인하지 않고 포맷 문자열 출력에 값으로 사용
// args[0]의 값으로 "%1$tY-%1$tm-%1$te"를 전달하면 시스템에서 가지고 있는
날짜(2014-10-14) 정보가 노출
import java.util.Calendar
.....
public static void main(String[] args) {
```

```

Calendar validDate = Calendar.getInstance();
validDate.set(2014, Calendar.OCTOBER, 14);
System.out.printf( args[0] + " did not match! HINT: It was issued on %1$terd
of some month", validate);
}

```

- 사용자로부터 입력 받은 문자열을 포맷 문자열에 직접 포함시키지 않고, %s 포맷 문자열을 사용함으로써 정보유출을 방지한다.

안전한 코드의 예

```

// 외부 입력값이 포맷 문자열 출력에 사용되지 않도록 수정
import java.util.Calendar
:
public static void main(String[] args) {
    Calendar validDate = Calendar.getInstance();
    validDate.set(2014, Calendar.OCTOBER, 14);
    System.out.printf("%s did not match! HINT: It was issued on %2$terd of some
month", args[0], validate);
}

```

라. 참고자료

- CWE-134 Uncontrolled Format String, MITRE,
<https://cwe.mitre.org/data/definitions/134.html>
- Exclude user input from format strings, CERT,
<https://wiki.sei.cmu.edu/confluence/display/c/FIO30-C.+Exclude+user+input+from+format+strings>
- Use valid format strings, CERT,
<https://wiki.sei.cmu.edu/confluence/display/c/FIO47-C.+Use+valid+format+strings>
- Format string attack, OWASP,
https://www.owasp.org/index.php/Format_string_attack

제2절 보안기능

보안기능(인증, 접근제어, 기밀성, 암호화, 권한관리 등)을 부적절하게 구현 시 발생할 수 있는 보안 약점으로 적절한 인증 없는 중요기능 허용, 부적절한 인가 등이 포함된다.

1. 적절한 인증 없는 중요기능 허용

가. 개요

적절한 인증과정이 없이 중요정보(계좌이체 정보, 개인정보 등)를 열람(또는 변경)할 때 발생하는 보안약점이다.

나. 보안대책

클라이언트의 보안검사를 우회하여 서버에 접근하지 못하도록 설계하고 중요한 정보가 있는 페이지는 재인증을 적용(은행 계좌이체 등)한다. 또한 안전하다고 검증된 라이브러리나 프레임워크 (OpenSSL 이나 ESAPI의 보안기능 등)를 사용하는 것이 중요하다.

다. 코드예제

- 회원정보 수정 시 수정을 요청한 사용자와 로그인한 사용자의 일치 여부를 확인하지 않고 처리하고 있다.

안전하지 않은 코드의 예

```
@RequestMapping(value = "/modify.do", method = RequestMethod.POST)
public ModelAndView memberModifyProcess(@ModelAttribute("MemberModel")
MemberModel memberModel, BindingResult result, HttpServletRequest request,
HttpSession session) {
    ModelAndView mav = new ModelAndView();
    //1. 로그인한 사용자를 불러온다.
    String userId = (String) session.getAttribute("userId");
    String passwd = request.getParameter("oldUserPw");
    ...
    //2. 실제 수정하는 사용자와 일치 여부를 확인하지 않고, 회원정보를 수정하여 안전하지 않다.
    if (service.modifyMember(memberModel)) {
        mav.setViewName("redirect:/board/list.do");
        session.setAttribute("userName", memberModel.getUserName());
        return mav;
    } else {
```

```

mav.addObject("errCode", 2);
mav.setViewName("/board/member_modify");
return mav;
}
}

```

- 로그인한 사용자와 요청한 사용자의 일치 여부를 확인한 후 회원정보를 수정하도록 한다

안전한 코드의 예

```

@RequestMapping(value = "/modify.do", method = RequestMethod.POST)
public ModelAndView memberModifyProcess(@ModelAttribute("MemberModel")
MemberModel memberModel, BindingResult result, HttpServletRequest request,
HttpSession session) {
    ModelAndView mav = new ModelAndView();
    //1. 로그인한 사용자를 불러온다.
    String userId = (String) session.getAttribute("userId");
    String passwd = request.getParameter("oldUserPw");
    //2. 회원정보를 실제 수정하는 사용자와 로그인 사용자와 동일한지 확인한다.
    String requestUser = memberModel.getUserId();
    if (userId != null && requestUser != null && !userId.equals(requestUser)) {
        mav.addObject("errCode", 1);
        mav.addObject("member", memberModel);
        mav.setViewName("/board/member_modify");
        return mav;
    }
    ...
    //3. 동일한 경우에만 회원정보를 수정해야 안전하다.
    if (service.modifyMember(memberModel)) {
        ...
    }
}

```

라. 참고자료

- CWE-306 Missing Authentication for Critical Function, MITRE, <https://cwe.mitre.org/data/definitions/306.html>
- Access Control, OWASP, https://owasp.org/www-community/Access_Control

2. 부적절한 인가

가. 개요

프로그램이 모든 가능한 실행경로에 대해서 접근제어를 검사하지 않거나 불완전하게 검사하는 경우, 공격자는 접근 가능한 실행경로를 통해 정보를 유출할 수 있다.

나. 보안대책

응용프로그램이 제공하는 정보와 기능을 역할에 따라 배분함으로써 공격자에게 노출되는 공격노출면⁶⁾ (Attack Surface)을 최소화하고 사용자의 권한에 따른 ACL(Access Control List)을 관리한다. 프레임워크를 사용해서 취약점을 피할 수도 있는데 예를 들면, JAAS Authorization Framework나 OWASP ESAPI Access Control 등을 인증 프레임워크로 사용 가능하다.

다. 코드예제

- 아래의 코드는 사용자 입력값에 따라 삭제작업을 수행하고 있으며, 사용자의 권한 확인을 위한 별도의 통제가 적용되지 않고 있다.

안전하지 않은 코드의 예

```
private BoardDao boardDao;
String action = request.getParameter("action");
String contentId = request.getParameter("contentId");
//요청을 하는 사용자의 delete 작업 권한 확인 없이 수행하고 있어 안전하지 않다.
if (action != null && action.equals("delete")) {
    boardDao.delete(contentId);
}
```

- 아래의 코드처럼 세션에 저장된 사용자 정보를 통해 해당 사용자가 삭제작업을 수행할 권한이 있는지 확인한 뒤 권한

6) OSSTMM 3 Defines Attack Surface as "The lack of specific separations and functional controls that exist for that vector"

이 있는 경우에만 수행하도록 해야 한다.

안전한 코드의 예

```
private BoardDao boardDao;
String action = request.getParameter("action");
String contentId = request.getParameter("contentId");
// 세션에 저장된 사용자 정보를 얻어온다.
User user= (User)session.getAttribute("user");
// 사용자정보에서 해당 사용자가 delete작업의 권한이 있는지 확인한 뒤 삭제 작업을 수행한다.
if (action != null && action.equals("delete") && checkAccessControlList(user,action)) {
    boardDao.delete(contentId);
}
```

라. 참고자료

- CWE-285 Improper Authorization, MITRE,
<https://cwe.mitre.org/data/definitions/285.html>
- Access Control, OWASP,
https://owasp.org/www-community/Access_Control

3. 중요한 자원에 대한 잘못된 권한 설정

가. 개요

SW가 중요한 보안관련 자원에 대하여 읽기 또는 수정하기 권한을 의도하지 않게 허가할 경우, 권한 을 갖지 않은 사용자가 해당 자원을 사용하게 된다.

나. 보안대책

설정파일, 실행파일, 라이브러리 등은 SW 관리자에 의해서만 읽고 쓰기가 가능하도록 설정하고 설정 파일과 같이 중요한 자원을 사용하는 경우, 허가 받지 않은 사용자가 중요한 자원에 접근 가능 한지 검사한다.

다. 코드예제

- `"/home/setup/system.ini"` 파일에 대해 모든 사용자가 읽고, 쓰고, 실행할 수 있도록 권한을 부여 하고 있다.
- `setExecutable(p1, p2)` : 첫 번째 파라미터의 `true/false` 값에 따라 실행가능 여부를 결정한다. 두 번째 파라미터가 `true` 일 경우 소유자만 실행 권한을 가지며, `false` 일 경우 모든 사용자가 실행 권한을 가진다.
- `setReadable(p1, p2)` : 첫 번째 파라미터의 `true/false` 값에 따라 읽기가능 여부를 결정한다. 두 번째 파라미터가 `true` 일 경우 소유자만 읽기권한을 가지며, `false` 일 경우 모든 사용자가 읽기 권한을 가진다.
- `setWritable(p1, p2)` : 첫 번째 파라미터의 `true/false` 값에 따라 쓰기가능 여부를 결정한다. 두 번째 파라미터가 `true` 일 경우 소유자만 쓰기권한을 가지며, `false` 일 경우 모든 사용자가 쓰기 권한을 가진다.

안전하지 않은 코드의 예

```
File file = new File("/home/setup/system.ini");
```

```
//모든 사용자에게 실행 권한을 허용하여 안전하지 않다.
file.setExecutable(true, false);
//모든 사용자에게 읽기 권한을 허용하여 안전하지 않다.
file.setReadable(true, false);
//모든 사용자에게 쓰기 권한을 허용하여 안전하지 않다.
file.setWritable(true, false);
```

- 파일에 대해서는 최소권한을 할당해야 한다. 즉 해당 파일의 소유자에게만 읽기 권한을 부여한다.
- setExecutable(p1) : 파라미터의 true/false 값에 따라 소유자의 실행권한 여부를 결정한다.
- setReadable(p1) : 파라미터의 true/false 값에 따라 소유자의 읽기권한 여부를 결정한다.
- setWritable(p1) : 파라미터의 true/false 값에 따라 소유자의 쓰기권한 여부를 결정한다.

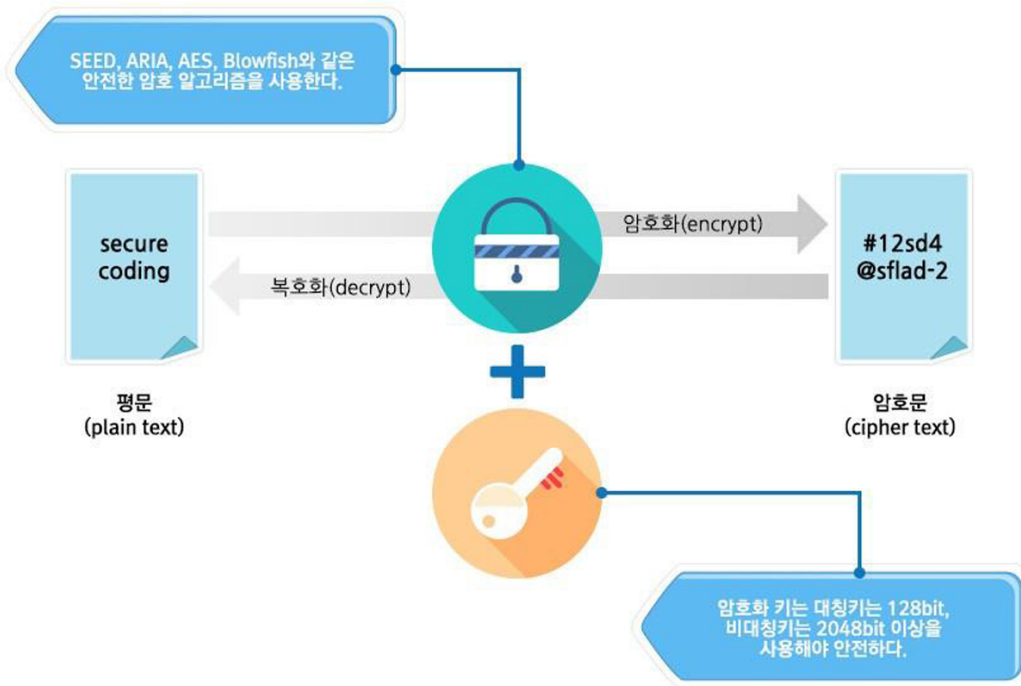
안전한 코드의 예

```
File file = new File("/home/setup/system.ini");
//소유자에게 실행 권한을 금지하였다.
file.setExecutable(false);
//소유자에게 읽기 권한을 허용하였다.
file.setReadable(true);
//소유자에게 쓰기 권한을 금지하였다.
file.setWritable(false);
```

라. 참고자료

- CWE-732 Incorrect Permission Assignment for Critical Resource, MITRE,
<https://cwe.mitre.org/data/definitions/732.html>
- Create files with appropriate access permissions, CERT,
<https://wiki.sei.cmu.edu/confluence/display/c/FIO06-C.+Create+files+with+appropriate+access+permissions>

4. 취약한 암호화 알고리즘 사용



가. 개요

- SW 개발자들은 환경설정 파일에 저장된 패스워드를 보호하기 위하여 간단한 인코딩 함수를 이용 하여 패스워드를 감추는 방법을 사용하기도 한다. 그렇지만 base64와 같은 지나치게 간단한 인코딩 함수로는 패스워드를 제대로 보호할 수 없다.
- 정보보호 측면에서 취약하거나 위험한 암호화 알고리즘을 사용해서는 안 된다. 표준화되지 않은 암호화 알고리즘을 사용하는 것은 공격자가 알고리즘을 분석하여 무력화시킬 수 있는 가능성을 높일 수도 있다. 몇몇 오래된 암호화 알고리즘의 경우는 컴퓨터의 성능이 향상됨에 따라 취약해지기도 해서, 예전에는 해독하는데 몇 십 억년이 걸릴 것이라고 예상되던 알고리즘이 며칠이나 몇 시간 내에 해독되기도 한다. RC2, RC4, RC5, RC6, MD4, MD5, SHA1, DES 알고리즘이 여기에 해당된다.

나. 보안대책

- 자신만의 암호화 알고리즘을 개발하는 것은 위험하며, 학계 및 업계에서 이미 검증된 표준화된 알고리즘을 사용한다. 기존에 취약하다고 알려진 DES, RC5등의 암호알고리즘을 대신하여, 3DES, AES, SEED 등의 안전한 암호알고리즘으로 대체하여 사용한다. 또한, 업무관련 내용, 개인정보 등에 대한 암호 알고리즘 적용 시, IT보안인증 사무국이 안전성을 확인한 검증필 암호모듈을 사용해야한다.
- 비밀번호 정보를 저장하는 경우 단방향(해시함수) 암호알고리즘인 SHA-224, SHA-256, SHA-384, SHA-512 등을 사용한다.
- 주민등록번호 및 계좌정보 등 금융정보를 저장하는 경우, 데이터 암호·복호화가 가능한 양방향(대칭키) 암호 알고리즘인 AES, SEED, HIGHT, ARIA, LEA등을 사용한다.
- 공개키 전자서명을 적용하려는 경우, 공개키(비대칭키) 암호 알고리즘인 RSAES 을 사용한다. 공개키/개인키 유효기간은 최대 2년으로 설정하기를 권장한다.
- 안전성을 2030년 이후까지 유지하기 위해서는 보안강도 128 비트 이상의 암호 알고리즘 사용을 권고한다.

다. 코드예제

- 다음 예제는 취약한 DES 알고리즘으로 암호화하고 있다.

안전하지 않은 코드의 예

```
import java.security.*;
import javax.crypto.Cipher;
import javax.crypto.NoSuchPaddingException;
public class CryptoUtils {
    public byte[] encrypt(byte[] msg, Key k) {
        byte[] rslt = null;
        try {
            //키 길이가 짧아 취약함 암호화 알고리즘인 DES를 사용하여 안전하지 않다.
            Cipher c = Cipher.getInstance("DES");
            c.init(Cipher.ENCRYPT_MODE, k);
            rslt = c.update(msg);
        }
    }
}
```

- 아래 코드처럼 안전하다고 알려진 AES 알고리즘 등을 적용한다.

안전한 코드의 예

```
import java.security.*;
import javax.crypto.Cipher;
import javax.crypto.NoSuchPaddingException;
public class CryptoUtils {
    public byte[] encrypt(byte[] msg, Key k) {
        byte[] rslt = null;
        try {
            //키 길이가 길어 강력한 알고리즘인 AES를 사용하여 안전하다.
            Cipher c = Cipher.getInstance("AES/CBC/PKCS5Padding");
            c.init(Cipher.ENCRYPT_MODE, k);
            rslt = c.update(msg);
        }
    }
}
```

- 다음은 블록 암호 체계인 ARIA 알고리즘을 적용한 예기이다.

안전한 코드의 예

```
import egovframework.rte.fdl.cryptography.EgovEnvCryptoService;

@Controller
public class EgovWebEditorImageController {
    .....

    @Resource(name = "egovEnvCryptoService")
    EgovEnvCryptoService cryptoService;

    .....

    private String encrypt(String encrypt) {
        try {
            return cryptoService.encrypt(encrypt); // Handles URLEncoding.
        } catch (IllegalArgumentException e) {
            LOGGER.error("[IllegalArgumentException] Try/Catch...usingParameters Runing : "+
e.getMessage());
        } catch (Exception e) {
            LOGGER.error("[ " + e.getClass() + " ] : " + e.getMessage());
        }
        return encrypt;
    }

    private String decrypt(String decrypt){
        try {
            return cryptoService.decryptNone(decrypt); // Does not handle URLDecoding.
        } catch (IllegalArgumentException e) {
            LOGGER.error("[IllegalArgumentException] Try/Catch...usingParameters Runing : "+
e.getMessage());
        }
    }
}
```

7) <https://github.com/eGovFramework/egovframe-common-components/blob/master/src/main/java/egovframework/com/utl/wed/web/EgovWebEditorImageController.java> 참조

```

    } catch (Exception e) {
        LOGGER.error("[ " + e.getClass() + " ] : " + e.getMessage());
    }
    return decrypt;
}
}

```

- 다음은 비밀번호를 단방향으로 암호화한 예제⁸⁾이다.

안전한 코드의 예

```

public static String encryptPassword(String data, byte[] salt) throws Exception {
    if (data == null) {
        return "";
    }
    byte[] hashValue = null; // 해쉬값
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    // 취약하지 않은 알고리즘인 SHA-256 사용
    md.reset();
    md.update(salt);
    hashValue = md.digest(data.getBytes());
    return new String(Base64.encodeBase64(hashValue));
}

```

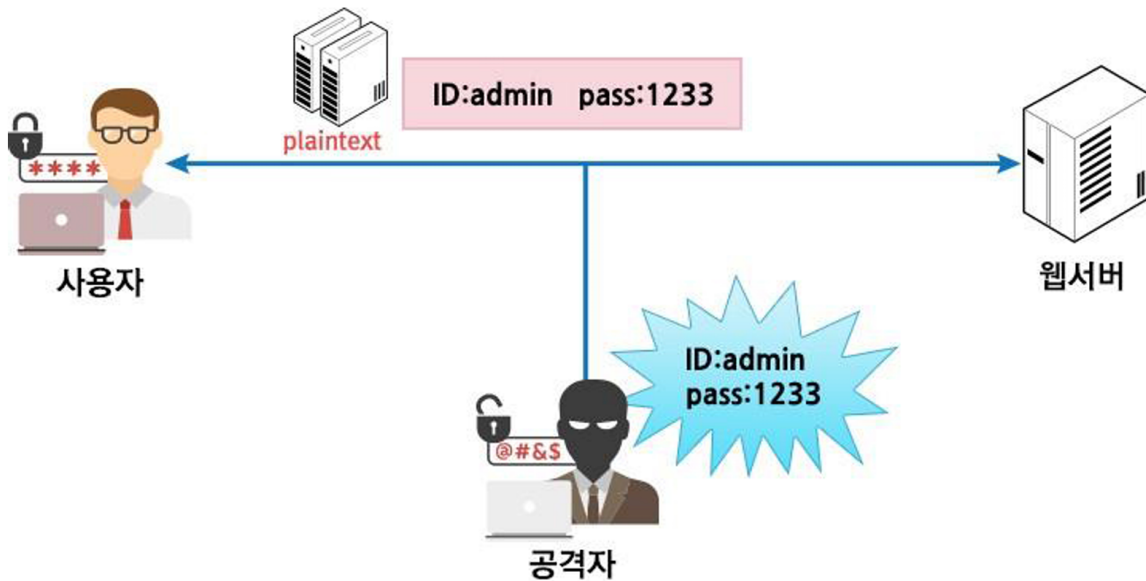
라. 참고자료

- CWE-327 Use of a Broken or Risky Cryptographic Algorithm, MITRE,
<https://cwe.mitre.org/data/definitions/327.html>
- Do not use insecure or weak cryptographic algorithms, CERT,
<https://wiki.sei.cmu.edu/confluence/display/java/MS61-J.+Do+not+use+insecure+or+weak+cryptographic+algorithms>
- Cryptanalysis, OWASP,
<https://owasp.org/www-community/attacks/Cryptanalysis>
- 암호이용활성화, KISA,
<https://seed.kisa.or.kr/kisa/Board/38/detailView.do>
- 표준프레임워크 ARIA 방식 암호화/복호화,

8) <https://github.com/eGovFramework/egovframe-common-components/blob/master/src/main/java/egovframework/com/utl/sim/service/EgovFileScrtty.java> 참조

https://www.egovframe.go.kr/wiki/doku.php?id=egovframework:com:v3.8:sec:%EC%95%94%ED%98%B8%ED%99%94_%EB%B3%B5%ED%98%B8%ED%99%94

5. 암호화되지 않은 중요정보



가. 개요

사용자 또는 시스템의 중요정보가 포함된 데이터를 평문으로 송수신 또는 저장할 때 인가되지 않은 사용자에게 민감한 정보가 노출될 수 있는 보안약점이다.

나. 보안대책

개인정보(주민등록번호, 여권번호 등), 금융정보(카드번호, 계좌번호 등), 패스워드 등 중요정보를 통신채널로 전송하거나 저장할 때는 반드시 암호화 과정을 거쳐야 한다. 필요한 경우 SSL 또는 HTTPS 등과 같은 암호채널을 사용해야 하며, HTTPS와 같은 보안 채널을 사용하여 브라우저 쿠키에 중요 데이터를 저장하는 경우, 쿠키객체에 보안속성을 반드시 설정하여 중요정보의 노출을 방지한다. 중요정보를 읽거나 쓸 경우에 권한인증 등으로 적합한 사용자가 중요정보에 접근하도록 해야 한다.

다. 코드예제

- 중요정보 평문저장

- 아래 예제는 인증을 통과한 사용자의 패스워드 정보가 평문으

로 DB에 저장된다.

안전하지 않은 코드의 예

```
String id = request.getParameter("id");
// 외부값에 의해 패스워드 정보를 얻고 있다.
String pwd = request.getParameter("pwd");
.....
String sql = " insert into customer(id, pwd, name, ssn, zipcode, addr)"
+ " values (?, ?, ?, ?, ?, ?)";
PreparedStatement stmt = con.prepareStatement(sql);
stmt.setString(1, id);
stmt.setString(2, pwd);
.....
// 입력받은 패스워드가 평문으로 DB에 저장되어 안전하지 않다.
stmt.executeUpdate();
```

- 다음 예제는 패스워드 등 중요 데이터를 해쉬값으로 변환하여 저장하고 있다.

안전한 코드의 예

```
String id = request.getParameter("id");
// 외부값에 의해 패스워드 정보를 얻고 있다.
String pwd = request.getParameter("pwd");
// 패스워드를 솔트값을 포함하여 SHA-256 해시로 변경하여 안전하게 저장한다.
MessageDigest md = MessageDigest.getInstance("SHA-256");
md.reset();
md.update(salt);
byte[] hashInBytes = md.digest(pwd.getBytes());
StringBuilder sb = new StringBuilder();
for (byte b : hashInBytes) {
    sb.append(String.format("%02x", b));
}
pwd = sb.toString();
.....
String sql = " insert into customer(id, pwd, name, ssn, zipcode, addr)"
+ " values (?, ?, ?, ?, ?, ?)";
PreparedStatement stmt = con.prepareStatement(sql);
stmt.setString(1, id);
stmt.setString(2, pwd);
.....
stmt.executeUpdate();
```

- 다음은 비밀번호를 단방향 암호화한 표준프레임워크의 공통컴포넌트의 예제⁹⁾이다.

안전한 코드의 예

9) <https://github.com/eGovFramework/egovframe-common-components/blob/master/src/main/java/egovframework/com/utl/sim/service/EgovFileScrtty.java> 참조

```

public static String encryptPassword(String data, byte[] salt) throws Exception {
    if (data == null) {
        return "";
    }
    byte[] hashValue = null; // 해쉬값
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    // 취약하지 않은 알고리즘인 SHA-256 사용
    md.reset();
    md.update(salt);
    hashValue = md.digest(data.getBytes());
    return new String(Base64.encodeBase64(hashValue));
}

```

● 중요정보 평문전송

- 아래 예제는 패스워드를 암호화하지 않고 네트워크를 통해 전송하고 있다. 이 경우 패킷 스니핑을 통하여 패스워드가 노출될 수 있다.

안전하지 않은 코드의 예

```

try {
    Socket s = new Socket("taranis", 4444);
    PrintWriter o = new PrintWriter(s.getOutputStream(), true);
    //패스워드를 평문으로 전송하여 안전하지 않다.
    String password = getPassword();
    o.write(password);
} catch (FileNotFoundException e) {
    .....
}

```

- 아래 예제는 패스워드를 네트워크를 통해 서버로 전송하기 전에 AES 등의 안전한 암호알고리즘으로 암호화한 안전한 프로그램이다.

안전한 코드의 예

```

// 패스워드를 암호화 하여 전송
try {
    Socket s = new Socket("taranis", 4444);
    PrintStream o = new PrintStream(s.getOutputStream(), true);
    //패스워드를 강력한 AES암호화 알고리즘을 통해 전송하여 사용한다.
    Cipher c = Cipher.getInstance("AES/CBC/PKCS5Padding");

    String password = getPassword();
    byte[] encPassword = c.update(password.getBytes());
    o.write(encPassword, 0, encPassword.length);
} catch (FileNotFoundException e) {
    .....
}

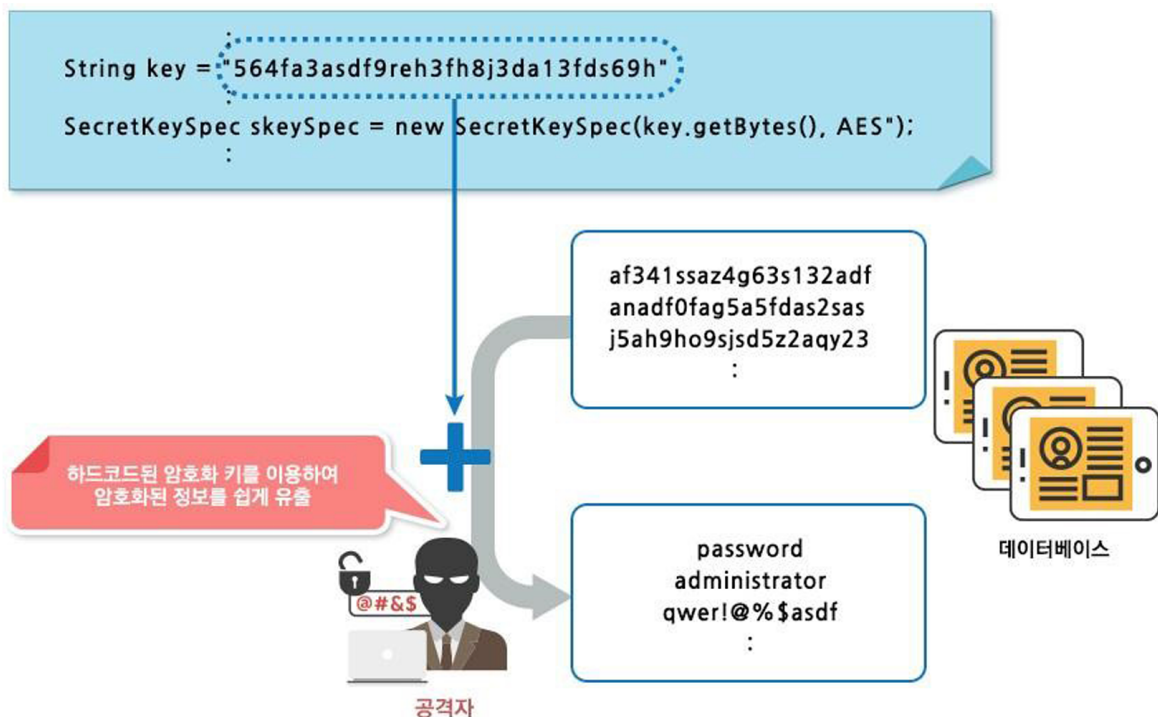
```

라. 참고자료

- CWE-312, Cleartext Storage of Sensitive Information, MITRE, <https://cwe.mitre.org/data/definitions/312.html>
- Clear sensitive information stored in reusable resources, CERT, <https://wiki.sei.cmu.edu/confluence/display/c/MEM03-C.+Clear+sensitive+information+stored+in+reusable+resources>
- Be careful while handling sensitive data, such as passwords, in program code, CERT, <https://wiki.sei.cmu.edu/confluence/display/c/MS18-C.+Be+careful+while+handling+sensitive+data%2C+such+as+passwords%2C+in+program+code>
- Never hard code sensitive information, CERT, <https://wiki.sei.cmu.edu/confluence/display/c/MS41-C.+Never+hard+code+sensitive+information>
- Do not store unencrypted sensitive information on the client side, CERT, <https://wiki.sei.cmu.edu/confluence/display/java/FIO52-J.+Do+not+store+unencrypted+sensitive+information+on+the+client+side>
- Password Plaintext Storage, OWASP https://owasp.org/www-community/vulnerabilities/Password_Plaintext_Storage
- CWE-319, Cleartext Transmission of Sensitive Information, MITRE, <https://cwe.mitre.org/data/definitions/319.html>
- Insecure Transport, OWASP, <https://owasp.org/www-community/vulnerabilities/Insecure>

e_Transport

6. 하드코드된 중요정보



가. 개요

프로그램 코드 내부에 하드코드 된 패스워드 또는 암호화키를 포함하여 내부 인증에 사용하거나 암호화를 수행하면 중요정보(관리자 정보, 암호화된 정보 등)가 유출될 수 있는 보안 약점이다.

나. 보안대책

패스워드는 암호화하여 별도의 파일에 저장하여 사용한다. 또한 중요정보를 암호화하면, 상수가 아닌 암호화 키를 사용하도록 하며 소스코드 내부에 상수형태의 암호화 키를 저장해서 사용하지 않도록 한다.

다. 코드예제

● 하드코드된 비밀번호

- 데이터베이스 연결을 위한 패스워드를 소스코드 내부에 상수 형태로 하드코딩 하는 경우, 접속 정보 가 노출될 수 있어 위험하다.

안전하지 않은 코드의 예

```
public class MemberDAO {
    private static final String DRIVER = "oracle.jdbc.driver.OracleDriver";
    private static final String URL = "jdbc:oracle:thin:@192.168.0.3:1521:ORCL";
    private static final String USER = "scott"; // DB ID;
    //DB 패스워드가 소스코드에 평문으로 저장되어 있다.
    private static final String PASS = "tiger"; // DB PW;
    .....
    public Connection getConn() {
        Connection con = null;
        try {
            Class.forName(DRIVER);
            con = DriverManager.getConnection(URL, USER, PASS);
            .....
        }
    }
}
```

- 패스워드는 안전한 암호화 방식으로 암호화하여 별도의 분리된 공간(파일)에 저장해야 하며, 암호화된 패스워드를 사용하기 위해서는 복호화 과정을 거쳐야 한다.

안전한 코드의 예

```
public class MemberDAO {
    private static final String DRIVER = "oracle.jdbc.driver.OracleDriver";
    private static final String URL = "jdbc:oracle:thin:@192.168.0.3:1521:ORCL";
    private static final String USER = "scott"; // DB ID
    .....
    public Connection getConn() {
        Connection con = null;
        try {
            Class.forName(DRIVER);
            //암호화된 패스워드를 프로퍼티에서 읽어들어 복호화해서 사용해야한다.
            String PASS = props.getProperty("EncryptedPswd");
            byte[] decryptedPswd = cipher.doFinal(PASS.getBytes());
            PASS = new String(decryptedPswd);
            con = DriverManager.getConnection(URL, USER, PASS);
            .....
        }
    }
}
```

- 표준프레임워크 3.8부터 ARIA 블록암호 알고리즘 기반 암호/복호화 설정을 간소화할 수 있는 방법을 제공한다. 환경설정 파일인 `globals.properties`의 데이터베이스 연결 항목인 `Url`, `UserName`, `Password`에 대해 인코딩 키 생성 방법을 제공한다. 여기서 `algorithmKey`, `algorithmKeyHash` 의 노출을 피하고 싶다면 설정파일에서 이부분을 삭제하고 `WAS VM arguments` 환경 변수를 등록하면 된다.

context-crypto.xml 설정

```
<egov-crypto:config id="egovCryptoConfig"
  initial="true"
  crypto="true"
  algorithm="SHA-256"
  algorithmKey="(생성값)"
  algorithmKeyHash="(생성값)"
  cryptoBlockSize="1024"
  cryptoPropertyLocation = "classpath:/egovframework/egovProps/globals.properties"
/>
```

다음은 위의 context-crypto.xml¹⁰⁾을 기반으로 외부 환경설정 파일에 저장할 인코딩 값을 생성하는 java 파일의 예제이다.

인코딩 값 생성하는 java 파일

```
// 데이터베이스 연결 항목(URL, Username, Password) 인코딩 값 생성 JAVA
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import egovframework.rte.fdl.cryptography.EgovEnvCryptoService;
import egovframework.rte.fdl.cryptography.impl.EgovEnvCryptoServiceImpl;

public class EgovEnvCryptoUserTest {

    private static final Logger LOGGER = LoggerFactory.getLogger(EgovEnvCryptoUserTest.class);

    public static void main(String[] args) {

        String[] arrCryptoString = {
            "userId",          //데이터베이스 접속 계정 설정
            "userPassword",    //데이터베이스 접속 패스워드 설정
            "url",              //데이터베이스 접속 주소 설정
            "databaseDriver"    //데이터베이스 드라이버
        };

        LOGGER.info("-----");
        ApplicationContext context = new ClassPathXmlApplicationContext(new
String[] {"classpath:/context-crypto.xml"});
        EgovEnvCryptoService cryptoService = context.getBean(EgovEnvCryptoServiceImpl.class);
        LOGGER.info("-----");

        String label = "";
        try {
            for(int i=0; i < arrCryptoString.length; i++) {
                if(i==0)label = "사용자 아이디";
                if(i==1)label = "사용자 비밀번호";
                if(i==2)label = "접속 주소";
                if(i==3)label = "데이터 베이스 드라이버";
            }
        }
    }
}
```

10) <https://github.com/eGovFramework/egovframe-common-components/blob/master/src/main/resources/egovframework/spring/com/context-crypto.xml> 참조

```

        LOGGER.info(label+" 원본(original):" + arrCryptoString[i]);
        LOGGER.info(label+" 인코딩(encrypted):" +
        cryptoService.encrypt(arrCryptoString[i]));
        LOGGER.info("-----");
    }
} catch (IllegalArgumentException e) {
    LOGGER.error("[ "+e.getClass()+" ] IllegalArgumentException : " + e.getMessage());
} catch (Exception e) {
    LOGGER.error("[ "+e.getClass()+" ] Exception : " + e.getMessage());
}
}
}
}

```

생성된 인코딩 값을 외부에 저장된 환경설정 파일인 `globals.properties`¹¹⁾에 입력 및 저장한다.

안전한 코드의 예 (globals.properties 설정 예)

```

Globals.mysql.DriverClassName=net.sf.log4jdbc.DriverSpy
Globals.mysql.Url=jdbc:log4jdbc:mysql://127.0.0.1:3306/com
Globals.mysql.UserName = com
Globals.mysql.Password = xz4fmrSdr1vGGI6UtwPLwA%3D%3D

```

● 하드코드된 암호화 키

- 소스코드 내부에 암호화 키를 상수 형태로 하드코딩하여 사용하면 악의적인 공격자에게 암호화 키가 노출될 위험이 있다.

안전하지 않은 코드의 예

```

import javax.crypto.KeyGenerator;
import javax.crypto.spec.SecretKeySpec;
import javax.crypto.Cipher;
.....
public String encryptString(String usr) {
    //암호화 키를 소스코드 내부에 사용하는 것은 안전하지 않다.
    String key = "22df3023sf~2:asn!@#/>as";
    if (key != null) {
        byte[] bToEncrypt = usr.getBytes("UTF-8");
        SecretKeySpec sKeySpec = new SecretKeySpec(key.getBytes(), "AES");
    }
}

```

- 암호화 과정에 사용하는 암호화 키는 외부 공간(파일)에 안전한 방식으로 암호화하여 보관해야 하며, 암호화된 암호화 키는 복호화하여 사용한다.

11) <https://github.com/eGovFramework/egovframe-common-components/blob/master/src/main/resources/egovframework/egovProps/globals.properties> 참조

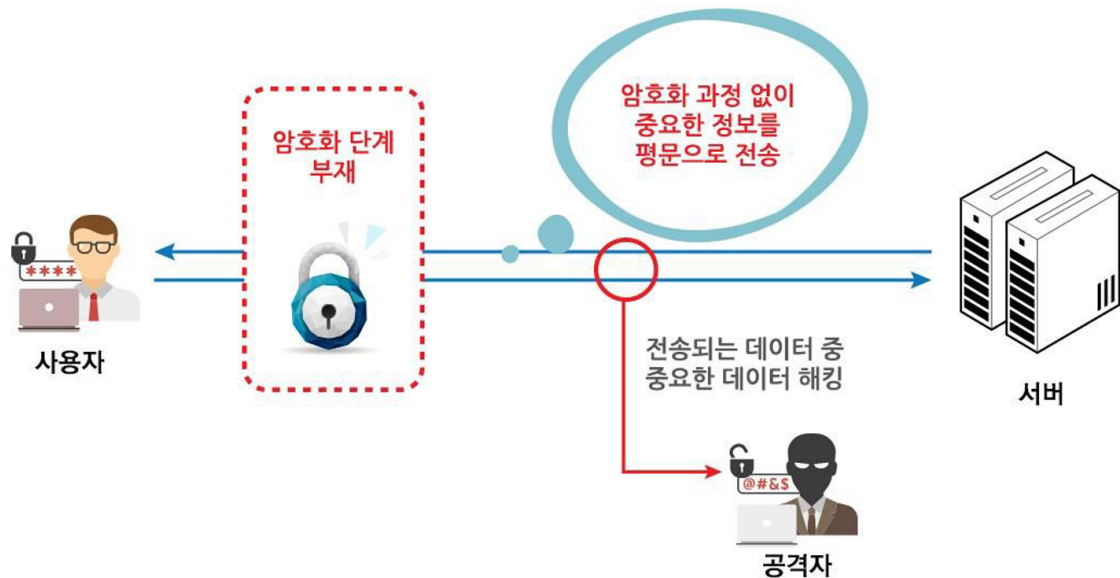
안전한 코드의 예

```
import javax.crypto.KeyGenerator;
import javax.crypto.spec.SecretKeySpec;
import javax.crypto.Cipher;
.....
public String encryptString(String usr) {
    //암호화 키는 외부 파일에서 암호화 된 형태로 저장하고, 사용시 복호화 한다.
    String key = getPassword("./password.ini");
    key = decrypt(key);
    if (key != null) {
        byte[] bToEncrypt = usr.getBytes("UTF-8");
        SecretKeySpec sKeySpec = new SecretKeySpec(key.getBytes(), "AES");
```

라. 참고자료

- CWE-259 Use of Hard-coded Password, MITRE,
<https://cwe.mitre.org/data/definitions/259.html>
- Be careful while handling sensitive data, such as passwords, in program code, CERT
<https://wiki.sei.cmu.edu/confluence/display/c/MS18-C.+Be+careful+while+handling+sensitive+data%2C+such+as+passwords%2C+in+program+code>
- Password Management: Hardcoded Password, OWASP
https://owasp.org/www-community/vulnerabilities/Password_Management_Hardcoded_Password
- 표준프레임워크 Crypto 간소화 서비스
https://www.egovframe.go.kr/wiki/doku.php?id=egovframework:rte2:fdl:crypto_simplify_v3_8
- CWE-321 Use of Hard-coded Cryptographic Key, MITRE,
<https://cwe.mitre.org/data/definitions/321.html>
- Use of hard-coded password, OWASP,
https://owasp.org/www-community/vulnerabilities/Use_of_hard-coded_password

7. 충분하지 않은 키 길이 사용



가. 개요

길이가 짧은 키를 사용하는 것은 암호화 알고리즘을 취약하게 만들 수 있다. 키는 암호화 및 복호화에 사용되는데, 검증된 암호화 알고리즘을 사용하더라도 키 길이가 충분히 길지 않으면 짧은 시간 안에 키를 찾아낼 수 있고 이를 이용해 공격자가 암호화된 데이터나 패스워드를 복호화할 수 있게 된다.

나. 보안대책

RSA 알고리즘은 적어도 2,048 비트 이상의 길이를 가진 키와 함께 사용해야 하고, 대칭암호화 알고리즘(Symmetric Encryption Algorithm)의 경우에는 적어도 128비트 이상의 키를 사용한다.

다. 코드예제

- 다음의 예제는 보안성이 강한 RSA 알고리즘을 사용함에도 불구하고, 키 사이즈를 작게 설정함으로써 프로그램의 보안 약점을 야기한 경우이다.

안전하지 않은 코드의 예

```

public static final String ALGORITHM = "RSA";
public static final String PRIVATE_KEY_FILE = "C:/keys/private.key";
public static final String PUBLIC_KEY_FILE = "C:/keys/public.key";
public static void generateKey() {
    try {
        final KeyPairGenerator keyGen = KeyPairGenerator.getInstance(ALGORITHM);
        //RSA 키 길이를 1024 비트로 짧게 설정하는 경우 안전하지 않다.
        keyGen.initialize(1024);
        final KeyPair key = keyGen.generateKeyPair();
        File privateKeyFile = new File(PRIVATE_KEY_FILE);
        File publicKeyFile = new File(PUBLIC_KEY_FILE);
    }
}

```

- 공개키 암호화에 사용하는 키의 길이는 적어도 2048비트 이상으로 설정한다.

안전한 코드의 예

```

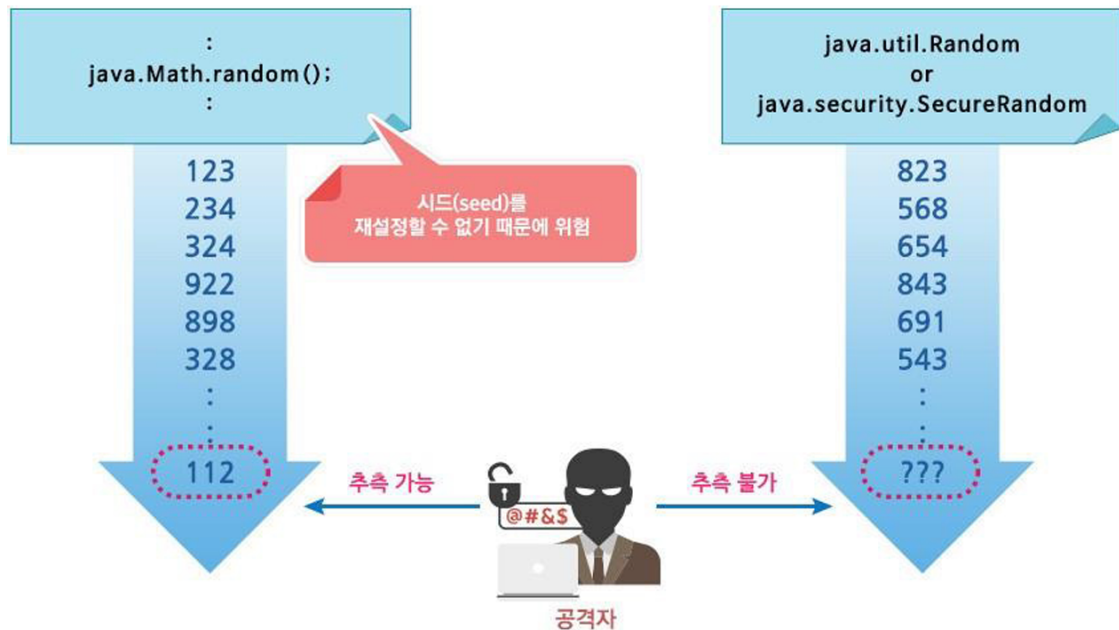
public static final String ALGORITHM = "RSA";
public static final String PRIVATE_KEY_FILE = "C:/keys/private.key";
public static final String PUBLIC_KEY_FILE = "C:/keys/public.key";
public static void generateKey() {
    try {
        final KeyPairGenerator keyGen = KeyPairGenerator.getInstance(ALGORITHM);
        keyGen.initialize(2048);
        final KeyPair key = keyGen.generateKeyPair();
        File privateKeyFile = new File(PRIVATE_KEY_FILE);
        File publicKeyFile = new File(PUBLIC_KEY_FILE);
    }
}

```

라. 참고자료

- CWE-326 Inadequate Encryption Strength, MITRE,
<https://cwe.mitre.org/data/definitions/326.html>

8. 적절하지 않은 난수값 사용



가. 개요

예측 가능한 난수를 사용하는 것은 시스템에 보안약점을 유발한다. 예측 불가능한 숫자가 필요한 상황에서 예측 가능한 난수를 사용한다면, 공격자는 SW에서 생성되는 다음 숫자를 예상하여 시스템을 공격하는 것이 가능하다.

나. 보안대책

- 컴퓨터의 난수발생기는 난수 값을 결정하는 시드(Seed)값이 고정될 경우, 매번 동일한 난수값이 발생 한다. 이를 최대한 피하기 위해 `Random()`과 `Math.random()` 사용 시 `java.util.Random` 클래스에서 기본값으로 현재시간을 기반으로 조합하여 매번 변경 되는 시드(Seed)값을 설정 하여야 한다.
- 그러나 세션 ID, 암호화키 등 보안결정을 위한 값을 생성하고 보안결정을 수행하는 경우에는, `Random()`과 `Math.random()`을 사용 하지 말아야 하며, 예측이 거의 불가능하게 암호학적으로 보호된 `java.security.SecureRandom` 클래스를 사용하는 것이 안전하다.

다. 코드예제

- java.util.Random 클래스의 random() 메소드 사용시, 고정된 seed를 설정하면 동일한 난수 값이 생성되어 안전하지 않다. 매번 변경되는 seed를 설정하더라도 보안결정을 위한 난수 이용시에는 안전하지 않다.

안전하지 않은 코드의 예

```
import java.util.Random;
...
public static int getRandomValue(int maxValue) {
    //고정된 시드값을 사용하여 동일한 난수값이 생성되어 안전하지 않다.
    Random random = new Random(100);
    return random.nextInt(maxValue);
}

public static String getAuthKey() {
    //매번 변경되는 시드값을 사용하여 다른 난수값이 생성되나 보안결정을 위한 난수로는 안전하지
    //않다.
    Random random = new Random();
    String authKey = Integer.toString(random.nextInt());
}
```

- java.util.Random 클래스는 setSeed를 통해 매번 변경되는 시드값을 설정 하거나, 현재 시간 기반 으로 매번 변경되는 별 도 seed를 설정하지 않는 기본값을 사용한다. 보안결정을 위 해 난수 사용 시에는 java.security.SecureRandom 클래스를 사용하는 것이 보다 안전하다.

안전한 코드의 예

```
import java.util.Random;
import java.security.SecureRandom;
...
public static int getRandomValue(int maxValue) {
    //setSeed를 통해 매번 변경되는 시드값을 설정 하거나, 기본값인 현재 시간 기반으로 매번
    //변경되는 시드값을 사용하도록 한다.
    Random random = new Random();
    return random.nextInt(maxValue);
}

public static String getAuthKey() {
    //보안결정을 위한 난수로는 예측이 거의 불가능하게 암호학적으로 보호된 SecureRandom을
    //사용한다.
    try{
        SecureRandom secureRandom = SecureRandom.getInstance("SHA1PRNG");
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
    }
```

```

secureRandom.setSeed(secureRandom.generateSeed(128));
String authKey = new String(digest.digest((secureRandom.nextLong() + "").getBytes()));
...
} catch (NoSuchAlgorithmException e) {

```

- 다음은 표준프레임워크 공통컴포넌트의 입력받은 일자 사이의 임의의 일자를 반환하는 예제¹²⁾이다.

안전한 코드의 예

```

import java.security.SecureRandom;

public static String getRandomDate(String sDate1, String sDate2) {
    String dateStr1 = validChkDate(sDate1);
    String dateStr2 = validChkDate(sDate2);

    String randomDate = null;

    int sYear, sMonth, sDay;
    int eYear, eMonth, eDay;

    sYear = Integer.parseInt(dateStr1.substring(0, 4));
    sMonth = Integer.parseInt(dateStr1.substring(4, 6));
    sDay = Integer.parseInt(dateStr1.substring(6, 8));

    eYear = Integer.parseInt(dateStr2.substring(0, 4));
    eMonth = Integer.parseInt(dateStr2.substring(4, 6));
    eDay = Integer.parseInt(dateStr2.substring(6, 8));

    GregorianCalendar beginDate = new GregorianCalendar(sYear, sMonth - 1, sDay, 0, 0);
    GregorianCalendar endDate = new GregorianCalendar(eYear, eMonth - 1, eDay, 23, 59);

    if (endDate.getTimeInMillis() < beginDate.getTimeInMillis()) {
        throw new IllegalArgumentException("Invalid input date : " + sDate1 + "~" + sDate2);
    }

    SecureRandom r = new SecureRandom();

    r.setSeed(new Date().getTime());

    long rand = ((r.nextLong() >>> 1) % (endDate.getTimeInMillis() - beginDate.getTimeInMillis()
    + 1)) + beginDate.getTimeInMillis();

    GregorianCalendar cal = new GregorianCalendar();
    //SimpleDateFormat calformat = new SimpleDateFormat("yyyy-MM-dd");
    SimpleDateFormat calformat = new SimpleDateFormat("yyyyMMdd", Locale.ENGLISH);
    cal.setTimeInMillis(rand);
    randomDate = calformat.format(cal.getTime());

    // 랜덤문자열을 리턴
    return randomDate;
}

```

12) <https://github.com/eGovFramework/egovframe-common-components/blob/master/src/main/java/egovframework/com/utl/fcc/service/EgovDateUtil.java> 참조

라. 참고자료

- CWE-330 Use of Insufficiently Random Values, MITRE,
<https://cwe.mitre.org/data/definitions/330.html>
- Generate strong random numbers, CERT,
[https://wiki.sei.cmu.edu/confluence/display/java/MS02-J.
+Generate+strong+random+numbers](https://wiki.sei.cmu.edu/confluence/display/java/MS02-J.+Generate+strong+random+numbers)
- Insecure Randomness, OWASP,
https://www.owasp.org/index.php/Insecure_Randomness

9. 취약한 비밀번호 허용

가. 개요

사용자에게 강한 패스워드 조합규칙을 요구하지 않으면, 사용자 계정이 취약하게 된다. 안전한 패스 워드를 생성하기 위해서는 「암호이용안내서」의 패스워드 설정규칙을 적용해야 한다.

나. 보안대책

패스워드 생성시 강한 조건 검증을 수행한다. 비밀번호(패스워드)는 숫자와 영문자, 특수문자 등을 혼합하여 사용하고, 주기적으로 변경하여 사용하도록 해야 한다.

다. 코드예제

- 가입자가 입력한 패스워드에 대한 복잡도 검증 없이 가입 승인 처리를 수행하고 있다.

안전하지 않은 코드의 예
<pre>String id = request.getParameter("id"); String pass = request.getParameter("pass"); UserVo userVO = new UserVo(id, pass); // 비밀번호의 자릿수, 특수문자 포함 여부 등 복잡도를 체크하지 않고 등록 String result = registerDAO.register(userVO);</pre>

- 사용자 계정을 보호하기 위해 가입 시, 패스워드 복잡도 검증 후 가입 승인처리를 수행한다.

안전한 코드의 예
<pre>String id = request.getParameter("id"); String pass = request.getParameter("pass"); //비밀번호에 자릿수, 특수문자 포함여부등의 복잡도를 체크하고 등록하게 한다. Pattern pattern = Pattern.compile("((?=.*[a-zA-Z])(?=.*[0-9@#%]). {9, })");</pre>

라. 참고자료

- CWE-521 Weak Password Requirements, MITRE,

<https://cwe.mitre.org/data/definitions/521.html>

- Password Complexity, OWASP

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

10. 부적절한 전자서명 확인

가. 개요

전자서명이란 서명자의 신원을 확인하고 서명된 파일의 무결성을 보장할 수 있는 디지털 정보이다. 전자서명이 사용된 경우, 전자서명을 검증하지 않거나 검증절차가 부적절하면 위변조된 파일로 악성코드에 감염될 수 있으므로 전자서명을 확인하여 위변조 여부를 판별하고 사용해야 한다.

나. 보안대책

전자서명을 포함하는 파일을 사용할 때는 항상 전자서명을 확인하여야 한다. 이 경우, 전자서명 파일의 출처 등을 확인하여 신뢰할 수 없는 곳에서 생성된 파일을 사용하지 않도록 한다.

다. 코드예제

- 다음 예제는 신뢰할 수 없는 곳에서 다운로드 한 JAR 파일의 서명을 확인하지 않고 사용한다. 이 경우, 악성코드가 삽입되어 실행될 수 있다.

안전하지 않은 코드의 예

```
File f = new File(downloadedFilePath);  
JarFile jf = new JarFile(f);
```

- 아래 예제는 JarFile 생성자에 boolean형 파라미터를 사용하여 전자서명을 확인한다. 전자서명 여부를 확인한 후, JarEntry.getCodeSigners()메소드를 사용하여 JAR 객체에 대한 전자서명 주체를 신뢰할 수 있는지 확인하여야 한다.

안전한 코드의 예

```
File f = new File(downloadedFilePath);  
JarFile jf = new JarFile(f, true);  
Enumeration<JarEntry> ens = jf.entries();  
while (ens.hasMoreElements()) {
```

```
JarEntry en = ens.nextElement();
    if (!en.isDirectory()) {
        if (en.toString().equals(path)) {
            byte[] data = readAll(jar.getInputStream(en), en.getSize());
            CodeSigner[] signers = en.getCodeSigners();
            ...
        }
    }
}
jf.close();
```

라. 참고자료

- CWE-347 : Improper Verification of Cryptographic Signature, MITRE,
<https://cwe.mitre.org/data/definitions/347.html>

11. 부적절한 인증서 유효성 검증

가. 개요

인증서를 확인하지 않거나 인증서 확인 절차를 적절하게 수행하지 않아, 악의적인 호스트에 연결되거나 신뢰할 수 없는 호스트에서 생성된 데이터를 수신하게 되는 보안약점이다.

나. 보안대책

인증서를 사용하기 전에 인증서의 유효성을 확인한다. 인증서의 Common Name과 실제 호스트가 일치하는지, 신뢰된 발급기관(CA, RootCA)의 서명 여부, 인증서의 유효기간, 인증서의 해지여부, 안전한 암호화 알고리즘 사용 여부 확인 등으로 유효한 인증서인지 검증하는 절차를 구현하여야 한다.

다. 코드예제

- 아래 예제는 인증서 DN 일치여부와 유효기간 등을 검증한다.

안전한 코드의 예

```
private boolean verifySignature(X509Certificate toVerify, X509Certificate signingCert) {  
    /* 검증하려는 호스트 인증서(toVerify)와 CA인증서(signing Cert)의 DN(Distinguished Name)이  
    일치하는지 여부를 확인한다.*/  
    if (!toVerify.getIssuerDN().equals(signingCert.getSubjectDN()))  
        return false;  
    try {  
        // 호스트 인증서가 CA인증서로 서명 되었는지 확인한다.  
        toVerify.verify(signingCert.getPublicKey());  
        // 호스트 인증서가 유효기간이 만료되었는지 확인한다.  
        toVerify.checkValidity();  
        return true;  
    } catch (GeneralSecurityException verifyFailed) {  
        return false;  
    }  
}
```

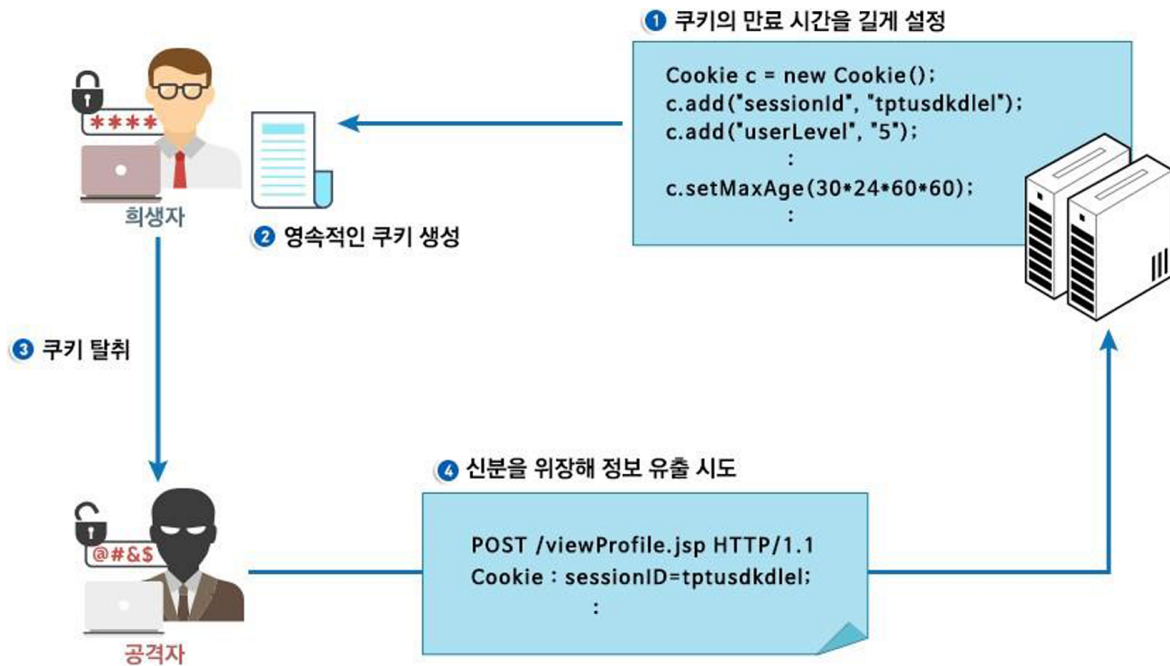
- 또한, 유효기간이 남아 있는 인증서의 해지여부를 확인하기 위해서는 CRL(Certificate revocation lists)로 인증서 해지목록 또는 OCSP(Online Certificate Status Protocol)로 실시간 인증서 상태확인이 필요하다. CRL 리스트는 해당 인증서를 참

조하여 인증기관에서 다운받을 수 있으며, 다운받은 CRL로
해지된 인증서를 확인할 수 있다.

라. 참고자료

- CWE-295: Improper Certificate Validation, MITRE,
<https://cwe.mitre.org/data/definitions/295.html>

12. 사용자 하드디스크에 저장되는 쿠키를 통한 정보노출



가. 개요

대부분의 웹 응용프로그램에서 쿠키는 메모리에 상주하며, 브라우저의 실행이 종료되면 사라진다. 프로그래머가 원하는 경우, 브라우저 세션에 관계없이 지속적으로 저장되도록 설정할 수 있으며, 이것은 디스크에 기록되고, 다음 브라우저 세션이 시작되었을 때 메모리에 로드된다. 개인정보, 인증 정보 등이 이와 같은 영속적인 쿠키(Persistent Cookie)에 저장된다면, 공격자는 쿠키에 접근할 수 있는 보다 많은 기회를 가지게 되며, 이는 시스템을 취약하게 만든다.

나. 보안대책

쿠키의 만료시간은 세션이 지속되는 시간을 고려하여 최소한으로 설정하고 영속적인 쿠키에는 사용자 권한 등급, 세션 ID 등 중요정보가 포함되지 않도록 한다.

다. 코드예제

- 쿠키의 만료시간을 1년으로 과도하게 길게 설정하고 있다.

쿠키의 유효기간이 긴 경우 사용자 하드 디스크에 쿠키가 저장되며 저장된 쿠키는 쉽게 도용될 수 있으므로 취약하다.

안전하지 않은 코드의 예

```
Cookie loginCookie = new Cookie("rememberme", "YES");
//쿠키의 만료시간을 1년으로 과도하게 길게 설정하고 있어 안전하지 않다.
loginCookie.setMaxAge(60*60*24*365);
response.addCookie(loginCookie);
```

- 쿠키의 만료시간은 해당 기능에 맞춰 최소로 설정하고 영속적인 쿠키에는 중요 정보가 포함되지 않도록 한다.

안전한 코드의 예

```
Cookie loginCookie = new Cookie("rememberme", "YES");
//쿠키의 만료시간은 해당 기능에 맞춰 최소로 사용한다.
loginCookie.setMaxAge(60*60*24);
response.addCookie(loginCookie);
```

- 다음은 표준프레임워크 공통컴포넌트의 쿠키의 유효시간을 설정하는 예제¹³⁾이다.

안전한 코드의 예

```
public static void setCookie(HttpServletResponse response, String cookieNm, String cookieVal,
int minute) throws UnsupportedEncodingException {

    // 특정의 encode 방식을 사용해 캐릭터 라인을 application/x-www-form-urlencoded 형식으로
    변환
    // 일반 문자열을 웹에서 통용되는 'x-www-form-urlencoded' 형식으로 변환하는 역할
    String cookieValue = URLEncoder.encode(cookieVal, "utf-8");

    // 쿠키생성 - 쿠키의 이름, 쿠키의 값
    Cookie cookie = new Cookie(cookieNm, cookieValue);

    cookie.setSecure(true);

    // 쿠키의 유효시간 설정
    int maxAge = 60 * minute;
    if(maxAge > 60 * 60 * 24) {
        maxAge = 60 * 60 * 24;
    }
    cookie.setMaxAge(maxAge);

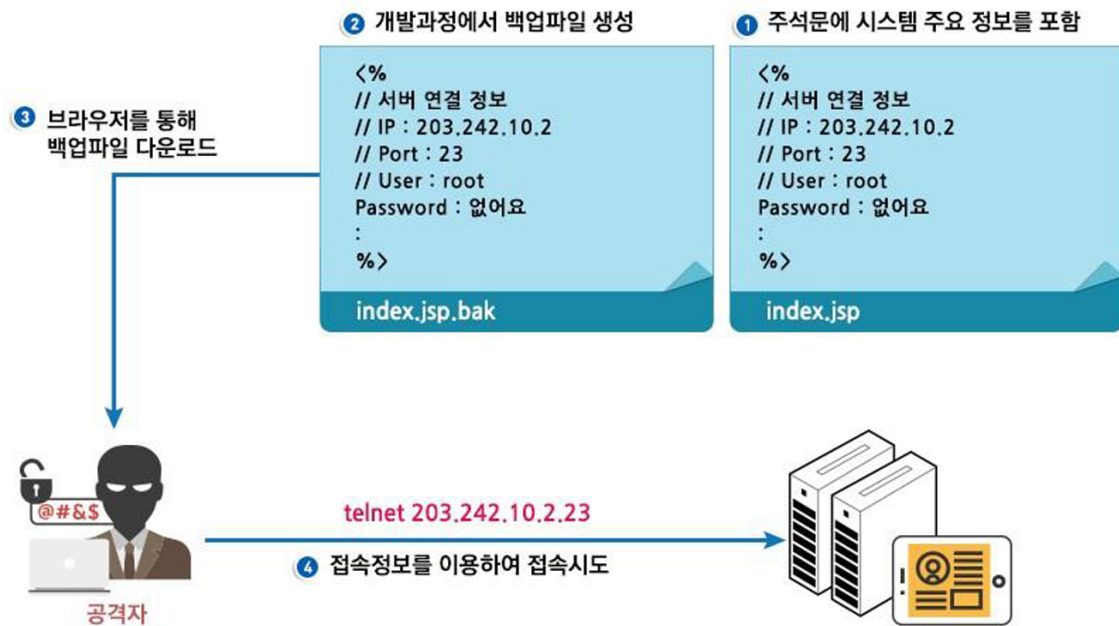
    // response 내장 객체를 이용해 쿠키를 전송
    response.addCookie(cookie);
}
```

13) <https://github.com/eGovFramework/egovframe-common-components/blob/master/src/main/java/egovframework/com/utl/cas/service/EgovSessionCookieUtil.java> 참조

라. 참고자료

- CWE-539 Information Exposure Through Persistent Cookies, MITRE,
<https://cwe.mitre.org/data/definitions/539.html>
- Do not store unencrypted sensitive information on the client side, CERT,
<https://wiki.sei.cmu.edu/confluence/display/java/FIO52-J.+Do+not+store+unencrypted+sensitive+information+on+the+client+side>
- Expire and Max-Age Attributes, OWASP,
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

13. 주석문 안에 포함된 시스템 주요정보



가. 개요

소스 코드 내의 주석문에 비밀번호를 저장하는 것은 시스템 보안에 위험을 초래할 수 있다. 소프트웨어 개발자가 편의를 위해 주석문에 비밀번호를 기록하는 경우, 소프트웨어가 완성된 후 이를 제거하기가 어려워진다. 또한, 공격자가 소스 코드에 접근할 수 있다면 시스템 침입의 가능성이 있다.

나. 보안대책

주석에는 아이디, 패스워드 등 보안과 관련된 내용을 기입하지 않는다.

다. 코드예제

- 다음 예제는 개발자의 이해를 돕기 위한 목적 등 편리성을 위해 비밀번호를 주석문 안에 서술하고 제대로 지우지 않아서 보안약점이 발생한 경우이다.

안전하지 않은 코드의 예

```
//주석문을 통해 DB연결 ID, 패스워드의 중요한 정보를 노출시켜 안전하지 않다.
```

```
// DB연결 root / a1q2w3r3f2!@  
con = DriverManager.getConnection(URL, USER, PASS);
```

- 프로그램 개발 시에 주석문 등에 남겨놓은 사용자 계정이나 패스워드 등의 정보는 개발 완료 시에 반드시 삭제하여야 한다.

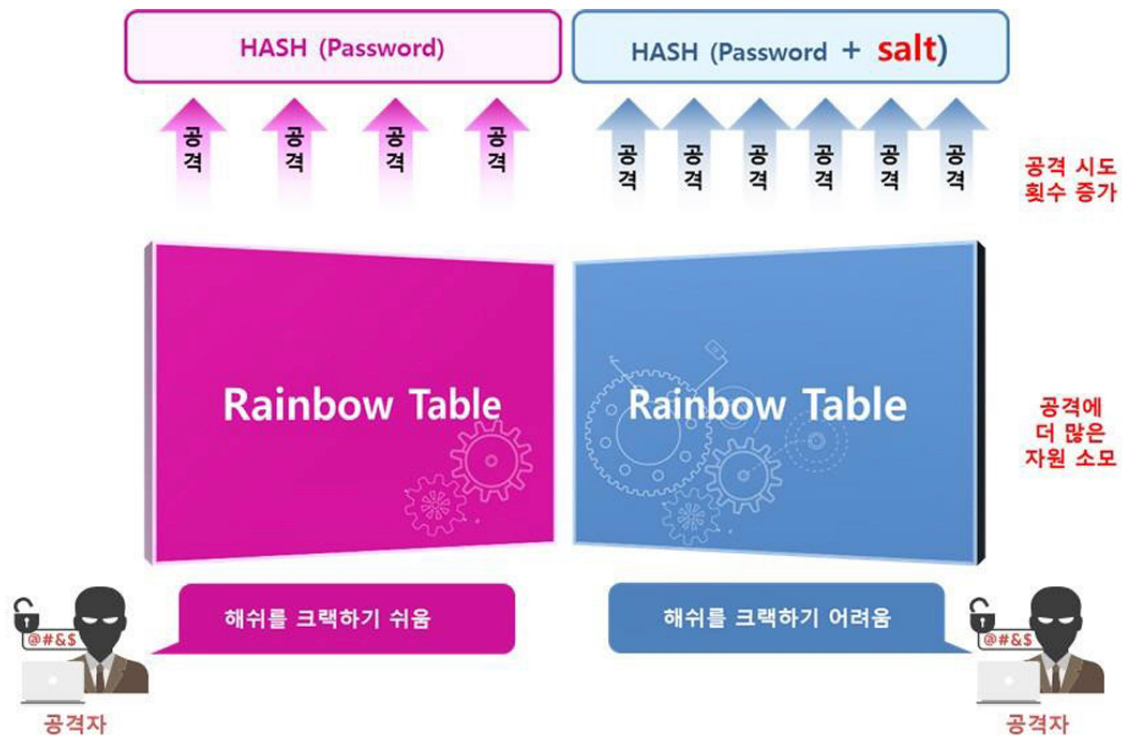
안전한 코드의 예

```
// ID, 패스워드등의 중요 정보는 주석에 포함해서는 안된다.  
con = DriverManager.getConnection(URL, USER, PASS);
```

라. 참고자료

- CWE-615 Information Exposure Through Comments, MITRE, <https://cwe.mitre.org/data/definitions/615.html>

14. 솔트 없이 일방향 해쉬함수 사용



가. 개요

패스워드를 저장 시 일방향 해쉬함수의 성질을 이용하여 패스워드의 해쉬값을 저장한다. 만약 패스워드를 솔트(Salt)없이 해쉬하여 저장한다면, 공격자는 레인보우 테이블과 같이 해쉬값을 미리 계산 하여 패스워드를 찾을 수 있게 된다.

나. 보안대책

패스워드를 저장 시 패스워드와 솔트를 해쉬함수의 입력으로 하여 얻은 해쉬값을 저장한다.

다. 코드예제

- 다음의 예제는 패스워드 저장 시 솔트 없이 패스워드에 대한 해쉬값을 얻는 과정을 보여준다.

```
안전하지 않은 코드의 예

public String getPasswordHash(String password) throws Exception {
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    //해쉬에 솔트를 적용하지 않아 안전하지 않다.
    md.update(password.getBytes());
```

```

byte byteData[] = md.digest();
StringBuffer hexString = new StringBuffer();
for (int i=0; i<byteData.length i++) {
    String hex=Integer.toHexString(0xff & byteData[i]);
    if (hex.length() == 1) {
        hexString.append('0');
    }
    hexString.append(hex);
}
return hexString.toString();
}

```

- 패스워드만을 해쉬함수의 입력으로 사용하기에 레인보우 테이블을 이용한 사전 공격이 가능하며, 이를 방지하기 위해 패스워드와 솔트를 함께 해쉬함수에 적용하여 사용한다.

안전한 코드의 예

```

public String getPasswordHash(String password, byte[] salt) throws Exception {
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    md.update(password.getBytes());
    //해시사용시에는 원문을 찾을 수 없도록 솔트를 사용하여야한다.
    md.update(salt);
    byte byteData[] = md.digest();
    StringBuffer hexString = new StringBuffer();
    for (int i=0; i<byteData.length i++) {
        String hex=Integer.toHexString(0xff & byteData[i]);
        if (hex.length() == 1) {
            hexString.append('0');
        }
        hexString.append(hex);
    }
    return hexString.toString()
}

```

- 다음은 비밀번호를 단방향으로 암호화할 때 솔트값을 포함하여 저장하는 예제14)이다.

안전한 코드의 예

```

public static String encryptPassword(String data, byte[] salt) throws Exception {
    if (data == null) {
        return "";
    }
    byte[] hashValue = null; // 해쉬값
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    md.reset();
    md.update(salt);
    hashValue = md.digest(data.getBytes());
    return new String(Base64.encodeBase64(hashValue));
}

```

라. 참고자료

- CWE-759, Use of a One-Way Hash without a Salt, MITRE,
<https://cwe.mitre.org/data/definitions/759.html>
- Store passwords using a hash function, CERT,
<https://wiki.sei.cmu.edu/confluence/display/java/MS62-J.+Store+passwords+using+a+hash+function>
- Use_a_cryptographically_strong_credential-specific_salt, OWASP,
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

14) <https://github.com/eGovFramework/egovframe-common-components/blob/master/src/main/java/egovframework/com/utl/sim/service/EgovFileScrtty.java> 참조

15. 무결성 검사 없는 코드 다운로드

가. 개요

원격으로부터 소스코드 또는 실행파일을 무결성 검사 없이 다운로드 받고, 이를 실행하는 제품들이 종종 존재한다. 이는 호스트 서버의 변조, DNS 스푸핑 (Spoofing) 또는 전송 시의 코드 변조 등의 방법을 이용하여 공격자가 악의적인 코드를 실행할 수 있도록 한다.

나. 보안대책

DNS 스푸핑(Spoofing)을 방어할 수 있는 DNS lookup을 수행하고 코드 전송 시 신뢰할 수 있는 암호 기법을 이용하여 코드를 암호화한다. 또한 다운로드한 코드는 작업 수행을 위해 필요한 최소한의 권한으로 실행하도록 한다.

다. 코드예제

- 이 예제는 URLClassLoader()를 통해 원격에서 파일을 다운로드한 뒤 로드하면서, 대상 파일에 대한 무결성 검사를 수행하지 않아 파일변조 등으로 인한 피해가 발생할 수 있는 경우이다. 이러한 경우 공격자는 악의적인 실행코드로 클래스의 내용을 수정할 수 있다.

안전하지 않은 코드의 예

```
URL[] classURLs = new URL[] { new URL("file:subdir/") };
URLClassLoader loader = new URLClassLoader(classURLs);
Class loadedClass = Class.forName("LoadMe", true, loader);
```

- 이를 안전한 코드로 변환하면 다음과 같다. 클래스를 로드하기 전 클래스의 체크섬(Checksum)을 실행하여 로드하는 코드가 변조되지 않았음을 확인한다.

안전한 코드의 예

```
// 공개키 방식의 암호화 알고리즘과 메커니즘을 이용하여 전송파일에 대한 시그니처를 생성하고
// 파일의 변조유무를 판단한다. 서버에서는 Private Key를 가지고 MyClass를 암호화한다.
```



```

String jarFile = "./download/util.jar";
byte[] loadFile = FileManager.getBytes(jarFile);
loadFile = encrypt(loadFile, privateKey);
// jarFileName으로 암호화된 파일을 생성한다.
FileManager.createFile(loadFile, jarFileName);
// 클라이언트에서는 파일을 다운로드 받을 경우 Public Key로 복호화한다.
URL[] classURLs = new URL[] { new URL("http://filesave.com/download/util.jar") };
URLConnection conn = classURLs.openConnection();
InputStream is = conn.getInputStream();
// 입력 스트림을 jarFile명으로 파일을 출력한다.
FileOutputStream fos = new FileOutputStream(new File(jarFile));
While (is.read(buf) != -1) {
    .....
}
byte[] loadFile = FileManager.getBytes(jarFile);
loadFile = decrypt(loadFile, publicKey);
// 복호화된 파일을 생성한다.
FileManager.createFile(loadFile, jarFile);
URLClassLoader loader = new URLClassLoader(classURLs);
Class loadedClass = Class.forName("MyClass", true, loader);

```

라. 참고자료

- CWE-494 Download of Code Without Integrity Check, MITRE, <https://cwe.mitre.org/data/definitions/494.html>
- Do not rely on the default automatic signature verification provided by URLClassLoader and java.util.jar, CERT, <https://wiki.sei.cmu.edu/confluence/display/java/SEC06-J.+Do+not+rely+on+the+default+automatic+signature+verification+provided+by+URLClassLoader+and+java.util.jar>

16. 반복된 인증시도 제한 기능 부재

가. 개요

일정 시간 내에 여러 번의 인증을 시도하여도 계정잠금 또는 추가 인증 방법 등의 충분한 조치가 수행 되지 않는 경우, 공격자는 성공할법한 ID와 비밀번호들을 사전 (Dictionary)으로 만들고 무차별 대입 (brute-force)하여 로그인 성공 및 권한획득이 가능하다.

나. 보안대책

인증시도 횟수를 적절한 횟수로 제한하고 설정된 인증실패 횟수를 초과했을 경우 계정을 잠금하거나 추가적인 인증과정을 거쳐서 시스템에 접근이 가능하도록 한다.

다. 코드예제

- 다음 예제는 로그인 정보를 잘못 입력하였을 경우 다시 입력을 시도하는데 있어 제한이 없다. 따라서 공격자는 여러 가지 비밀번호로 인증을 재시도하여 올바른 비밀번호를 알아내고 로그인에 성공할 수 있다.

안전하지 않은 코드의 예

```
private static final String SERVER_IP = "127.0.0.1";
private static final int SERVER_PORT = 8080;
private static final int FAIL = -1;
public void login() {
    String username = null;
    String password = null;
    Socket socket = null;
    int result = FAIL;
    try {
        socket = new Socket(SERVER_IP, SERVER_PORT);
        //인증 실패에 대해 제한을 두지 않아 안전하지 않다.
        while (result == FAIL) {
            ...
            result = verifyUser(username, password);
        }
    }
```

- 다음 예제는 사용자 인증시도 횟수를 기록하는 MAX_ATTEMPTS 변수를 정의하고, 이를 인증시도 횟수를

제한하는 카운터로 사용함으로써 무차별 공격에 대응하는 코드이다.

안전한 코드의 예

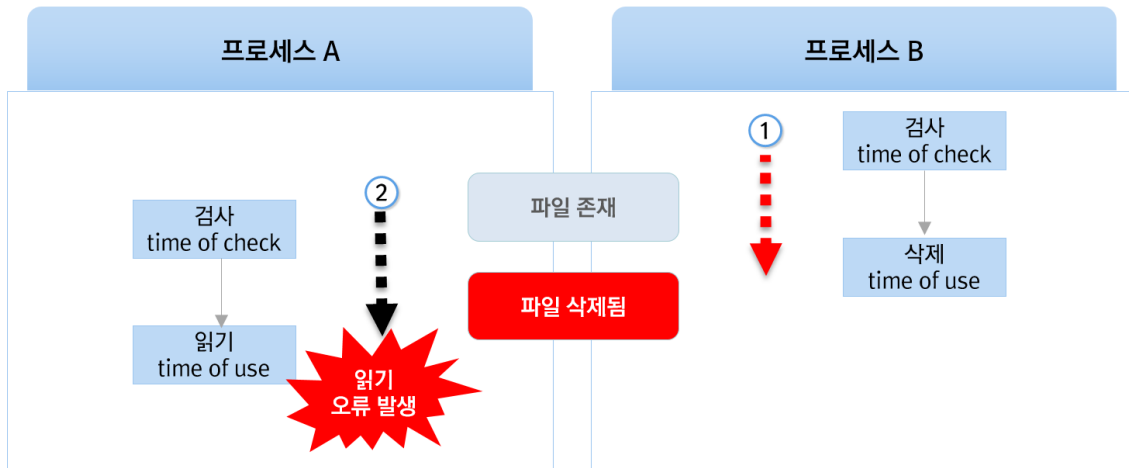
```
private static final String SERVER_IP = "127.0.0.1";
private static final int SERVER_PORT = 8080;
private static final int FAIL = -1;
private static final int MAX_ATTEMPTS = 5;
public void login() {
    String username = null;
    String password = null;
    Socket socket = null;
    int result = FAIL;
    int count = 0;
    try {
        socket = new Socket(SERVER_IP, SERVER_PORT);
        //인증 실패 및 시도 횟수에 제한을 두어 안전하다.
        while (result == FAIL && count < MAX_ATTEMPTS) {
            ...
            result = verifyUser(username, password);
            count++;
        }
    }
```

라. 참고자료

- CWE-307, Improper Restriction of Excessive Authentication Attempts, MITRE,
<https://cwe.mitre.org/data/definitions/307.html>
- Blocking Brute Force Attacks, OWASP,
https://www.owasp.org/index.php/Blocking_Brute_Force_Attacks

제3절 시간 및 상태

동시 또는 거의 동시 수행을 지원하는 병렬 시스템이나 하나 이상의 프로세스가 동작되는 환경에서 시간 및 상태를 부적절하게 관리하여 발생할 수 있는 보안약점이다.



1. 경쟁조건: 검사시점과 사용시점(TOCTOU Race Condition)

가. 개요

- 병렬시스템(멀티프로세스로 구현한 응용프로그램)에서는 자원(파일, 소켓 등)을 사용하기에 앞서 자원의 상태를 검사한다. 하지만, 자원을 사용하는 시점과 검사하는 시점이 다르기 때문에, 검사하는 시점(Time Of Check)에 존재하던 자원이 사용하던 시점(Time Of Use)에 사라지는 등 자원의 상태가 변하는 경우가 발생한다.
- 예를 들어, 프로세스 A와 B가 존재하는 병렬시스템 환경에서 프로세스 A는 자원사용(파일 읽기)에 앞서 해당 자원(파일)의 존재 여부를 검사(TOC) 한다. 이때는 프로세스 B가 해당 자원(파일)을 아직 사용(삭제)하지 않았기 때문에, 프로세스 A는 해당 자원(파일)이 존재한다고 판단한다. 그러나 프로세스 A가 자원 사용(파일읽기)을 시도하는 시점(TOU)에 해당 자원(파일)은 사용불가능 상태이기 때문에 오류 등이 발생할 수 있다.

- 이와 같이 하나의 자원에 대하여 동시에 검사시점과 사용시점이 달라 생기는 보안약점으로 인해 동기화 오류뿐만 아니라 교착상태 등과 같은 문제점이 발생할 수 있다.

나. 보안대책

공유자원(예: 파일)을 여러 프로세스가 접근하여 사용할 경우, 동기화 구문을 사용하여 한 번에 하나의 프로세스만 접근 가능하도록(synchronized, mutex 등) 하는 한편, 성능에 미치는 영향을 최소화하기 위해 임계코드 주변만 동기화 구문을 사용한다.

다. 코드예제

- 다음의 예제는 파일을 대한 읽기와 삭제가 두 개의 스레드에 동작하게 되므로 이미 삭제된 파일을 읽으려고 하는 레이 스컨디션¹⁵⁾이 발생할 수 있다.

안전하지 않은 코드의 예

```
class FileMgmtThread extends Thread {
    private String manageType = "";
    public FileMgmtThread (String type) {
        manageType = type;
    }
    //멀티스레드 환경에서 공유자원에 여러프로세스가 사용하여 동시에 접근할 가능성이 있어
    안전하지 않다.
    public void run() {
        try {
            if (manageType.equals("READ")) {
                File f = new File("Test_367.txt");
                if (f.exists()) {
                    BufferedReader br = new BufferedReader(new FileReader(f));
                    br.close();
                }
            } else if (manageType.equals("DELETE")) {
                File f = new File("Test_367.txt");
                if (f.exists()) {
                    f.delete();
                } else { ... }
            }
        } catch (IOException e) { ... }
    }
}

public class CWE367 {
    public static void main (String[] args) {
        FileMgmtThread fileAccessThread = new FileMgmtThread("READ");
    }
}
```

```

FileMgmtThread fileDeleteThread = new FileMgmtThread("DELETE");
//파일의 읽기와 삭제가 동시에 수행되어 안전하지 않다.
fileAccessThread.start();
fileDeleteThread.start();
}
}

```

- 따라서 다음 예제와 같이 동기화 구문인 synchronized를 사용하여 공유자원 (Test_367.txt)에 대한 안전한 읽기/쓰기를 수행할 수 있도록 한다.

안전한 코드의 예

```

class FileMgmtThread extends Thread {
    private String manageType = "";
    public FileMgmtThread (String type) {
        manageType = type;
    }
    //멀티쓰레드 환경에서 synchronized를 사용하여 동시에 접근할 수 없도록 사용해야한다.
    public void run() {
        synchronized(SYNC) {
            try {
                if (manageType.equals("READ")) {
                    File f = new File("Test_367.txt");
                    if (f.exists()) {
                        BufferedReader br = new BufferedReader(new FileReader(f));
                        br.close();
                    }
                } else if (manageType.equals("DELETE")) {
                    File f = new File("Test_367.txt");
                    if (f.exists()) {
                        f.delete();
                    } else { ... }
                }
            } catch (IOException e) { ... }
        }
    }
}

public class CWE367 {
    public static void main (String[] args) {
        FileMgmtThread fileAccessThread = new FileMgmtThread("READ");
        FileMgmtThread fileDeleteThread = new FileMgmtThread("DELETE");
        fileAccessThread.start();
        fileDeleteThread.start();
    }
}

```

- 다음은 표준프레임워크 공통컴포넌트의 드라이버 로드시 동시

15) 레이스컨디션(Race Condition): Race Condition은 두 개 이상의 프로세스가 공유 자원을 병행적으로(concurrently) 읽거나 쓸 때, 공유 데이터에 대한 접근이 어떤 순서에 따라 이루어졌는지에 따라 그 실행 결과가 달라지는 상황을 말한다.

에 접근 할 수 없도록 선점권을 보장하는 예제¹⁶⁾이다.

안전한 코드의 예

```
protected static synchronized void loadDriver() {
    BasicDataSource bds = new BasicDataSource();

    bds.setDriverClassName(JDBC_DRIVER);
    bds.setUrl(JDBC_URL);
    bds.setUsername(JDBC_USER);
    bds.setPassword(JDBC_PASSWORD);
    bds.setMaxTotal(MAX_TOTAL);
    bds.setMaxIdle(MAX_IDLE);
    bds.setMinIdle(MIN_IDLE);
    bds.setMaxWaitMillis(MAX_WAIT_MILLIS);
    bds.setDefaultAutoCommit(DEFAULT_AUTOCOMMIT);
    bds.setDefaultReadOnly(DEFAULT_READONLY);

    createPools(JDBC_ALIAS, bds);
    isDriverLoaded = true;
    LOGGER.info("Initialized pool : {}", JDBC_ALIAS);
}
```

- 다음은 표준프레임워크 배치실행환경의 여러 쓰레드에 접근하는 환경에서 선점권을 보장하는 예제¹⁷⁾이다.

안전한 코드의 예

```
public synchronized void write(List<? extends T> items) throws Exception {
    if (!getOutputState().isInitialized()) {
        throw new WriterNotOpenException("Writer must be open before it can be written to");
    }

    // slf4J logger 로 변경, logger.isDebugEnabled() 삭제 : 2014.04.30
    LOGGER.info("Writing to flat file with {} items", items.size());

    OutputState state = getOutputState();
    StringBuilder lines = new StringBuilder();
    int lineCount = 0;

    for (T item : items) {
        lines.append(lineAggregator.aggregate(item) + lineSeparator);
        lineCount++;
    }

    try {
        state.write(lines.toString());
    } catch (IOException e) {
        throw new WriteFailedException("Could not write data. The file may be corrupt.", e);
    }
}
```

16) <https://github.com/eGovFramework/egovframe-common-components/blob/master/src/main/java/egovframework/com/cop/sms/service/impl/SmsBasicDBUtil.java> 참조

17)

<https://github.com/eGovFramework/egovframe-runtime/blob/master/Batch/org.egovframe.rte.bat.core/src/main/java/org/egovframe/rte/bat/core/item/file/EgovPartitionFlatFileItemWriter.java> 참조

```
state.linesWritten += lineCount;  
}
```

라. 참고자료

- CWE-367 Time-of-check Time-of-use(TOCTOU) Race Condition, MITRE
<https://cwe.mitre.org/data/definitions/367.html>
- Avoid TOCTOU race conditions while accessing files, CERT
<https://wiki.sei.cmu.edu/confluence/display/c/FIO45-C.+Avoid+TOCTOU+race+conditions+while+accessing+files>

2. 종료되지 않는 반복문 또는 재귀함수

가. 개요

재귀의 순환횟수를 제어하지 못하여 할당된 메모리나 프로그램 스택 등의 자원을 과다하게 사용하면 위험하다. 대부분의 경우, 귀납 조건(Base Case)이 없는 재귀 함수는 무한 루프에 빠져 들게 되고 자원고갈을 유발함으로써 시스템의 정상적인 서비스를 제공할 수 없게 한다.

나. 보안대책

모든 재귀 호출 시, 재귀 호출 횟수를 제한하거나, 초기값을 설정(상수)하여 재귀 호출을 제한해야 한다.

다. 코드예제

- factorial 함수는 함수 내부에서 자신을 호출하는 재귀함수로, 재귀문을 빠져 나오는 조건을 정의하고 있지 않아 무한 재귀에 빠져 시스템 장애를 유발할 수 있다.

안전하지 않은 코드의 예

```
public int factorial(int n)
{
    //재귀 호출이 조건문/반복문 블록 외부에서 일어나면 대부분 무한 재귀를 유발한다.
    return n * factorial(n - 1);
}
```

- 모든 재귀 호출은 조건문이나 반복문 블록 안에서 이루어져야 한다.

안전한 코드의 예

```
public int factorial(int n)
{
    int i;
    //모든 재귀 호출은 조건문이나 반복문 블록 안에서 이루어져야한다.
    if (n == 1)
    {
        i = 1;
    }
    else
    {
```

```
i = n * factorial(n - 1);  
}  
return i;  
}
```

라. 참고자료

- CWE-674 Uncontrolled Recursion,
<https://cwe.mitre.org/data/definitions/674.html>

제4절 에러 처리

에러를 처리하지 않거나, 불충분하게 처리하여 에러 정보에 중
요정보(시스템 내부정보 등)가 포함될 때, 발생할 수 있는 취약
점으로 에러를 부적절하게 처리하여 발생하는 보안약점이다.

1. 오류 메시지를 통한 정보노출



가. 개요

응용프로그램이 실행환경, 사용자 등 관련 데이터에 대한 민감한 정보를 포함하는 오류 메시지를 생성하여 외부에 제공하는 경우, 공격자의 악성 행위를 도울 수 있다. 예외발생 시 예외이름이나 스택 트레이스를 출력하는 경우, 프로그램 내부구조를 쉽게 파악할 수 있기 때문이다.

나. 보안대책

오류 메시지는 정해진 사용자에게 유용한 최소한의 정보만 포함하도록 한다. 소스코드에서 예외 상황은 내부적으로 처리하고 사용자에게 민감한 정보를 포함하는 오류를 출력하지 않도록 미리 정의된 메시지를 제공하도록 설정한다.

다. 코드예제

- 다음 예제는 오류메시지에 예외 이름이나 오류추적 정보를 출력하여 프로그램 내부 정보가 유출되는 경우이다.

안전하지 않은 코드의 예
<pre>try { rd = new BufferedReader(new FileReader(new File(filename))); } catch(IOException e) { // 예외 메시지를 통해 스택 정보가 노출됨 e.printStackTrace(); }</pre>

- 아래 코드와 같이 예외 이름이나 오류추적 정보를 출력하지 않도록 한다.

안전한 코드의 예
<pre>try { rd = new BufferedReader(new FileReader(new File(filename))); } catch(IOException e) { // 예외 코드와 정보를 별도로 정의하고 최소 정보만 로깅 logger.error("ERROR-01: 파일 열기 예외"); }</pre>

라. 참고자료

- CWE-209 Information Exposure Through an Error Message, MITRE,
<https://cwe.mitre.org/data/definitions/209.html>
- Do not allow exceptions to expose sensitive information, CERT,
<https://wiki.sei.cmu.edu/confluence/display/java/ERR01-J.+Do+not+allow+exceptions+to+expose+sensitive+information>
- Error Handling, OWASP
https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html

2. 오류 상황 대응 부재

가. 개요

오류가 발생할 수 있는 부분을 확인하였으나, 이러한 오류에 대하여 예외 처리를 하지 않을 경우, 공격자는 오류 상황을 악용하여 개발자가 의도하지 않은 방향으로 프로그램이 동작하도록 할 수 있다.

나. 보안대책

오류가 발생할 수 있는 부분에 대하여 제어문을 사용하여 적절하게 예외 처리(try-catch 등)를 한다.

다. 코드예제

- 다음 예제는 try 블록에서 발생하는 오류를 포착(catch)하고 있지만, 그 오류에 대해서 아무 조치를 하고 있지 않음을 보여준다. 아무 조치가 없으므로 프로그램이 계속 실행되기 때문에 프로그램에서 어떤 일이 일어났는지 전혀 알 수 없게 된다.

안전하지 않은 코드의 예

```
protected Element createContent(WebSession s) {
    .....
    try {
        username = s.getParser().getRawParameter(USERNAME);
        password = s.getParser().getRawParameter(PASSWORD);
        if (!"webgoat".equals(username) || !password.equals("webgoat")) {
            s.setMessage("Invalid username and password entered.");
            return (makeLogin(s));
        }
    } catch (NullPointerException e) {
        //요청 파라미터에 PASSWORD가 존재하지 않을 경우 Null Pointer Exception이 발생하고 해당
        //오류에 대한 대응이 존재하지 않아 인증이 된 것으로 처리
    }
}
```

- 예외를 포착(catch)한 후, 각각의 예외 사항(Exception)에 대하여 적절하게 처리해야 한다.

안전한 코드의 예

```
protected Element createContent(WebSession s) {
    .....
    try {
        username = s.getParser().getRawParameter(USERNAME);
        password = s.getParser().getRawParameter(PASSWORD);
        if (!"webgoat".equals(username) || !password.equals("webgoat")) {
            s.setMessage("Invalid username and password entered.");
            return (makeLogin(s));
        }
    } catch (NullPointerException e) {
        //예외 사항에 대해 적절한 조치를 수행하여야 한다.
        s.setMessage(e.getMessage());
        return (makeLogin(s));
    }
}
```

- 다음은 표준프레임워크 공통컴포넌트의 LDAP조직도관리에서의 예외 사항에 대해 적절한 조치를 수행한 예제¹⁸⁾이다.

안전한 코드의 예

```
public List<Object> selectUserManageList(String dn) {
    List<Object> ucorgList = null;
    String filter = "objectclass=user";

    try {
        ucorgList = ldapTemplate.search(dn, filter, SearchControls.ONELEVEL_SCOPE, new
        ObjectMapper<UserVO>(
            UserVO.class));
    } catch (NameNotFoundException e) {
        logger.error("[NameNotFoundException] : search fail");
    }

    return ucorgList;
}
```

라. 참고자료

- CWE-390 Detection of Error Condition Without Action, MITRE, <https://cwe.mitre.org/data/definitions/390.html>
- Do not suppress or ignore checked exceptions, CERT, <https://wiki.sei.cmu.edu/confluence/display/java/ERR00-J.+Do+not+suppress+or+ignore+checked+exceptions>

18) <https://github.com/eGovFramework/egovframe-common-components/blob/master/src/main/java/egovframework/com/ext/ldapumt/service/impl/UserManageLdapDAO.java> 참조

3. 부적절한 예외 처리

가. 개요

프로그램 수행 중에 함수의 결과값에 대한 적절한 처리 또는 예외 상황에 대한 조건을 적절하게 검사 하지 않을 경우, 예기치 않은 문제를 야기할 수 있다.

나. 보안대책

값을 반환하는 모든 함수의 결과값을 검사하여, 그 값이 의도했던 값인지 검사하고, 예외 처리를 사용 하는 경우에 광범위한 예외 처리 대신 구체적인 예외 처리를 수행한다.

다. 코드예제

- 다음 예제는 try 블록에서 다양한 예외가 발생할 수 있음에도 불구하고 예외를 세분화하지 않고 광범위한 예외 클래스인 Exception을 사용하여 예외를 처리하고 있다.

안전하지 않은 코드의 예

```
try {  
    ...  
    reader = new BufferedReader(new InputStreamReader(url.openStream()));  
    String line = reader.readLine();  
    SimpleDateFormat format = new SimpleDateFormat("MM/DD/YY");  
    Date date = format.parse(line);  
    //예외처리를 세분화 할 수 있음에도 광범위하게 사용하여 예기치 않은 문제가 발생 할 수 있다.  
} catch (Exception e) {  
    System.err.println("Exception : " + e.getMessage());  
}
```

- 발생 가능한 예외를 세분화하고 발생 가능한 순서에 따라 예외를 처리하고 있다.

안전한 코드의 예

```
try {  
    ...  
    reader = new BufferedReader(new InputStreamReader(url.openStream()));  
    String line = reader.readLine();  
    SimpleDateFormat format = new SimpleDateFormat("MM/DD/YY");  
    Date date = format.parse(line);  
    // 발생할 수 있는 오류의 종류와 순서에 맞춰서 예외 처리 한다. getMessage() 사용에 대해서는
```

제6절 캡슐화 3.시스템 데이터 정보노출을 참조 한다.

```
} catch (MalformedURLException e) {  
    System.err.println("MalformedURLException : " + e.getMessage());  
}  
} catch (IOException e) {  
    System.err.println("IOException : " + e.getMessage());  
}  
} catch (ParseException e) {  
    System.err.println("ParseException : " + e.getMessage());  
}  
}
```

라. 참고자료

- CWE-754 Improper Check for Unusual or Exceptional Conditions, MITRE
<https://cwe.mitre.org/data/definitions/754.html>
- Do not complete abruptly from a finally block, CERT,
<https://wiki.sei.cmu.edu/confluence/display/java/ERR04-J.+Do+not+complete+abruptly+from+a+finally+block>
- Exception Handling in Spring MVC, Spring,
<https://spring.io/blog/2013/11/01/exception-handling-in-spring-mvc>

제5절 코드 오류

타입 변환 오류, 자원(메모리 등)의 부적절한 반환 등과 같이 개발자가 범할 수 있는 코딩오류로 인해 유발되는 보안약점이다.

1. Null Pointer 역참조

가. 개요

널 포인터(Null Pointer) 역참조는 '일반적으로 그 객체가 널(Null)이 될 수 없다'라고 하는 가정을 위반했을 때 발생한다. 공격자가 의도적으로 널 포인터 역참조를 발생시키는 경우, 그 결과 발생하는 예외 상황을 이용하여 추후의 공격을 계획하는 데 사용될 수 있다.

나. 보안대책

널이 될 수 있는 레퍼런스(Reference)는 참조하기 전에 널 값인지를 검사하여 안전한 경우에만 사용 한다.

다. 코드예제

- 다음의 예제의 경우 obj가 null이고, elt가 null이 아닌 경우 널(Null) 포인터 역참조가 발생한다.

안전하지 않은 코드의 예

```
public static int cardinality (Object obj, final Collection col) {  
    int count = 0;  
    if (col == null) {  
        return count;  
    }  
    Iterator it = col.iterator();  
    while (it.hasNext()) {  
        Object elt = it.next();  
        //obj가 null이고 elt가 null이 아닐 경우, Null.equals 가 되어 널(Null) 포인터 역참조가 발생한다.  
        if ((null == obj && null == elt) || obj.equals(elt)) {  
            count++;  
        }  
    }  
    return count;  
}
```

- obj가 null인지 검사 후 참조해야 한다.

안전한 코드의 예

```
public static int cardinality (Object obj, final Collection col) {
    int count = 0;
    if (col == null) {
        return count;
    }
    Iterator it = col.iterator();
    while (it.hasNext()) {
        Object elt = it.next();
        //obj가 null인지 검사 후 비교한다.
        if ((null == obj && null == elt) || (null != obj && obj.equals(elt))) {
            count++;
        }
    }
    return count;
}
```

- 다음예제의 경우 request.getParameter에 의해 null이 들어오게 되면 널(Null) 포인터 역참조가 발생한다.

안전하지 않은 코드의 예

```
String url = request.getParameter("url");
//url 에 null이 들어오면 널(Null) 포인터 역참조가 발생한다.
if ( url.equals("") )
```

- null을 가질 수 있는 참조 변수를 사용해 객체의 속성이나 메소드를 사용하는 경우 null 검사를 수행 하고 사용한다.

안전한 코드의 예

```
String url = request.getParameter("url");
//null값을 가지는 참조 변수를 사용할 경우, null 검사를 수행하고 사용한다.
if ( url == null || url.trim().equals("") )
```

라. 참고자료

- CWE-476 NULL Pointer Dereference, MITRE,
<https://cwe.mitre.org/data/definitions/476.html>
- Do not dereference null pointers, CERT,
<https://wiki.sei.cmu.edu/confluence/display/c/EXP34-C.+D>

o+not+dereference+null+pointers

- Null Dereference, OWASP,
https://owasp.org/www-community/vulnerabilities/Null_Dereference

2. 부적절한 자원 해제

가. 개요

프로그램의 자원, 예를 들면 열린 파일디스크립터(Open File Descriptor), 힙 메모리(Heap Memory), 소켓(Socket) 등은 유한한 자원이다. 이러한 자원을 할당받아 사용한 후, 더 이상 사용하지 않는 경우에는 적절히 반환하여야 하는데, 프로그램 오류 또는 예외로 사용이 끝난 자원을 반환하지 못하는 경우이다.

나. 보안대책

자원을 획득하여 사용한 다음에는 반드시 자원을 해제하여 반환한다.

다. 코드예제

- try구문 내 처리 중 오류가 발생할 경우, close()메서드가 실행되지 않아 사용한 자원이 반환되지 않을 수 있다.

안전하지 않은 코드의 예

```
InputStream in = null;
OutputStream out = null;
try {
    in = new FileInputStream(inputFile);
    out = new FileOutputStream(outputFile);
    ...
    FileCopyUtils.copy(fis, os);
    //자원반환 실행 전에 오류가 발생할 경우 자원이 반환되지 않으며, 할당된 모든 자원을 반환해야 한다.
    in.close();
    out.close();
} catch (IOException e) {
    logger.error(e);
}
```

- 예외상황이 발생하여 함수가 종료될 때, 예외의 발생 여부와 상관없이 항상 실행되는 finally 블록에서 할당받은 모든 자원을 반드시 반환하도록 한다.

안전한 코드의 예

```

InputStream in = null;
OutputStream out = null;
try {
    in = new FileInputStream(inputFile);
    out = new FileOutputStream(outputFile);
    ...
    FileCopyUtils.copy(fis, os);
} catch (IOException e) {
    logger.error(e);
    //항상 수행되는 finally 블록에서 할당받은 모든 자원에 대해 각각 null검사를 수행 후 예외처리를
    하여 자원을 해제하여야 한다.
} finally {
    if (in != null) {
        try {
            in.close();
        } catch (IOException e) {
            logger.error(e);
        }
    }
    if (out != null) {
        try {
            out.close();
        } catch (IOException e) {
            logger.error(e);
        }
    }
}
}

```

라. 참고자료

- CWE-404 Improper Resource Shutdown or Release, MITRE,
<https://cwe.mitre.org/data/definitions/404.html>
- Release resources when they are no longer needed, CERT,
<https://wiki.sei.cmu.edu/confluence/display/java/FIO04-J.+Release+resources+when+they+are+no+longer+needed>
- Unreleased Resource, OWASP,
https://owasp.org/www-community/vulnerabilities/Unreleased_Resource

3. 신뢰할 수 없는 데이터의 역직렬화

가. 개요

직렬화(Serialization)는 프로그램에서 특정 클래스의 현재 인스턴스 상태를 다른 서버로 전달하기 위해 클래스의 인스턴스 정보를 바이트 스트림으로 복사하는 작업으로, 메모리 상에서 실행되고 있는 객체의 상태를 그대로 복제하여 파일로 저장하거나 수신 측에 전달하게 된다.

역직렬화(Deserialization)는 반대 연산으로 바이너리 파일이나 바이트 스트림으로부터 객체 구조로 복원하게 된다.

이 때, 송신자가 네트워크를 이용하여 직렬화된 정보를 수신자에게 전달하는 과정에서 공격자가 전송 또는 저장된 스트림을 조작할 수 있는 경우에는 신뢰할 수 없는 역직렬화를 이용하여 무결성 침해, 원격 코드 실행, 서비스 거부 공격 등이 발생할 수 있는 보안약점이다.

나. 보안대책

초기화되지 않은 스택 메모리 영역의 변수는 임의값이라 생각해서 대수롭지 않게 생각 할 수 있으나, 사실은 이전 함수에서 사용되었던 내용을 포함하고 있다. 모든 변수를 사용 전에 반드시 올바른 초기값을 할당함으로서 이러한 문제를 예방한다.

신뢰할 수 없는 데이터를 역직렬화 하지 않도록 응용프로그램을 구성한다. 민감정보 또는 중요정보를 전송 시 암호화 통신을 적용하지 못하는 경우, 송신 측에서 서명을 추가하고 수신 측에서 서명을 확인하여 데이터의 무결성을 검증한다.

또는, 신뢰할 수 있는 데이터의 식별을 위해 역직렬화 대상의 데이터가 사전에 검증된 클래스만을 포함하는지 검증하거나, 제한된 실행 권한을 구성하여 역직렬화 코드를 실행한다.

다. 코드예제

- 다음 예제는 맵(map)을 직렬화하고 역직렬화 하는 코드이다. 데이터를 전송할 경우 공격자가 바이트스트림을 조작하여 역직렬화 공격이 가능한 개체를 생성할 수 있는 예제이다.

안전하지 않은 코드의 예

```
public static void main(String[] args) throws
IOException, GeneralSecurityException, ClassNotFoundException {
    ....
    // map을 역직렬화 한다.
    ObjectInputStream in = new ObjectInputStream(new FileInputStream("data"));
    sealedMap = (SealedObject) in.readObject();
    in.close();

    // 객체를 추출한다.
    cipher = Cipher.getInstance("AES");
    cipher.init(Cipher.DECRYPT_MODE, key);
    signedMap = (SignedObject) sealedMap.getObject(cipher);

    map = (SerializableMap<String, Integer>) signedMap.getObject();
}
```

- 다음 예제는 서명 값을 검증하여 메시지 위변조를 방지할 수 있는 코드이다.

안전한 코드의 예

```
public static void main(String[] args) throws
IOException, GeneralSecurityException, ClassNotFoundException {
    ....
    // map을 역직렬화 한다.
    ObjectInputStream in = new ObjectInputStream(new FileInputStream("data"));
    sealedMap = (SealedObject) in.readObject();
    in.close();

    // 객체를 추출한다.
    cipher = Cipher.getInstance("AES");
    cipher.init(Cipher.DECRYPT_MODE, key);
    signedMap = (SignedObject) sealedMap.getObject(cipher);

    // 서명값 검증 과정에서 불일치 시 예외를 리턴하고, 일치 시 map 값을 읽는다.
    if (!signedMap.verify(kp.getPublic(), sig)) {
        throw new GeneralSecurityException("Map failed verification");
    }
    map = (SerializableMap<String, Integer>) signedMap.getObejct();
}
```

- 다음 예제는 바이트 배열의 입력 값을 검증 없이 readObject()

로 역직렬화하여 악의적인 코드가 실행될 수 있다.

안전하지 않은 코드의 예

```
class DeserializeExample {
    public static Object deserialize(byte[] buffer)
        throws IOException, ClassNotFoundException {
        Object ret = null;
        try (ByteArrayInputStream bais = new ByteArrayInputStream(buffer)) {
            try (ObjectInputStream ois = new ObjectInputStream(bais)) {
                ret = ois.readObject();
            }
        }
        return ret;
    }
}
```

- 이를 안전한 코드로 변환하기 위해서는 ObjectInputStream을 상속 받아 WhiteListedObjectInputStream 객체를 구현하여 사용한다. WhiteListedObjectInputStream에서는 readObject()를 실행 시, resolveClass 함수를 호출하여 설정한 화이트리스트와 비교하여 리스트에 없는 데이터일 경우 예외를 발생시킨다.

안전한 코드의 예

```
public class WhitelistedObjectInputStream extends ObjectInputStream {
    public Set<String> whitelist;
    // WhitelistedObjectInputStream을 생성할 때 화이트리스트를 입력받는다.
    public WhitelistedObjectInputStream(InputStream inputStream, Set<String> wl)
        throws IOException {
        super(inputStream); whitelist = wl;
    }

    @Override
    protected Class<?> resolveClass(ObjectStreamClass cls) throws IOException,
        ClassNotFoundException {
        // ObjectStreamClass의 클래스명이 화이트리스트에 있는지 확인한다.
        if (!whitelist.contains(cls.getName())) {
            throw new InvalidClassException("Unexpected serialized class", cls.getName());
        }
        return super.resolveClass(cls);
    }
}

@RequestMapping(value = "/upload", method = RequestMethod.POST)
public Student upload(@RequestParam("file") MultipartFile multipartFile) throws
    ClassNotFoundException, IOException {
    Student student = null;
    File targetFile = new File("/temp/" + multipartFile.getOriginalFilename());
}
```



```
// 역직렬화 대상 클래스 이름의 화이트리스트 생성한다.
Set<String> whitelist = new HashSet<String>(Arrays.asList(
    new String[] {
        "Student"
    }));
try (InputStream fileStream = multipartFile.getInputStream()) {
    try (WhitelistedObjectInputStream ois = new WhitelistedObjectInputStream(fileStream,
whitelist)) {
        // 화이트리스트에 없는 역직렬화 데이터의 경우 예외 발생시킨다.
        student = (Student) ois.readObject();
    }
}
return student;
}
```

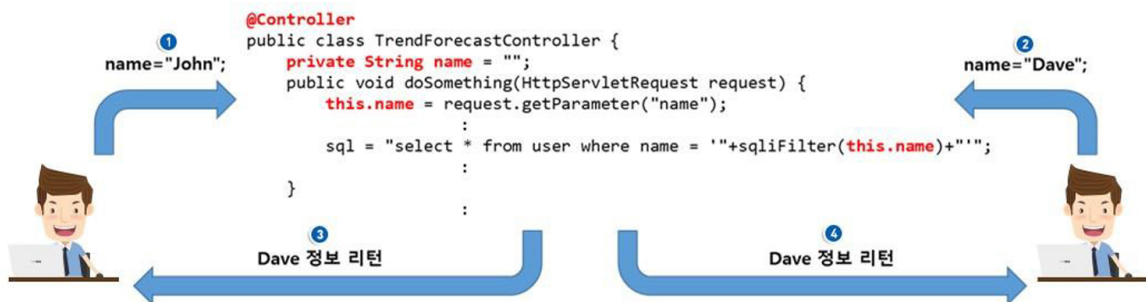
라. 참고자료

- CWE-502 Deserialization of Untrusted Data, MITRE,
<https://cwe.mitre.org/data/definitions/502.html>

제6절 캡슐화

중요한 데이터 또는 기능을 불충분하게 캡슐화하거나 잘못 사용함으로써 발생하는 보안약점으로 정보노출, 권한문제 등이 발생할 수 있다.

1. 잘못된 세션에 의한 데이터 정보노출



가. 개요

다중 스레드 환경에서는 싱글톤(Singleton)¹⁹⁾ 객체 필드에 경쟁조건(Race Condition)이 발생할 수 있다. 따라서, 다중 스레드 환경인 Java의 서블릿(Servlet) 등에서는 정보를 저장하는 멤버변수가 포함되지 않도록 하여, 서로 다른 세션에서 데이터를 공유하지 않도록 해야 한다.

나. 보안대책

싱글톤 패턴을 사용하는 경우, 변수 범위(Scope)에 주의를 기울여야 한다. 특히 Java에서는 HttpServlet 클래스의 하위 클래스에서 멤버 필드를 선언하지 않도록 하고, 필요한 경우 지역 변수를 선언하여 사용한다.

다. 코드예제

- JSP 선언부(<%! 소스코드 %>)에 선언한 변수는 해당 JSP에 접근하는 모든 사용자에게 공유된다. 먼저 호출한 사용자가

19) 싱글톤 패턴 : GOF 32가지 패턴 중 하나. 하나의 프로그램 내에서 하나의 인스턴스만을 생성해야만 하는 패턴. Connection Pool, Thread Pool과 같이 Pool 형태로 관리되는 클래스의 경우 프로그램 내에서 단하나의 인스턴트로 관리해야 하는 경우를 말함. java에서는 객체로 제공됨

값을 설정하고 사용하기 전에 다른 사용자의 호출이 발생하게 되면, 뒤에 호출 한 사용자가 설정한 값이 모든 사용자에게 적용되게 된다.

안전하지 않은 코드의 예
<pre> <%@page import="javax.xml.namespace.*"%> <%@page import="gov.mogaha.ntis.web.frs.gis.cmm.util.*" %> <%! // JSP에서 String 필드들이 멤버 변수로 선언됨 String username = "/"; String imagePath = commonPath + "img/"; String imagePath_gis = imagePath + "gis/cmm/btn/"; %> </pre>

- JSP의 서블릿(<% 소스코드 %>)에 정의한 변수는 _jspService 메소드의 지역변수로 선언되므로 공유가 발생하지 않아 안전하다.

안전한 코드의 예
<pre> <%@page import="javax.xml.namespace.*"%> <%@page import="gov.mogaha.ntis.web.frs.gis.cmm.util.*" %> <% // JSP에서 String 필드들이 로컬 변수로 선언됨 String commonPath = "/"; String imagePath = commonPath + "img/"; String imagePath_gis = imagePath + "gis/cmm/btn/"; %> </pre>

- Controller에 멤버 변수를 사용하면 공유가 발생하여 동기화 오류가 발생할 수 있다.

안전하지 않은 코드의 예
<pre> @Controller public class TrendForecastController { // Controller에서 int 필드가 멤버 변수로 선언되어 스레드간에 공유됨 private int currentPage = 1; public void doSomething(HttpServletRequest request) { currentPage = Integer.parseInt(request.getParameter("page")); } </pre>

- Controller에 멤버 변수를 사용하지 않고 지역 변수로 사용한다.

안전한 코드의 예

```
@Controller
public class TrendForecastController {
    public void doSomething(HttpServletRequest request) {
        // 지역변수로 사용하여 스레드간 공유되지 못하도록 한다.
        int currentPage = Integer.parseInt(request.getParameter("page"));
    }
    .....
```

라. 참고자료

- CWE-488 Exposure of Data Element to Wrong Session, MITRE,
<https://cwe.mitre.org/data/definitions/488.html>
- CWE-543 Use of Singleton Pattern Without Synchronization in a Multithreaded Context, MITRE,
<https://cwe.mitre.org/data/definitions/543.html>
- Do not let session information leak within a servlet, CERT,
<https://wiki.sei.cmu.edu/confluence/display/java/MS11-J.+Do+not+let+session+information+leak+within+a+servlet>

2. 제거되지 않고 남은 디버그 코드

가. 개요

디버깅 목적으로 삽입된 코드는 개발이 완료되면 제거해야 한다. 디버그 코드는 설정 등의 민감한 정보를 담거나 시스템을 제어하게 허용하는 부분을 담고 있을 수 있다. 만일, 남겨진 채로 배포될 경우, 공격자가 식별 과정을 우회하거나 의도하지 않은 정보와 제어 정보가 노출될 수 있다.

나. 보안대책

소프트웨어 배포 전, 반드시 디버그 코드를 확인 및 삭제한다. 일반적으로 Java 개발자의 경우 웹 응용프로그램을 제작할 때 디버그용도의 코드를 main()에 개발한 후 이를 삭제하지 않는 경우가 많다. 디버깅이 끝나면 main() 메소드를 삭제해야 한다.

다. 코드예제

- 다음의 예제는 main() 메소드 내에 화면에 출력하는 디버깅 코드를 포함하고 있다. MVC 패턴으로 개발하여 WAS에서 실행되는 웹어플리케이션은 main() 메소드 사용이 필요 없으며, 개발자들이 콘솔 응용프로그램으로 화면에 디버깅코드를 사용하는 경우가 일반적이다.

안전하지 않은 코드의 예

```
class Base64 {  
    public static void main(String[] args) {  
        if (debug) {  
            byte[] a = { (byte) 0xfc, (byte) 0x0f, (byte) 0xc0 };  
            byte[] b = { (byte) 0x03, (byte) 0xf0, (byte) 0x3f };  
            .....  
        }  
    }  
    public void otherMethod() { ... }  
}
```

- 이에 따라 WAS로 실행되는 웹어플리케이션에서 main() 메

소드는 삭제한다. main() 메소드의 경우 디버깅 코드인 경우가 일반적이다.

안전한 코드의 예

```
class Base64 {  
    public void otherMethod() { ... }  
}
```

라. 참고자료

- CWE-489 Leftover Debug Code, MITRE,
<https://cwe.mitre.org/data/definitions/489.html>
- Production code must not contain debugging entry points, CERT,
<https://wiki.sei.cmu.edu/confluence/display/java/ENV06-J.+Production+code+must+not+contain+debugging+entry+points>

3. Public 메소드부터 반환된 Private 배열

가. 개요

private로 선언된 배열을 public으로 선언된 메소드를 통해 반환(return)하면, 그 배열의 레퍼런스가 외부에 공개되어 외부에서 배열수정과 객체 속성변경이 가능해진다.

나. 보안대책

private로 선언된 배열을 public으로 선언된 메소드를 통해 반환하지 않도록 해야 한다. private 배열에 대한 복사본을 반환하도록 하고 배열의 원소에 대해서는 clone() 메소드를 통해 복사된 원소를 저장하도록 하여 private 선언된 배열과 객체속성에 대한 의도하지 않게 수정되는 것을 방지 한다. 만약 배열의 원소가 String 타입 등과 같이 변경이 되지 않는 경우에는 private 배열의 복사본 을 만들고 이를 반환하도록 작성한다.

다. 코드예제

- 멤버 변수 colors는 private로 선언되었지만 public으로 선언된 getColors() 메소드를 통해 참조를 얻을 수 있다. 이를 통해 의도하지 않은 수정이 발생할 수 있다.
- 아래의 코드는 멤버 변수 colors는 private로 선언되었지만 public으로 선언된 getUserColors 메소드를 통해 private 배열에 대한 reference를 얻을 수 있다. 이를 통해 의도하지 않은 수정이 발생할 수 있다.

안전하지 않은 코드의 예

```
// private 인 배열을 public인 메소드가 return한다.  
private Color[] colors;  
public Color[] getUserColors(Color[] userColors) { return colors; }
```

- private배열에 대한 복사본을 만들고, 복사된 배열의 원소로는 clone() 메소드를 통해 private 배열의 원소의 복사본을 만들어 저장하여 반환하도록 작성하면, private선언된 배열과 원소에 대한 의도하지 않은 수정을 방지 할 수 있다.

안전한 코드의 예

```
private Color[] colors;
//메소드를 private으로 하거나, 복제본 반환, 수정하는 public 메소드를 별도로 만든다.
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    Color[] newColors = getUserColors();
    .....
}

public Color[] getUserColors(Color[] userColors) {
    //배열을 복사한다.
    Color[] colors = new Color [userColors.length];
    for (int i = 0; i < colors.length; i++)
        //clone()메소드를 이용하여 배열의 원소도 복사한다.
        colors[i] = this.colors[i].clone();
    return colors;
}
```

- 아래의 코드는 멤버 변수 colors는 private로 선언되었지만, public으로 선언된 getColors() 메소드를 통해 reference를 얻을 수 있다. 이를 통해 의도하지 않은 배열의 수정이 발생할 수 있다.

안전하지 않은 코드의 예

```
// private 인 배열을 public인 메소드가 return한다.
private String[] colors;
public String[] getColors() { return colors; }
```

- private배열의 복사본을 만들고, 이를 반환하도록 작성하면, private 선언된 배열에 대한 의도하지 않은 수정을 방지 할 수 있다.

안전한 코드의 예

```
private String[] colors;
// 메소드를 private으로 하거나, 복제본 반환, 수정하는 public 메소드를 별도로 만든다.
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    String[] newColors = getColors();
}
```



```

.....
}

public String[] getColors() {
    String[] ret = null;
    if ( this.colors != null ) {
        ret = new String[colors.length];
        for (int i = 0; i < colors.length; i++) { ret[i] = this.colors[i]; }
    }
    return ret;
}

```

라. 참고자료

- CWE-495 Private Array-Typed Field Returned From A Public Method, MITRE,
<https://cwe.mitre.org/data/definitions/495.html>
- Do not return references to private mutable class members, CERT,
<https://wiki.sei.cmu.edu/confluence/display/java/OBJ05-J.+Do+not+return+references+to+private+mutable+class+members>

4. Private 배열에 Public 데이터 할당

가. 개요

public으로 선언된 메소드의 인자가 private선언된 배열에 저장되면, private배열을 외부에서 접근하여 배열수정과 객체 속성변경이 가능해진다.

나. 보안대책

public으로 선언된 메서드의 인자를 private선언된 배열로 저장되지 않도록 한다. 인자로 들어온 배열의 복사본을 생성하고 clone() 메소드를 통해 복사된 원소를 저장하도록 하여 private변수에 할당하여 private선언된 배열과 객체속성에 대한 의도하지 않게 수정되는 것을 방지한다. 만약 배열 객체의 원소가 String 타입 등과 같이 변경이 되지 않는 경우에는 인자로 들어온 배열의 복사본을 생성하여 할당한다.

다. 코드예제

- 아래의 코드는 멤버 변수 userRoles는 private로 선언되었지만 public으로 선언된 setUserRoles 메소드를 통해 인자가 할당되어 배열의 원소를 외부에서 변경할 수 있다. 이를 통해 의도하지 않은 배열과 원소에 대한 객체속성 수정이 발생할 수 있다.

안전하지 않은 코드의 예

```
//userRoles 필드는 private이지만, public인 setUserRoles()를 통해 외부의 배열이 할당되면,
//사실상 public 필드가 된다.
private UserRole[] userRoles;
public void setUserRoles(UserRole[] userRoles) {
    this.userRoles = userRoles;
}
```

- 인자로 들어온 배열의 복사본을 생성하고 clone() 메소드를 통해 복사된 원소를 저장하도록 하여 private변수에 할당하면 private으로 할당된 배열과 원소에 대한 의도하지 않은

수정을 방지 할 수 있다.

안전한 코드의 예

```
//객체가 클래스의 private member를 수정하지 않도록 한다.  
private UserRole[] userRoles;  
public void setUserRoles(UserRole[] userRoles) {  
    this.userRoles = new UserRole[userRoles.length];  
    for (int i = 0; i < userRoles.length; ++i)  
        this.userRoles[i] = userRoles[i].clone();  
}
```

라. 참고자료

- CWE-496 Public Data Assigned to Private Array-Typed Field, MITRE,

<https://cwe.mitre.org/data/definitions/496.html>

제7절 API 오용

의도된 사용에 반하는 방법으로 API를 사용하거나, 보안에 취약한 API를 사용하여 발생할 수 있는 보안약점이다.

1. DNS lookup에 의존한 보안결정

가. 개요

공격자가 DNS 엔트리를 속일 수 있으므로 도메인명에 의존에서 보안결정(인증 및 접근 통제 등)을 하지 않아야 한다. 만약, 로컬 DNS 서버의 캐시가 공격자에 의해 오염된 상황이라면, 사용자와 특정 서버간의 네트워크 트래픽이 공격자를 경유하도록 할 수도 있다. 또한, 공격자가 마치 동일 도메인에 속한 서버인 것처럼 위장할 수도 있다.

나. 보안대책

보안결정에서 도메인명을 이용한 DNS lookup을 하지 않도록 한다.

다. 코드예제

- 다음의 예제는 도메인명을 통해 해당 요청을 신뢰할 수 있는지를 검사한다. 그러나 공격자는 DNS 캐쉬 등을 조작해서 쉽게 이러한 보안 설정을 우회할 수 있다.

안전하지 않은 코드의 예

```
public void doGet(HttpServletRequest req, HttpServletResponse res)
throws ServletException, IOException {
    boolean trusted = false;
    String ip = req.getRemoteAddr();
    InetAddress addr = InetAddress.getByName(ip);
    //도메인은 공격자에 의해 실행되는 서버의 DNS가 변경될 수 있으므로 안전하지 않다.
    if (addr.getCanonicalHostName().endsWith("trustme.com")) {
        do_something_for_Trust_System();
    }
}
```

- 그러므로, 다음의 예제와 같이 DNS lookup에 의한 호스트

이름 비교를 하지 않고, IP 주소를 직접 비교하도록 수정한다.

안전한 코드의 예

```
public void doGet(HttpServletRequest req, HttpServletResponse res)
throws ServletException, IOException {
    String ip = req.getRemoteAddr();
    if (ip == null || "".equals(ip)) return ;
    //이용하려는 실제 서버의 IP 주소를 사용하여 DNS변조에 방어한다.
    String trustedAddr = "127.0.0.1";
    if (ip.equals(trustedAddr)) {
        do_something_for_Trust_System();
    }
}
```

라. 참고자료

- CWE-247 Reliance on DNS Lookups in a Security Decision, MITRE,
<https://cwe.mitre.org/data/definitions/247.html>

2. 취약한 API 사용

가. 개요

취약한 API는 더 이상 사용되지 않고 앞으로 사라지게 될 classes 또는 methods를 의미한다. 이들 범주의 API에 대해 확인하지 않고 사용할 때 보안 문제를 발생시킬 수 있다. 또한 보안상 문제가 없다 하더라도 잘못된 방식으로 함수를 사용할 때도 역시 보안 문제를 발생시킬 수 있다.

나. 보안대책

- 보안 문제로 인해 금지된 함수는 이를 대체할 수 있는 안전한 함수를 사용한다.
- 취약한 API의 분류는 일반적인 것은 아니지만 개발 조직에 따라 이를 명시한 경우가 있다면 반드시 준수한다.

다. 코드예제

- 다음 예제는 응용 프로그램에서 프레임워크 메소드 호출 대신에 소켓을 직접 사용하고 있어, 프레임워크에서 제공하는 보안기능을 제공 받지 못한다.

안전하지 않은 코드의 예

```
public class S246 extends javax.servlet.http.HttpServlet {
    private Socket socket
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response) throws ServletException {
        try {
            //프레임워크의 메소드 호출 대신 소켓을 직접사용하고 있어 프레임워크에서 제공하는
            //보안기능을 제공받지 못해 안전하지 않다.
            socket = new Socket("kisa.or.kr", 8080);
        } catch (UnknownHostException e) {
            .....
        }
    }
}
```

- 타겟이 WAS로 작성될 경우 아래의 코드처럼 보안기능을 제공하는 프레임워크 메소드인 URL Connection 을 이용하여야한다.

안전한 코드의 예

```

public class S246 extends javax.servlet.http.HttpServlet {
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response) throws ServletException {
        ObjectOutputStream oos = null;
        ObjectInputStream ois = null;
        try {
            URL url = new URL("http://127.0.0.1:8080/DataServlet");
            //보안기능을 제공하는 프레임워크의 메소드를 사용해야한다.
            URLConnection urlConn = url.openConnection();
            urlConn.setDoOutput(true);
            .....
        }
    }
}

```

라. 참고자료

- CWE-676 Use of Potentially Dangerous Function, MITRE,
<https://cwe.mitre.org/data/definitions/676.html>
- CWE-242 Use of Inherently Dangerous Function, MITRE,
<https://cwe.mitre.org/data/definitions/242.html>
- CWE-246 J2EE Bad Practices: Direct Use of Sockets,
 MITRE, <https://cwe.mitre.org/data/definitions/246.html>
- CWE-382 J2EE Bad Practices: Use of System.exit(), MITRE,
<https://cwe.mitre.org/data/definitions/382.html>
- Do not use deprecated or obsolete classes or methods,
 CERT,
[https://wiki.sei.cmu.edu/confluence/display/java/MET02-J.
 +Do+not+use+deprecated+or+obsolete+classes+or+methods](https://wiki.sei.cmu.edu/confluence/display/java/MET02-J.+Do+not+use+deprecated+or+obsolete+classes+or+methods)

붙임 1

표준프레임워크 예제 포함된 보안 항목

보안 유형	보안 항목	표준프레임워크 예제 포함	위키가이드 참조 추가	비고
입력 데이터 검증 및 표현	SQL Injection			
	코드 삽입			
	경로 조작 및 자원 삽입	●		2018년 공통컴포넌트 취약점 패치 내용
	XSS (Cross Site Scripting)		●	
	운영체제 명령어 삽입	●		2023년 공통컴포넌트 취약점 패치 내용
	위험한 형식 파일 업로드			
	신뢰되지 않는 URL 주소로 자동접속 연결	●		2019년 공통컴포넌트 보안 패치 (화이트리스트)
	부적절한 XML 외부개체 참조			
	XML 삽입			
	LDAP 삽입			
	크로스사이트 요청 위조	●	●	Spring Security 설정 간소화 서비스
	서버사이드 요청 위조			
	HTTP 응답분할			
	정수형 오버플로우			
	보안기능 결정에 사용되는 부적절한 입력값			
보안기능	포맷 스트링 삽입			
	적절한 인증 없는 중요기능 허용			
	부적절한 인가			
	중요한 자원에 대한 잘못된 권한 설정			
	취약한 암호화 알고리즘 사용	●	●	2018년 공통컴포넌트 취약점 패치 내용
	암호화되지 않은 중요정보			
	하드코딩된 중요정보	●	●	Crypto 간소화 서비스
	충분하지 않은 키 길이 사용			
	적절하지 않은 난수값 사용	●		
	취약한 비밀번호 허용			
	부적절한 전자서명 확인			
	부적절한 인증서 유효성 검증			
	사용자 하드디스크에 저장되는 쿠키를 통한 정보노출	●		
	주석문 안에 포함된 시스템 주요정보			
	솔트없이 일방향 해쉬함수 사용	●		
	무결성 검사 없는 코드 다운로드			
	반복된 인증시도 제한 기능 부재			
시간 및	경쟁조건: 검사시점과 사용시점	●		

보안 유형	보안 항목	표준프레임워크 예제 포함	위키가이드 참조 추가	비고
상태	종료되지 않는 반복문 또는 재귀함수			
에러 처리	오류 메시지를 통한 정보노출			
	오류 상황 대응 부재	●		
	부적절한 예외 처리			
코드 오류	Null Pointer 역참조			
	부적절한 자원 해제			
	신뢰할 수 없는 데이터의 역직렬화			
캡슐화	잘못된 세션에 의한 데이터 정보노출			
	제거 되지 않고 남은 디버그 코드			
	Public 메소드로부터 반환된 Private 배열			
	Private 배열에 Public 데이터 할당			
API 오용	DNS lookup에 의존한 보안결정			
	취약한 API 사용			

붙임 2

개발환경의 보안 항목별 관련 툴 보유 여부

보안 유형	보안 항목	Spotbugs	FindSecurityBugs ²⁰⁾	PMD
입력 데이터 검증 및 표현	SQL Injection	●	●	
	코드 삽입		●	
	경로 조작 및 자원 삽입	●	●	
	XSS (Cross Site Scripting)	●	●	
	운영체제 명령어 삽입		●	
	위험한 형식 파일 업로드		●	
	신뢰되지 않는 URL 주소로 자동접속 연결		●	
	부적절한 XML 외부개체 참조		●	
	XML 삽입		●	
	LDAP 삽입		●	
	크로스사이트 요청 위조		●	
	서버사이드 요청 위조		●	
	HTTP 응답분할	●	●	
	정수형 오버플로우	●		
	보안기능 결정에 사용되는 부적절한 입력값		●	
	포맷 스트링 삽입		●	
보안기능	적절한 인증 없는 중요기능 허용		●	
	부적절한 인가		●	
	중요한 자원에 대한 잘못된 권한 설정		●	
	취약한 암호화 알고리즘 사용		●	
	암호화되지 않은 중요정보		●	
	하드코딩된 중요정보		●	●
	충분하지 않은 키 길이 사용		●	
	적절하지 않은 난수값 사용	●	●	
	취약한 비밀번호 허용			
	부적절한 전자서명 확인			
	부적절한 인증서 유효성 검증	●	●	
	사용자 하드디스크에 저장되는 쿠키를 통한 정보노출		●	
	주석문 안에 포함된 시스템 주요정보			
	솔트없이 일방향 해쉬함수 사용		●	
	무결성 검사 없는 코드 다운로드		●	
	반복된 인증시도 제한 기능 부재			
시간 및 상태	경쟁조건: 검사시점과 사용시점	●		
	종료되지 않는 반복문 또는 재귀함수	●		
에러 처리	오류 메시지를 통한 정보노출		●	●
	오류 상황 대응 부재			●
	부적절한 예외 처리	●		●
코드 오류	Null Pointer 역참조	●		●
	부적절한 자원 해제	●	●	●
	신뢰할 수 없는 데이터의 역직렬화		●	

보안 유형	보안 항목	Spotbugs	FindSecurityBugs ²⁰⁾	PMD
캡슐화	잘못된 세션에 의한 데이터 정보노출		●	
	제거 되지 않고 남은 디버그 코드			
	Public 메소드로부터 반환된 Private 배열	●		●
	Private 배열에 Public 데이터 할당	●		●
API 오용	DNS lookup에 의존한 보안결정			
	취약한 API 사용		●	

20) <https://www.egovframe.go.kr/wiki/doku.php?id=egovframework:dev4:findsecuritybugs>

붙임 3

표준프레임워크 4.2 PMD²¹⁾ 를 설명

PMD 룰	설명	SW보안약점
AbstractClassWithoutAbstractMethod	추상클래스 정의 오류	
ArraysStoredDirectly	배열객체 참조 외부 노출	Public 메소드부터 반환된 Private 배열
AssignmentInOperand	피연산자 내 할당문 사용	
AssignmentToNonFinalStatic	static 필드의 잘못된 사용	
AvoidArrayLoops	루프문을 이용한 배열복사	
AvoidCatchingGenericException	부적절한 예외처리	부적절한 예외처리
AvoidPrintStackTrace	오류 메시지를 통한 정보 노출 (시스템 데이터 정보 노출)	오류 메시지를 통한 정보 노출
AvoidReassigningParameters	parameter 값 재할당 시도	
AvoidSynchronizedAtMethodLevel	synchronization의 과다적용	
AvoidThrowingNullPointerException	NullPointerException 사용	
AvoidThrowingRawExceptionTypes	비가공 Exception 사용	
AvoidUsingHardCodedIP	하드코딩된 아이피 사용	하드코딩된 중요정보
BrokenNullCheck	잘못된 널(Null) 체크	널(Null) 포인터 역참조
CloseResource	부적절한 자원 해제	부적절한 자원 해제
ConstantsInterface	인터페이스에 상수 적용	
EmptyCatchBlock	빈 catch 문 사용, 오류 상황 대응 부재	빈 catch 문 사용, 오류 상황 대응 부재
EmptyControlStatement	빈 finally 블록 사용 빈 if 문 사용 빈 try 블록 사용 빈 while문 사용	
EqualsNull	equals()을 이용한 null 비교	
FieldNamingConventions	변수명에 밑줄 사용	
FinalFieldCouldBeStatic	final field의 static 전환	
FormalParameterNamingConventions	변수명에 밑줄 사용	
HardCodedCryptoKey	하드코딩된 암호화키 사용	하드코딩된 중요정보
ImmutableField	생성자 지정 변수의 final 미적용	
InefficientEmptyStringCheck	빈문자열 확인	
InefficientStringBuffering	StringBuffer 함수내 결합코드 사용	
LocalVariableNamingConventions	변수명에 잘못된 prefix 사용	
MethodReturnsInternalArray	클래스내 내부배열을 직접 반환하는 메소드	Private 배열에 Public 데이터 할당
MisplacedNullCheck	널(Null) 체크의 잘못된 위치	널(Null) 포인터 역참조
SimpleDateFormatNeedsLocale	지역코드없는 SimpleDateFormat 사용	
SimplifyBooleanExpressions	불필요 boolean 연산 시도	
StringInstantiation	불필요 String Instance 생성	
StringToString	불필요한 toString() 메소드 사용	
SwitchStmtsShouldHaveDefault	default 없는 switch 구문 사용	

21) <https://www.egovframe.go.kr/wiki/doku.php?id=egovframework:dev4.2:imp:inspection>

PMD 룰	설명	SW보안약점
SystemPrintln	System.out.print 사용	
UncommentedEmptyMethodBody	빈 메소드 주석표기	
UnnecessaryBoxing	불필요한 WrapperObject 생성	
UnnecessaryConversionTemporary	불필요한 String 변환작업	
UnnecessaryImport	불필요한 import문 선언	
UnnecessarySemicolon	필요없는 ; 문장 존재	
UnusedFormalParameter	미사용 method parameter	
UnusedPrivateField	미사용 private field	
UnusedPrivateMethod	미사용 private method	
UselessParentheses	불필요한 괄호사용	
UselessStringValueOf	불필요한 String.valueOf 사용	